

Melius - The Better Profile

DIPLOMARBEIT

verfasst im Rahmen der

Reife- und Diplomprüfung

an der

Höheren Abteilung für IT-Medientechnik

Eingereicht von:

Jakob Schlager

Josef Stieg

Betreuer:

Natascha Rammelmüller

Leonding, April 2025

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt bzw. die wörtlich oder sinngemäß entnommenen Stellen als solche kenntlich gemacht habe.

Die Arbeit wurde bisher in gleicher oder ähnlicher Weise keiner anderen Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

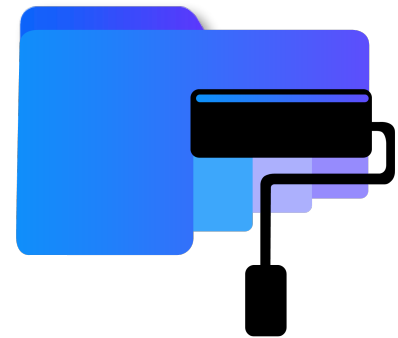
Die vorliegende Diplomarbeit ist mit dem elektronisch übermittelten Textdokument identisch.

Leonding, April 2025

J. Schlager & J. Stieg

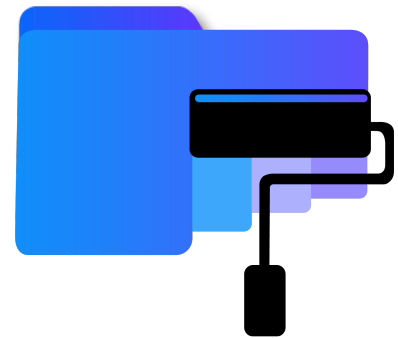
Abstract

Melius is a platform that helps users to present their personal and professional skills in an attractive way. With Melius, users can create portfolios that clearly present their projects, skills and experience. The platform also offers a group function that allows teams to organise and collaborate on projects. The focus is on customisation and user-friendliness to ensure an optimal presentation of individual strengths.



Zusammenfassung

Melius ist eine Plattform, die Nutzer:innen dabei unterstützt, ihre persönlichen und beruflichen Fähigkeiten ansprechend zu präsentieren. Mit Melius können Portfolios erstellt werden, die Projekte, Kompetenzen und Erfahrungen übersichtlich darstellen. Zusätzlich bietet die Plattform eine Gruppenfunktion, die es ermöglicht, Teams zu organisieren und gemeinsam an Projekten zu arbeiten. Individualisierung und Benutzerfreundlichkeit stehen im Vordergrund, um eine optimale Darstellung der eigenen Stärken zu ermöglichen.



Inhaltsverzeichnis

1 Einleitung - Josef Stieg

1.1 Beschreibung

1.1.1 Grundfunktion

Die innovative Online-Plattform *Melius* bietet Nutzer:innen die Möglichkeit, professionelle und personalisierte Portfolios zu erstellen, die insbesondere im Bewerbungsprozess von großem Nutzen sind. Die Intention der Plattform besteht in der Simplifikation und Effektivierung des Bewerbungsprozesses. Dies wird erreicht, indem alle relevanten Informationen und Materialien an einem zentralen Ort gesammelt und übersichtlich dargestellt werden.

Die Plattform bietet drei Hauptkategorien, in denen Nutzer:innen ihre Informationen speichern und strukturieren können:

Der "Lebenslauf" bietet die Möglichkeit, klassische Daten wie Kontaktdaten, Ausbildung und Berufserfahrung einzugeben und übersichtlich aufzubereiten. Der integrierte Editor erlaubt die Gestaltung eines individuellen und professionellen Lebenslaufs, der sich nahtlos in das gesamte Portfolio einfügt.

Die "Projektseite" ermöglicht die Präsentation bisheriger Projekte. Die Möglichkeit der Integration visueller Elemente wie Bilder oder Screenshots erlaubt eine anschauliche Darstellung der Projekte. Für Nutzer:innen mit technischem Hintergrund besteht zudem die Option der direkten Integration von GitHub-Repositories. Dies erlaubt die authentische und nachvollziehbare Präsentation von Programmierprojekten oder anderen Softwareentwicklungen.

Die Seite "Kompetenzen" bietet die Möglichkeit, spezifische Fähigkeiten und Kompetenzen hervorzuheben, die für die angestrebte Position von Relevanz sind. Diesbezüglich besteht die Möglichkeit, die ausschlaggebenden Eigenschaften und sozialen Kompetenzen in einer übersichtlichen und klar strukturierten Form darzustellen.

Ein besonderes Merkmal von *Melius* ist die Integration und Präsentation aller Daten in einem einheitlichen, ästhetisch ansprechenden Format. Der zeitaufwändige, manuelle Prozess der Zusammenführung verschiedener Dateien und Formate wie PDF-Lebensläufe, Portfolio-Dokumente oder Projektbilder wird somit obsolet.

Ein weiteres herausragendes Merkmal der Plattform ist die Möglichkeit, das vollständige Portfolio unkompliziert über einen einzigen Link zu teilen. Dies ist insbesondere für Bewerberinnen und Bewerber von Vorteil, da sie sich nicht länger mit dem Anhängen umfangreicher Dateien oder der Überschreitung von E-Mail-Datenlimits befassen müssen. Der Link führt direkt zu einer eigenen, personalisierten Online-Präsenz, welche potenziellen Arbeitgeberinnen und Arbeitgebern einen unmittelbaren und umfassenden Überblick über die Qualifikationen und Erfahrungen der Bewerberinnen und Bewerber ermöglicht.

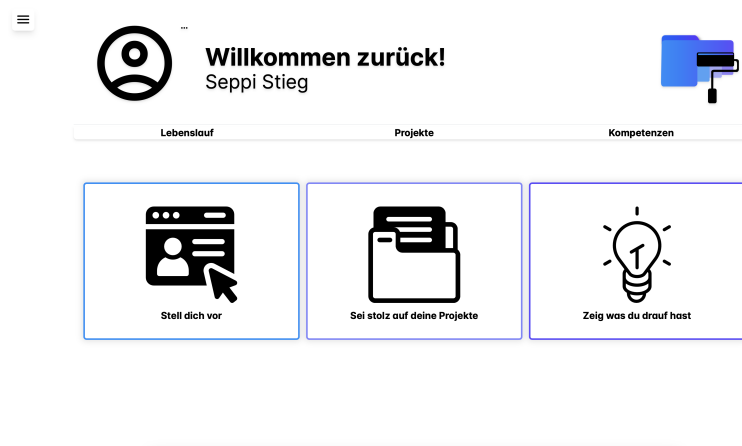


Abbildung 1: Abbildung der Melius Home Page

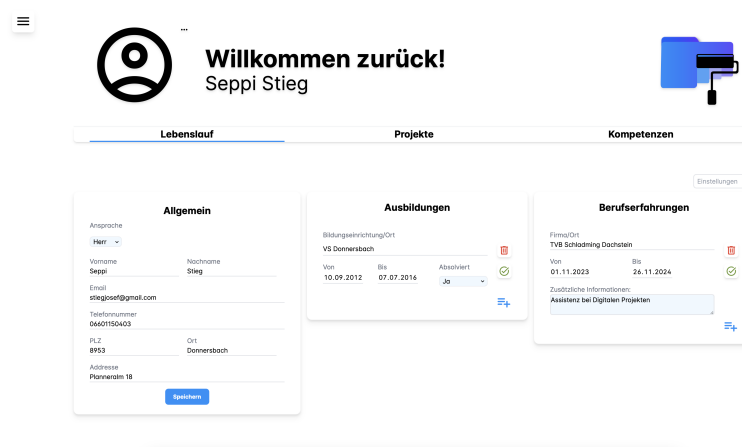


Abbildung 2: Abbildung der Melius Home Page

1.1.2 Gruppenfunktion

Neben der Erstellung individueller, personalisierter Portfolios bietet *Melius* eine zusätzliche Funktionalität, die insbesondere auf die Bedürfnisse von Teams, Unternehmen und dem Personalmanagement zugeschnitten ist. Hierbei handelt es sich um die sogenannte "Gruppenfunktion". Die Erweiterung erlaubt die Erstellung, den Beitritt zu sowie die kollaborative Verwaltung von Talenten und Kompetenzen in Gruppen.

Die Gruppenfunktion ist insbesondere darauf ausgerichtet, sowohl die interne Organisation als auch die externe Zusammenarbeit zu erleichtern. Unternehmen, Projektteams oder Organisationen haben die Möglichkeit, Gruppen anzulegen, um alle relevanten Mitglieder, deren Profile und Kompetenzen an einem zentralen Ort zu bündeln. Dies erleichtert nicht nur die Übersicht, sondern auch die gezielte Suche nach spezifischen Fähigkeiten oder Projektanforderungen.

Die Erstellung einer Gruppe ist unkompliziert und anpassungsfähig gestaltet. Administratorinnen und Administratoren sind in der Lage, Gruppen zu definieren und einzuladen, wodurch ein strukturierter Raum für die Mitglieder entsteht. Des Weiteren ist auch der Beitritt zu bereits bestehenden Gruppen mit geringem Aufwand verbunden. Die Registrierung neuer, sowie die Einladung bereits registrierter Nutzerinnen und Nutzer erfolgt mit nur wenigen Klicks. Dadurch wird es allen Anwenderinnen und Anwendern ermöglicht, ihre Profile in den Kontext der Gruppe einzubringen und von einem koordinierten Netzwerk zu profitieren.

Die Gruppenfunktion verfügt über eine besonders nützliche Eigenschaft, nämlich die Möglichkeit, Mitglieder basierend auf ihren Kompetenzen und Qualifikationen zu filtern. Dies erlaubt die selektive Auswahl von Personen, die den Anforderungen eines spezifischen Projekts oder einer bestimmten Aufgabe entsprechen. Beispielsweise kann ein Unternehmen innerhalb der Gruppe nach Mitgliedern mit Kenntnissen in einer bestimmten Programmiersprache, Erfahrung in Projektmanagement-Tools oder anderen spezialisierten Fähigkeiten suchen. Diese Funktion reduziert den Zeitaufwand für die Suche nach geeigneten Talenten erheblich und gewährleistet die effiziente Nutzung der verfügbaren Ressourcen. Auch bei kurzfristigen Anforderungen kann schnell auf relevante Expertise zurückgegriffen werden.

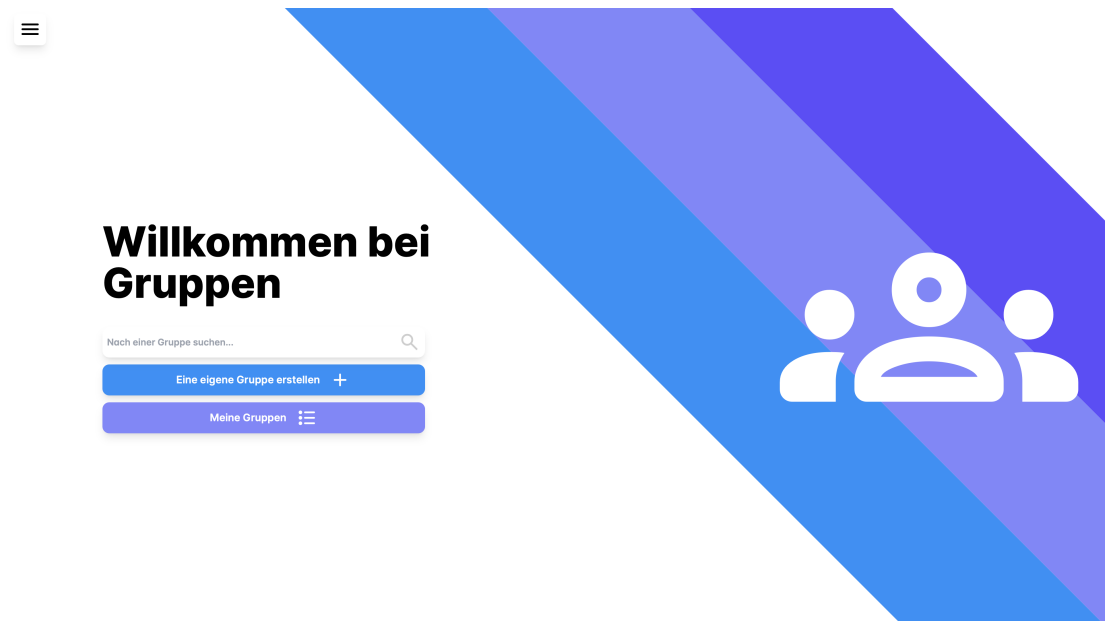


Abbildung 3: Abbildung der Melius Gruppen Seite

1.2 Namensfindung

Der Name *Melius – The Better Profile* setzt sich aus verschiedenen Namensideen zusammen. Zu Beginn wurde das Projekt lediglich unter der Bezeichnung *Better Profile* geführt. Das Wort *Melius* leitet sich vom lateinischen *Melior* ab, was *besser* bedeutet. Demgemäß würde die lateinische Bezeichnung für *Better Profile*, *Melius Profile* lauten. Aus der Kombination dieser beiden Elemente entstand schließlich die Bezeichnung *Melius – The Better Profile*.

2 Umfeldanalyse - Josef Stieg

2.1 Zielgruppe

Melius adressiert Personen aus der IT- und Kreativbranche, die auf der Suche nach einer neuen beruflichen Herausforderung sind oder ihre beruflichen Kompetenzen auf attraktive Weise präsentieren möchten. Gleichzeitig bietet die Plattform Unternehmen aus diesen Bereichen die Möglichkeit, mithilfe der Gruppenfunktion ihre Teammitglieder basierend auf deren Stärken und Kompetenzen effizient und übersichtlich zu filtern. Die Nutzung von *Melius* ist nicht auf eine bestimmte Altersgruppe beschränkt, sodass sowohl Berufseinsteiger als auch erfahrene Fachkräfte von den Funktionen profitieren können.

2.1.1 Bedarf

Melius adressiert eine zentrale Problematik im Bewerbungsprozess, die sich aus der umständlichen und oft speicherplatzbegrenzten Übermittlung von Portfolios in Form mehrerer einzelner Dateien ergibt. Anstatt Bewerbungen mit zahlreichen Anhängen zu verschicken, ermöglicht *Melius* eine kompakte und übersichtliche Darstellung aller relevanten Inhalte – von Lebenslauf und Kompetenzen bis hin zu Projekten und Bildern – auf einer einzigen, personalisierten Portfolio-Seite. Für Bewerber reduziert sich der Aufwand auf das Teilen eines Links, was die Notwendigkeit betrifft, mit Datei-Limits und unübersichtlichen Anhängen zu arbeiten.

Die Gruppenfunktion von *Melius* bietet Unternehmen und Teams eine wertvolle Unterstützung. Innerhalb einer Gruppe können alle Mitglieder die Profile der anderen einsehen und gezielt nach bestimmten Fähigkeiten filtern. Dies ermöglicht es nicht nur Personalverantwortlichen, sondern auch den Mitarbeitenden selbst, schnell und einfach die richtige Person für eine Aufgabe oder ein Projekt zu finden. Die Effizienz der Zusammenarbeit wird gesteigert und das vorhandene Know-how innerhalb eines Teams besser genutzt.

2.2 Ähnliche Online - Portfolio Plattformen

2.2.1 Pixpa

Pixpa ist eine All-in-One-Plattform, die es Kreativen und kleinen Unternehmen ermöglicht, professionelle Portfolio-Websites zu erstellen, ohne über Programmierkenntnisse zu verfügen. Der benutzerfreundliche Drag-and-Drop-Editor und die über 200 anpassbaren, responsiven Vorlagen ermöglichen es den Nutzer:innen, ihre Arbeiten ansprechend zu präsentieren. Darüber hinaus umfasst Pixpa integrierte Funktionen wie E-Commerce, Blogging, SEO- und Marketing-Tools sowie Kundengalerien und stellt somit eine umfassende Lösung für die Online-Präsenz bereit. Die Kosten für die Nutzung der Plattform betragen ab 3,60 USD pro Monat bei einer zweijährlichen Abrechnungsperiode.

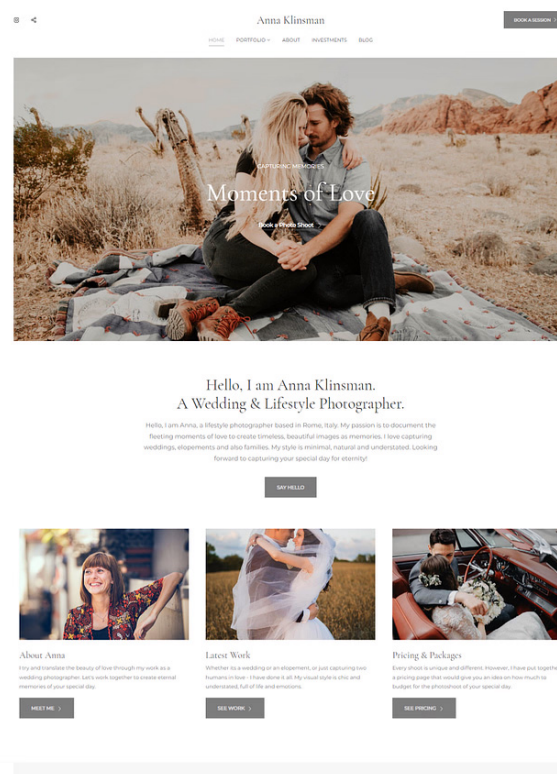


Abbildung 4: Pixpa - Portfolio

2.2.2 Canva

Canva ist für seine benutzerfreundliche Design-Plattform bekannt und bietet auch eine einfache Möglichkeit, Portfolio-Websites zu erstellen. Mit einer Vielzahl von Vorlagen und einem intuitiven Editor können Nutzer:innen ohne Vorkenntnisse schnell ansprechende Portfolios gestalten. Canva eignet sich besonders für Anfänger, die eine unkomplizierte Lösung suchen, um ihre Arbeiten online zu präsentieren.

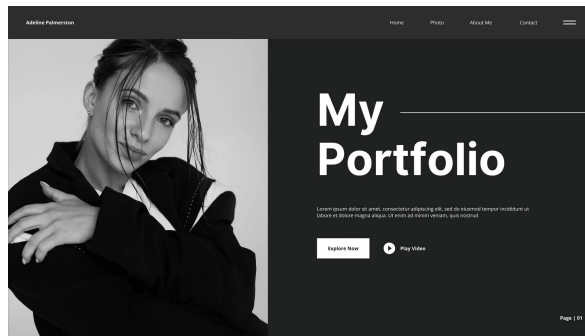


Abbildung 5: Canva - Portfolio

2.2.3 Adobe Portfolio

Adobe Portfolio ist ein Element der Adobe Creative Cloud und ermöglicht es professionellen Kreativschaffenden, nahtlos Portfolio-Websites zu erstellen, die eine Integration mit anderen Adobe-Produkten erlauben. Mithilfe einer Auswahl an modernen, responsiven Vorlagen können Nutzer:innen ihre Arbeiten stilvoll präsentieren. Adobe Portfolio ist ideal für diejenigen, die bereits andere Adobe-Tools verwenden und eine kohärente Plattform für ihre Projekte wünschen.

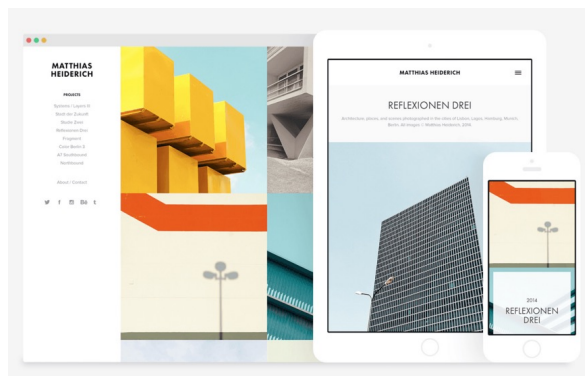


Abbildung 6: Adobe - Portfolio

2.2.4 Squarespace

Squarespace stellt eine Vielzahl von professionell gestalteten Vorlagen bereit, die insbesondere für Designer geeignet sind. Ein leistungsstarker Editor und umfangreiche Anpassungsmöglichkeiten ermöglichen es den Nutzer:innen, individuelle Portfolio-Websites zu erstellen. Darüber hinaus bietet Squarespace Funktionen wie Blogging, E-Commerce und SEO-Tools, um die Online-Präsenz zu stärken.

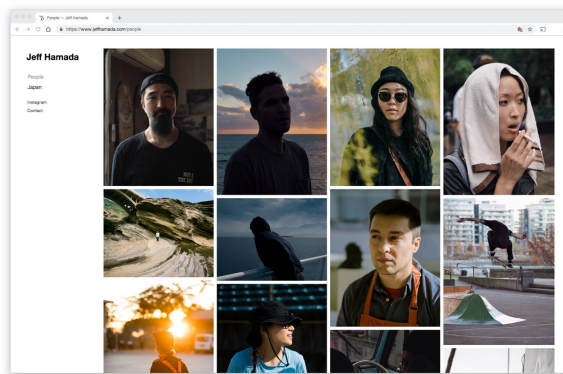


Abbildung 7: Squarespace - Portfolio

2.2.5 Wix

Wix ist für seine Flexibilität bekannt und bietet eine große Auswahl an Vorlagen sowie einen intuitiven Drag-and-Drop-Editor. Nutzer:innen können ihre Portfolio-Websites nach ihren Vorstellungen gestalten und von zahlreichen Apps und Integrationen profitieren. Wix eignet sich für Personen, die eine anpassbare Lösung suchen, um ihre Arbeiten online zu präsentieren.

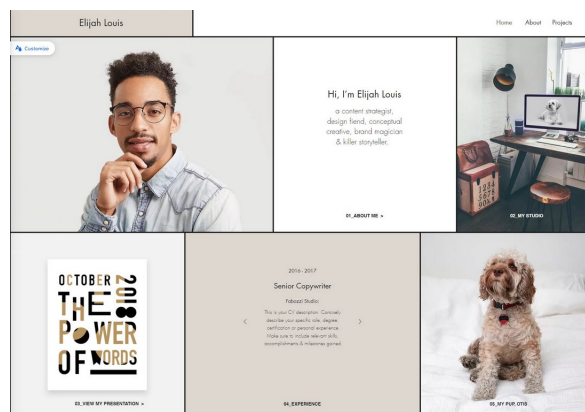


Abbildung 8: Wix - Portfolio

2.2.6 Vergleich mit Melius

Melius bietet einige einzigartige Funktionen, die es von anderen bekannten Portfolio-Website-Buildern wie Pixpa, Canva, Adobe Portfolio, Squarespace und Wix abheben. Ein besonders hervorstechendes Merkmal ist die Gruppenfunktion, die es Unternehmen ermöglicht, ihre Mitarbeiter oder Kollegen zu organisieren und nach spezifischen Kompetenzen zu filtern. Diese Funktion ist besonders nützlich für Personalabteilungen, die eine detaillierte Übersicht über die Fähigkeiten ihrer Mitarbeiter benötigen, was bei den anderen Plattformen nicht der Fall ist.

Ein weiterer Vorteil von *Melius* ist der spezifische Fokus auf die IT- und kreative Branche. Während viele andere Tools allgemein für kreative Berufe entwickelt wurden, richtet sich *Melius* speziell an IT-Profis und kreative Fachkräfte und ermöglicht es, Projekte wie GitHub-Repositories oder technische Arbeiten direkt in das Portfolio zu integrieren.

Melius bietet außerdem eine tiefere Backend-Integration und Datenbankfunktionen, die über einfache visuelle Designoptionen hinausgehen. Nutzer:innen können ihre Portfolios und Bewerbungsunterlagen in einer strukturierten Weise verwalten, was bei den anderen Plattformen meist nicht der Fall ist, da diese sich vor allem auf Frontend-Design und Präsentation konzentrieren.

Schließlich ermöglicht *Melius* eine zentralisierte Verwaltung von Lebensläufen, Projekten und Kompetenzen an einem Ort. Dies erleichtert die Erstellung und Verwaltung von Bewerbungsmaterialien erheblich, während andere Plattformen in der Regel auf die Gestaltung und Präsentation des Portfolios fokussiert sind, ohne diese integrierten Verwaltungstools zu bieten.

Dank dieser einzigartigen Funktionen stellt *Melius* eine umfassendere und spezialisierten Lösung für IT-Profis und Kreative dar und hebt sich so von den traditionellen Portfolio-Website-Buildern ab.

3 Technologien - Josef Stieg

3.1 Backend

Das Backend wurde in der Programmiersprache Java entwickelt und nutzt moderne Technologien wie *Hibernate ORM* und *Jakarta*, um eine robuste und effiziente Architektur bereitzustellen. Es basiert auf dem *Quarkus* Framework, bietet eine RESTful API und arbeitet mit einer PostgreSQL Datenbank, die in einem Docker Container betrieben wird.

3.1.1 Quarkus

Quarkus ist ein modernes, Java-basiertes Framework, das speziell für Cloud-native Anwendungen und Microservices entwickelt wurde. Es zeichnet sich durch hohe Performance und geringen Speicherbedarf aus, was es ideal für containerisierte und serverlose Umgebungen macht. *Quarkus* unterstützt die Erstellung von RESTful APIs und bietet eine Vielzahl integrierter Erweiterungen, die die Entwicklung beschleunigen. Darüber hinaus ist *Quarkus* kompatibel mit Standards wie Jakarta EE und MicroProfile, was die Nutzung bewährter Java-Technologien ermöglicht.



Abbildung 9: Quarkus

3.1.2 Datenmodellierung mit Jakarta

Für die Datenmodellierung und die Anbindung an die Datenbank werden im Backend die Standards von *Jakarta EE* verwendet. Mit Jakarta werden Entitäten (Objekte, die

Tabellen in der Datenbank repräsentieren), Repository-Klassen (für Datenbankoperationen) und Ressourcenklassen (für die REST-API-Endpunkte) implementiert. Dies gewährleistet eine klare Trennung der Verantwortlichkeiten und erleichtert die Wartung und Erweiterung des Codes.

3.1.3 PostgreSQL-Datenbank

Die Anwendung verwendet *PostgreSQL*, ein leistungsfähiges objektrelationales Datenbankmanagementsystem (ORDBMS). *PostgreSQL* ist bekannt für seine Zuverlässigkeit, Skalierbarkeit und erweiterten Funktionen wie Unterstützung für JSON, geografische Daten (PostGIS) und benutzerdefinierte Datentypen. Es ist ideal für Anwendungen mit komplexen Datenstrukturen und hohen Anforderungen an die Datenintegrität.

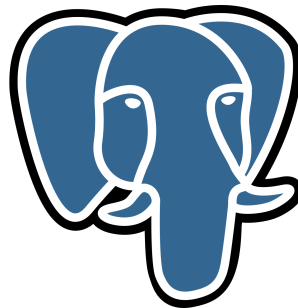


Abbildung 10: Postgresql

3.1.4 Docker und Docker-Composer

Die PostgreSQL Datenbank läuft in einem Docker Container. Docker ist eine Containerisierungsplattform, die es ermöglicht, Anwendungen und ihre Abhängigkeiten in isolierten Umgebungen (Containern) zu betreiben. Container sind leichtgewichtig und portabel, was eine konsistente Ausführung der Anwendung in verschiedenen Umgebungen gewährleistet.

Um die Container-Umgebung zu konfigurieren und zu starten, wird eine Docker-Compose-Datei verwendet. Docker Compose ist ein Werkzeug, mit dem mehrere Containerdienste definiert und orchestriert werden können. In diesem Fall wird es verwendet, um die PostgreSQL-Datenbank zu starten und alle notwendigen Parameter wie Netzwerkverbindungen und Volumes zu setzen.



Abbildung 11: Docker

3.1.5 Zusammenspiel der Technologien

Der Backend-Technologie-Stack kombiniert diese Komponenten, um eine effiziente, skalierbare und wartbare Architektur zu schaffen. *Quarkus* bildet die Basis für die Geschäftslogik und API-Implementierung, während PostgreSQL als zuverlässige und hochperformante Datenbanklösung dient. Docker sorgt für eine flexible Bereitstellung der Infrastruktur und Jakarta EE stellt sicher, dass die Interaktion zwischen Code und Datenbank auf bewährten Standards basiert.

3.2 Frontend

Das Frontend basiert auf modernen Web-Technologien und setzt auf TypeScript, *Angular* und Tailwind CSS, um eine performante, wartbare und benutzerfreundliche Benutzeroberfläche zu entwickeln.

3.2.1 TypeScript

TypeScript ist eine Open-Source-Programmiersprache, die auf JavaScript basiert, jedoch zusätzliche Funktionen wie statische Typisierung und moderne ECMAScript-Features bietet. Die Einführung von Typen erhöht die Sicherheit und Lesbarkeit des Codes erheblich, was insbesondere bei umfangreichen Projekten die Wartung vereinfacht. Die Übersetzung von TypeScript in standardmäßiges JavaScript durch den Compiler gewährleistet die Ausführung in allen Webbrowsern.



Abbildung 12: Typescript

3.2.2 Angular

Das von Google entwickelte Framework *Angular* stellt eine der leistungsstärksten Plattformen für die Erstellung von Single-Page Applications (SPAs) dar. Es basiert auf TypeScript und bietet eine umfangreiche Struktur, die Entwicklern dabei hilft, komplexe Anwendungen zu erstellen und zu organisieren. Zu den Hauptmerkmalen von *Angular* gehören:

- **Komponentenbasierte Architektur:** Anwendungen werden in wiederverwendbare und leicht testbare Komponenten zerlegt
- **Datenbindung:** *Angular* unterstützt Zwei-Wege-Datenbindung, wodurch Änderungen im Model automatisch in der Benutzeroberfläche reflektiert werden und umgekehrt.

- **Integrierte Tools:** *Angular* bietet direkt integrierte Funktionen wie Routing, Formulare, HTTP-Client und Dependency Injection.
- **RxJS:** Die reaktive Programmierbibliothek RxJS wird verwendet, um asynchrone Datenströme zu verarbeiten, was besonders in Anwendungen mit hohem Datenverkehr oder Echtzeitanforderungen nützlich ist.

Die Entscheidung für *Angular* basierte auf den bisherigen Erfahrungen des Entwicklungsteams, welches *Angular* im Rahmen des Softwareentwicklungsunterrichts erlernt hatte. Dadurch waren die Teammitglieder bereits mit den grundlegenden Konzepten und der Arbeitsweise des Frameworks vertraut, was einen schnellen Einstieg in die Entwicklung ermöglichte und die Notwendigkeit reduzierte, neue Technologien von Grund auf zu erlernen.

Zusätzlich bot *Angular* durch seine umfassende Plattform und integrierte Funktionen wie Routing, Formularverwaltung und Dependency Injection eine strukturierte und effiziente Entwicklungsumgebung, was sich positiv auf die Arbeitsatmosphäre auswirkte. Die Vertrautheit mit dem Framework und die damit verbundene Sicherheit im Umgang trugen entscheidend dazu bei, dass sich das Team in der Arbeit mit *Angular* wohlfühlte.



Abbildung 13: Angular

3.2.3 Tailwind CSS

Tailwind CSS ist ein Utility-First-Framework für die Gestaltung von Benutzeroberflächen. Es divergiert von klassischen CSS-Ansätzen insofern, als dass es eine große Anzahl vorgefertigter CSS-Klassen bereitstellt, die direkt in HTML verwendet werden können. In der Konsequenz wird der Bedarf, eigene CSS-Dateien zu schreiben, oft eliminiert, was zu einer Beschleunigung der Entwicklung und einer Sicherstellung der Konsistenz des Designs führt.

Hauptvorteile von Tailwind CSS:

- **Flexibilität:** Entwickler haben die volle Kontrolle über das Design, ohne auf starre Vorgaben angewiesen zu sein.
- **Konsistenz:** Durch den Einsatz von vordefinierten Klassen wie `text-blue-500` oder `p-4` wird ein einheitliches Design über die gesamte Anwendung hinweg gewährleistet.
- **Optimierung:** Tailwind bietet eine automatische Optimierung und entfernt ungenutzte CSS-Klassen in der Produktionsumgebung, was die Dateigröße reduziert und die Ladezeit verbessert.



Abbildung 14: Tailwind

4 Design - Jakob Schlager

4.1 Grundidee

Bei der Entwicklung des Designs der Applikation wurde besonders darauf geachtet, eine benutzerfreundliche und gleichzeitig schlichte Benutzeroberfläche zu schaffen. Der Fokus lag darauf, die Anwendung so einfach wie möglich zu gestalten, ohne auf wichtige Funktionalitäten oder Flexibilität zu verzichten. Die Entscheidung, ein minimalistisches Design zu wählen, basiert auf der Überzeugung, dass eine schöne und aufgeräumte Oberfläche den/die Benutzer:in nicht überfordert, sondern dabei hilft, sich besser auf die wesentlichen Inhalte und Funktionen zu konzentrieren.

Ein zentraler Aspekt des Designprozesses war, den Entwickler:innen die Möglichkeit zu geben, den grundlegenden Aufbau der Anwendung festzulegen, während der/die Benutzer:in die Freiheit hat, das Layout ganz nach ihren eigenen Bedürfnissen und Vorstellungen anzupassen. So können sie entscheiden, wie die Informationsboxen angeordnet sind, welche Primärfarbe für den Hintergrund verwendet wird und viele weitere Anpassungen vornehmen, um die Anwendung zu personalisieren. Dies fördert nicht nur die individuelle Nutzungserfahrung, sondern unterstützt auch eine größere Anpassungsfähigkeit und Benutzerzentrierung, da verschiedene Nutzer:innen unterschiedliche Bedürfnisse und Vorlieben haben.

Inspiziert von der minimalistischen und flexiblen Struktur von Applikationen wie **Notion** (Abbildung 15), wurde das Design mit dem Ziel entwickelt, eine hohe Anpassungsfähigkeit zu ermöglichen, während gleichzeitig eine klare und intuitiv verständliche Benutzeroberfläche erhalten bleibt.

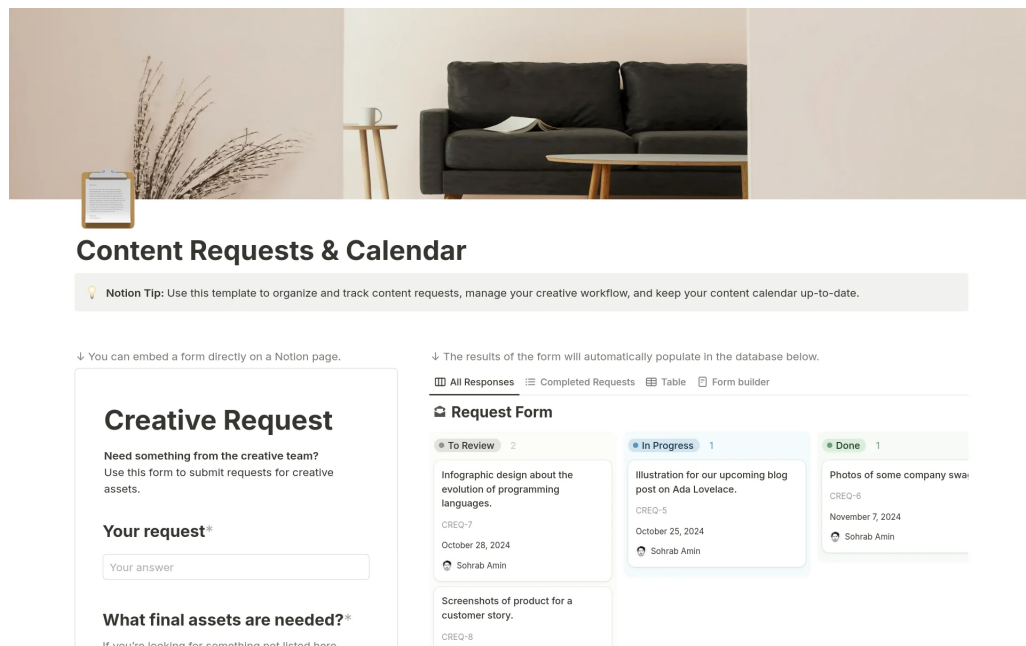


Abbildung 15: Abbildung des Notion Layouts

Ein weiterer Vorteil eines simplen Designs ist die Reduzierung von kognitiver Belastung. Durch die Wahl eines minimalistischen Ansatzes werden unnötige Elemente und Ablenkungen vermieden, was es dem/der Benutzer:in ermöglicht sich zuerst auf seine wirklich wichtigen Aufgaben zu konzentrieren, und anschließend sich mit dem Aussehen seiner Seite auseinanderzusetzen. Diese Kombination bietet eine hohe Benutzerbindung und fördert zudem die Zufriedenheit. Dies war ein entscheidender Punkt in der Designentwicklung, da man davon überzeugt war, dass eine klare und anpassbare Benutzeroberfläche langfristig zu einer besseren User Experience führt und die Akzeptanz der Anwendung steigert.

Letztlich wurde beim Design der Applikation eine Balance zwischen Schlichtheit und Funktionalität angestrebt, bei der die Benutzeroberfläche nicht nur ästhetisch ansprechend, sondern auch funktional und benutzerfreundlich bleibt

4.2 Das Logo und die ersten Entwürfe

Das Logo stellt eines der zentralen Elemente der visuellen Identität einer Applikation, Firma, Unternehmens etc. dar. Auch bei *Melius* wurde eine intensive Auseinandersetzung mit der Frage vorgenommen, wie die mögliche Kreativität und die drei primären Farben auf einer Website so zusammenzufassen, so dass alle Punkte in einem schönen und einfachen Logo adäquat reflektiert werden können. Da ein Farbbroller wichtig für die

Arbeit eines Malers ist, wurde sich überlegt, diesen in Zusammensetzung mit dem Ordner als primäres Symbol für das Projekt zu wählen.

4.2.1 Erster Entwurf

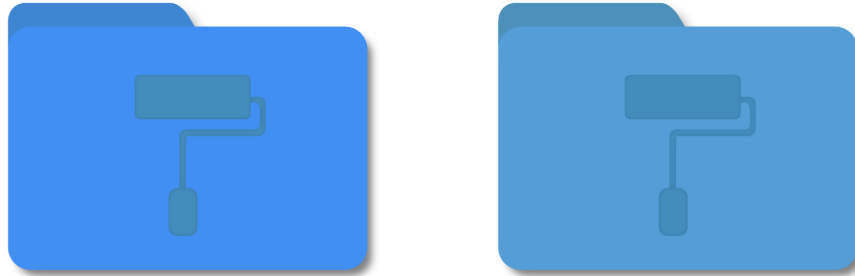


Abbildung 16: Erster Entwurf des Melius Logo

Der erste Entwurf des Logos ist bewusst schlicht gehalten und erinnert durch seine klare, geometrische Form sowie die ausgewogene Struktur an einen Datei-Ordner des Betriebssystems **macOS**. Diese Assoziation wurde bewusst gewählt, um dem Logo eine moderne, benutzerfreundliche und zugleich funktionale Ästhetik zu verleihen, die eine positive Wirkung auf den/die Nutzer:in hat.

Der Schwerpunkt der ersten Version lag auf der Basisstruktur des Logos sowie der grundlegenden Idee seiner zukünftigen Entwicklung. Das Ziel war die Kreation eines einfachen, aber aussagekräftigen Symbols, welches eine unmittelbare Wiedererkennung ermöglicht und gleichzeitig hinreichend Flexibilität bietet, um in späteren Versionen des Designs eine weitere Verfeinerung zu ermöglichen. Die erste Version war weniger auf Detailtreue bedacht, sondern vielmehr auf eine klare und minimalistische Darstellung der wesentlichen Designelemente, welche das visuelle Konzept des Logos in seiner Essenz widerspiegeln.

4.2.2 Zweiter Entwurf



Abbildung 17: Zwischenentwurf des Melius Logo

Der zweite Entwurf des Logos weist im Vergleich zur ersten Version bereits signifikante Fortschritte auf. Während der erste Entwurf durch eine zurückgenommene Farbgebung sowie eine minimalistische Gestaltung geprägt war, wurden in dieser Version mehrere entscheidende Verbesserungen und Erweiterungen vorgenommen.

Veränderungen und Verbesserungen

Die Integration der Hauptfarben stellt eine wesentliche Veränderung dar. In der vorliegenden Version wurden drei Hauptfarben eingeführt, welche sowohl das Logo lebendiger erscheinen lassen als auch als Fundament für die visuelle Identität der Applikation dienen. Die genannten Farben wurden zu einem späteren Zeitpunkt konsequent in der Gestaltung der Website sowie anderer Applikationen verwendet, wodurch eine stärkere Wiedererkennung und Einheitlichkeit des Brandings gewährleistet wird.

Verwendung von Farbverläufen

Im Vergleich zur ersten Version, die durch flache Farben dominiert wurde, weisen die Elemente des Logos in dieser Version sanfte Farbverläufe auf. Dies resultiert in einer modernen und dynamischen Ästhetik, welche die visuelle Tiefe verstärkt und das Design insgesamt ansprechender gestaltet.

Das Farbschema

Im ersten Entwurf wurden die Elemente des Logos, wie beispielsweise die Farbrolle, in dezenten, einheitlichen Blautönen gehalten. In der zweiten Version hingegen wurden kontrastierende Farben verwendet. So wurde die Farbrolle beispielsweise in einem kräftigen Weiß oder mit einem Farbverlauf hervorgehoben, was sie optisch in den Vordergrund rückt und die Möglichkeiten für Kreativität und Anpassbarkeit der Anwendung besser vermitteln soll.

4.2.3 Finaler Entwurf

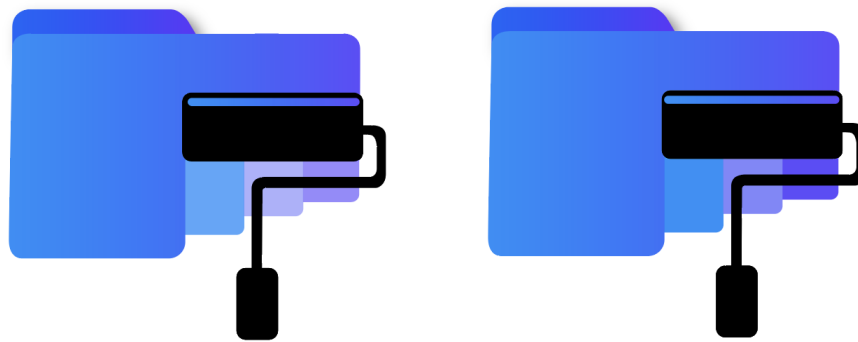


Abbildung 18: Finaler Entwurf des Melius Logo

Wie man hier sehr gut erkennen kann, hat das Logo in der endgültigen Version im Vergleich zu den vorherigen Visualisierungen seine endgültige Form angenommen und wirkt nun deutlich ausgereifter. Der Schwerpunkt liegt auf der Harmonie zwischen den Elementen, der Modernität des Designs und der Klarheit der Symbolik.

Der einzige Unterschied zwischen den beiden Abbildungen des endgültigen Entwurfs, der nur schwer zu erkennen ist, besteht in der Sättigung der Farbstreifen unter dem Farbröller. Diese kleine, aber entscheidende Anpassung war Gegenstand einer ausführlichen Diskussion innerhalb der Schulkasse, ergänzt durch Rückmeldungen von verschiedenen Lehrer:innen. Nach einer gemeinsamen Abstimmung entschied man sich schließlich für die **rechte Version** des Logos.

Das endgültige Logo hebt die kreative Idee hinter dem Farbröller besonders hervor. Der Farbröller wurde so integriert, dass er nicht nur den Ordner zu „rollen“ scheint, sondern auch eine Verbindung zwischen Kreativität und Organisation schafft. Diese Darstellung soll symbolisieren, dass die im Ordner enthaltenen Informationen durch den Farbröller nicht nur zugänglich gemacht, sondern auch mit einer kreativen Note angereichert werden. Dadurch wird eine visuelle Verbindung zwischen der praktischen Funktionalität und der kreativen Inspiration des Projekts geschaffen.

Besonders hervorzuheben ist, dass die klaren Linien und die ausgewogene Farbgestaltung dem Logo eine professionelle und moderne Ästhetik verleihen. Die subtilen Farbübergänge im Ordner und die akzentuierte Darstellung des Farbröllers erzeugen eine dynamische Wirkung, die den Kerngedanken von *Melius* optimal transportiert. Das Ergebnis ist ein Logo, das nicht nur funktional und ästhetisch ansprechend ist, sondern auch die Kernbotschaften des Projekts - Kreativität, Struktur und Innovation - perfekt visualisiert.

4.3 Entwürfe der Website

4.3.1 Die Landing Page

Die Landing page der Anwendung *Melius* wurde mit dem Ziel entwickelt, den Benutzer:innen bereits beim ersten Eindruck von der modernen Funktionalität zu überzeugen. Dabei wurden bewusste Entscheidungen bei dem Design und der Struktur getroffen, um sowohl die Kreativität als auch die Datensicherheit - zwei zentrale Werte der Anwendung - zu verdeutlichen.

Design und Struktur

Die Gestaltung der Landing Page folgt einem minimalistischen Ansatz, um den/die Nutzer:in nicht zu überfordern und den Fokus auf das Wesentliche zu lenken. Die Seite ist zweigeteilt: Auf der linken Seite befindet sich in zentraler Position der Name der Anwendung *Melius*. Der Name ist in einer Kombination aus drei Primärfarben gehalten, was ihn nicht nur hervorhebt, sondern auch die Markenidentität stärkt. Direkt darunter befinden sich zwei Call-to-Actions (CTAs): Der blaue „Get Started“-Button, der als primärer CTA den/die Nutzer:in zur Registrierung motiviert und der dezentere „Login“-Button, der für bereits registrierte Nutzer:innen gedacht ist.

Die rechte Seite der Landing Page wird vom zentralen Logo dominiert. Der Hintergrund wird durch subtile, leicht transparente blaue Radiowellen ergänzt, die dem Design Dynamik und Tiefe verleihen, ohne vom Hauptinhalt abzulenken.

Das Logo als zentrales Element

Das Logo wurde bewusst als zentrales Element in den Vordergrund gerückt, da es die Kernfunktionen der Anwendung symbolisiert: die individuelle Anpassung und die sichere Verwaltung von Daten. Um die Vielseitigkeit und Kreativität von *Melius* zu unterstreichen, wurden mehrere Varianten für die Startseite entwickelt:

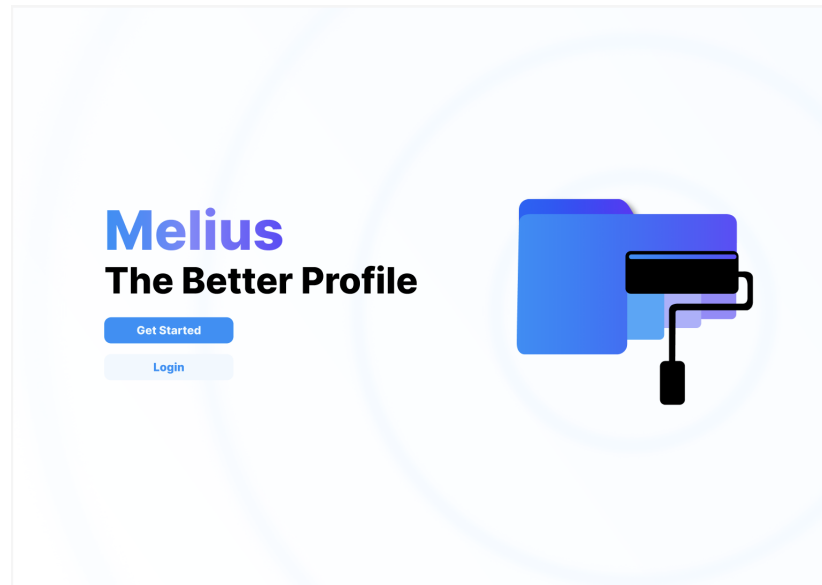
Variation 1: Minimalistisch und Schlicht

Abbildung 19: Erster Entwurf der Landing Page

In der ersten Version präsentiert sich das Logo in einer reduzierten Form, ohne visuelle Elemente, was einen aufgeräumten und klaren Eindruck vermittelt.

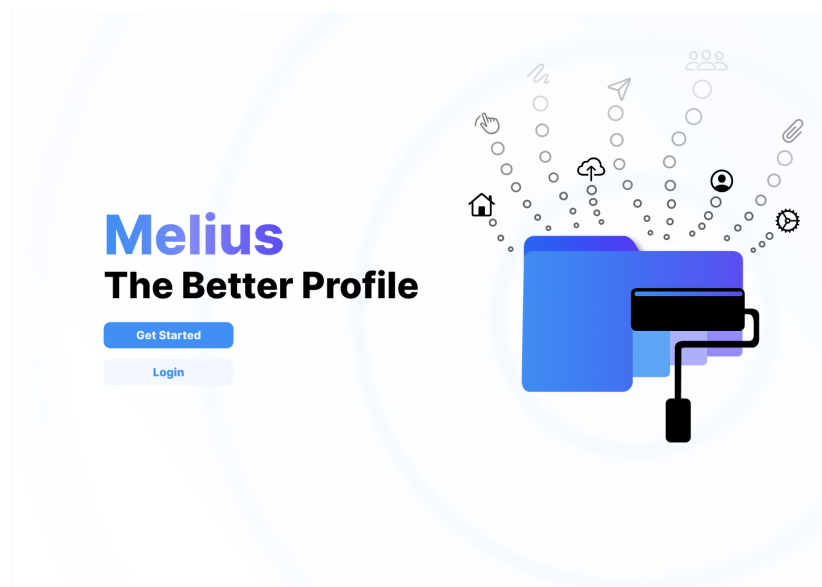
Variation 2: Integration kreativer Elemente

Abbildung 20: Zweiter Entwurf der Landing Page

In der zweiten Version werden Icons und Symbole – darunter ein Haus, ein Upload-Icon und ein Personen-Symbol – in das Design integriert, die symbolisch für die vielfältigen Funktionen stehen. Die Datenpunkte, die aus der Mappe herauszuströmen scheinen, verdeutlichen die Offenheit und Interaktivität der Applikation.

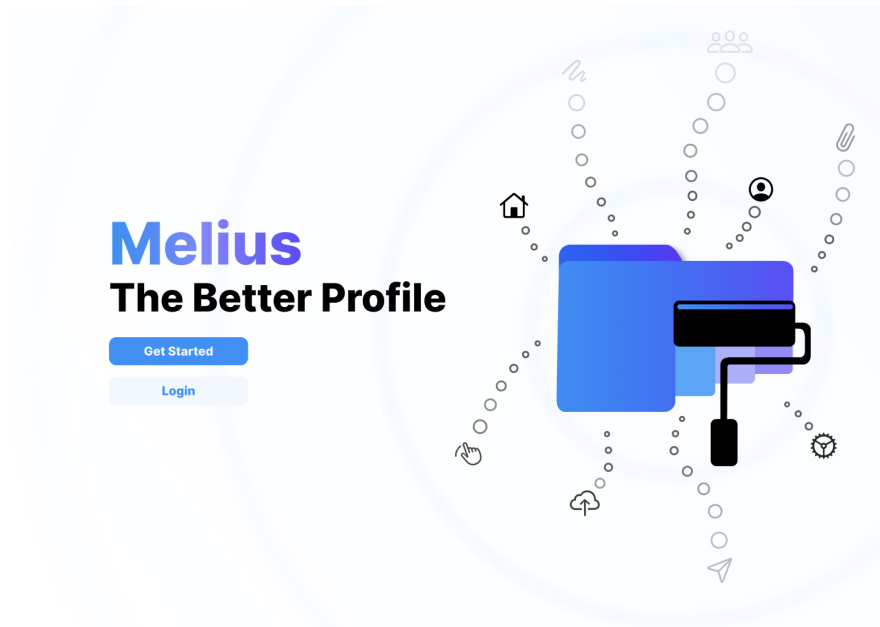
Variation 3: Finalisierte, strukturierte Darstellung

Abbildung 21: Finaler Entwurf der Landing Page

In der finalen Version wurde das Konzept verfeinert, indem die Symbole und Datenpunkte sich radial um das Logo verteilen, wodurch die Struktur und Ordnung stärker hervorgehoben werden. Die kreative Freiheit und die Sicherheit der Nutzer:innen bei der Nutzung der Applikation werden durch die aus der Mappe hervorgehenden Symbole und Datenpunkte veranschaulicht. Die Möglichkeiten der Individualisierung von Profilen und der Visualisierung von Daten werden durch die Symbole und Datenpunkte verdeutlicht. Gleichzeitig signalisiert die klare und konsistente Farbgebung Vertrauen und Stabilität.

4.3.2 Anmeldung und Registrierung

Es lassen sich gemeinsame Designelemente feststellen. Sowohl die Anmelde- als auch die Registrierungsseite basieren auf einem zweispaltigen Layout, das eine klare Aufteilung zwischen der Informations- und Navigationsseite (links - Abbildung 22) und der interaktiven Formularseite (rechts - Abbildung 22) bietet. Die linke Seite enthält den Titel *Melius* und den Slogan "The Better Profile", die den zentralen Markenwert kommunizieren. Diese statischen Elemente bieten Orientierung und schaffen eine visuelle Verbindung zur allgemeinen Markenidentität, wie sie bereits auf der Landing Page etabliert wurde.

Die Anmeldeseite

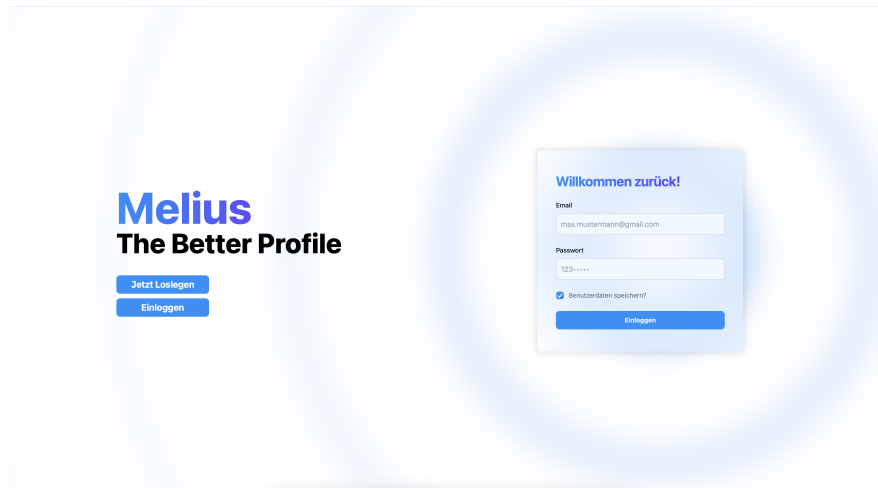


Abbildung 22: Anmeldeformular

Die Anmeldeseite wurde speziell für bestehende Benutzer:innen konzipiert und zeichnet sich durch ein schlankes und effizientes Design aus, wobei der Fokus auf der schnellen und einfachen Ermöglichung des Zugangs zu dem Benutzerkonto liegt. Das Formulardesign ist auf Effizienz ausgelegt, wobei die Felder für E-Mail-Adresse und Passwort zentral und übersichtlich angeordnet sind. Beide Felder sind großzügig dimensioniert, um eine angenehme Eingabeerfahrung zu gewährleisten, und es stehen klare Platzhaltertexte zur Orientierung zur Verfügung.

- Das Formular umfasst die Felder E-Mail und Passwort, die zentral und übersichtlich platziert sind. Beide Felder sind ausreichend groß gestaltet, um die Eingabe komfortabel zu machen, und bieten klare Platzhaltertexte zur Orientierung.
- Eine zusätzliche Checkbox mit der Beschriftung “Benutzerdaten speichern?” bietet den Benutzer:innen die Möglichkeit, ihre Anmeldedaten für zukünftige Sitzungen zu speichern, was die Benutzerfreundlichkeit insbesondere für wiederkehrende :innen erhöht.
- Der Button “Einloggen” ist in einem kräftigen Blauton gestaltet, um Aufmerksamkeit zu erzeugen und den nächsten Schritt klar zu kommunizieren.

Darüber hinaus trägt die visuelle Abgrenzung dazu bei, dass die einzelnen Elemente auf dem Formular klar voneinander getrennt sind.

- Das Formular ist durch einen leicht transparenten und verschwommenen Hintergrund optisch hervorgehoben, ohne aufdringlich zu wirken, was es dem/der Benutzer:in erleichtert, sich auf die wesentlichen Elemente zu konzentrieren.

Die Registrierungsseite

Abbildung 23: Registrierungsformular

Die Registrierungsseite ist für neue Benutzer:innen konzipiert, die ein Konto erstellen möchten. Da dieser Prozess eine höhere Anzahl an Informationen erfordert, ist das Formular umfangreicher gestaltet, ohne dabei an Klarheit zu verlieren. Das Formulardesign ist darauf ausgerichtet, den Prozess der Kontoerstellung für neue Benutzer:innen zu vereinfachen und zu beschleunigen. **Das Formular enthält die folgenden Eingabefelder:**

- Vorname und Nachname: Zur Eingabe der persönlichen Daten.
- E-Mail: Für die Registrierung der Kontaktadresse.
- Des Weiteren besteht die Option, ein zusätzliches Kontaktfeld für Telefonnummern anzulegen, um die Kontaktmöglichkeiten zu erweitern.
- Des Weiteren sind ein Passwort und dessen Bestätigung zur Erstellung sowie Anmeldung eines sicheren Kontos erforderlich.
- Am Ende des Formulars befindet sich eine Checkbox, mit der de/die Benutzer:in die Geschäftsbedingungen akzeptieren muss. Diese ist klar gekennzeichnet, und es wird eine Fehlermeldung angezeigt, wenn sie nicht angeklickt wurde.
- Der Button “Erstelle meinen Account” ist in einem kräftigen Blau gehalten und leitet den/die Benutzer:in zum Abschluss der Registrierung.

Das Formular ist durch eine logische Reihenfolge der Eingabefelder und deutliche Platzhaltertexte leicht verständlich. Die visuelle Abgrenzung durch den leicht transparenten und verschwommenen Hintergrund hilft, das Formular klar hervorzuheben.

4.3.3 Die Startseite

Das Design der Startseite folgt einem zeitgemäßen, schlichten und nutzerfreundlichen Konzept, das durch klare Strukturen und intuitive Navigation besteht. Die Gestaltung zielt darauf ab, dem/der Benutzer:in einen einfachen Einstieg in die verschiedenen Inhalte der Plattform zu ermöglichen, während es gleichzeitig eine ästhetische und professionelle Atmosphäre schafft.

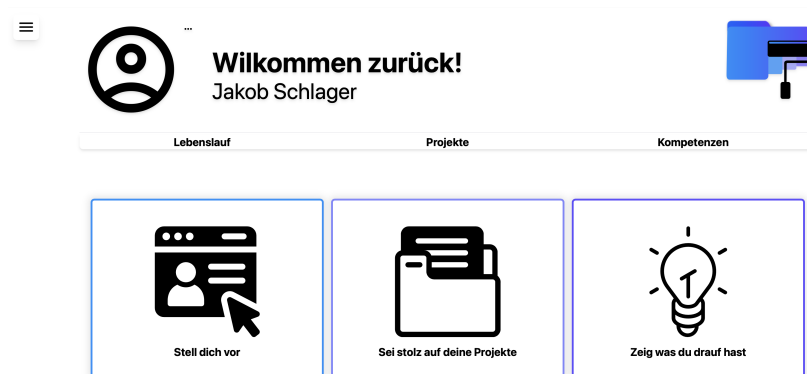


Abbildung 24: Startseite

Der Willkommens-Bereich

Die Kopfzeile der Startseite präsentiert die Benutzerinformationen und zentralen Funktionen in einer klaren und übersichtlichen Form. Im linken Bereich wird ein großes, kreisförmiges Standard-Profilbild angezeigt, unmittelbar gefolgt von einer personalisierten Begrüßung mit dem Namen des/der Benutzer:in, was eine maßgeschneiderte Benutzererfahrung gewährleistet. Ein zentrales Element ist das Dropdown-Menü, das durch ein kleines Dreipunkt-Symbol aufgerufen werden kann. Dieses Menü bietet zwei Optionen:

- “Profilbild hinzufügen”: Der/die Benutzer:in kann ein eigenes Bild hochladen.
- “Profilbild entfernen”: Falls bereits ein Bild gesetzt wurde, kann es wieder entfernt werden.

Auf der rechten Seite des Headers befindet sich das *Melius*-Logo, das aus einer blauen, stilisierten Akte mit einer schwarzen Farbrolle besteht. Dieses visuelle Element unterstreicht das Corporate Design der Plattform und sorgt für einen professionellen Wiedererkennungswert.

Die Kopfzeile vereint somit eine klare Benutzerführung mit einer personalisierten Gestaltung und bietet dem/der Benutzer:in eine intuitive Möglichkeit, sein Profil anzupassen.



Abbildung 25: Kopfzeile

Die interaktiven Boxen

Der Fokus liegt auf den drei großen, visuell ansprechend gestalteten, interaktiven Kacheln im Zentrum der Seite, die gleichzeitig als zusätzliche Navigation dienen. Sie repräsentieren die Unterseiten “Allgemein”, “Projekte” und “Kompetenzen”.

- Die linke Box, die den Titel Stell dich vor “Lebenslauf” trägt, ermutigt den/die Benutzer:in, persönliche Informationen zu hinterlegen oder sich vorzustellen. Das Icon eines Profilbilds auf einer Webseite symbolisiert diesen Bereich perfekt.
- Die Präsentation der eigenen Projekte “Projekte” wird in der mittleren Box angeregt. Die Darstellung einer Mappe mit Dokumenten verdeutlicht die Funktionalität, eigene Arbeiten zu strukturieren und sichtbar zu machen.
- Die rechte Box, die mit einer Glühbirne versehen ist, steht für Kreativität und “Kompetenzen” und lädt dazu ein, eigene Kompetenzen und Stärken zu präsentieren und hervorzuheben.

Die Abgrenzung der einzelnen Boxen sowie die Verwendung von Primärfarben tragen zur optischen Anmutung bei und ziehen die Aufmerksamkeit der Benutzer:innen auf sich. Die reduzierte Farbpalette in Kombination mit den minimalistischen Icons erzeugt eine aufgeräumte und professionelle Optik.

Die Navigation

Unterhalb des Willkommen-Bereichs befindet sich eine horizontale Navigationsleiste mit drei Hauptkategorien: “Lebenslauf”, “Projekte” und “Kompetenzen”. Diese Tabs bieten einen schnellen Zugriff auf die Hauptinhalte der Plattform und machen die Struktur der Website auf den ersten Blick nachvollziehbar. Die klare Trennung der Kategorien sorgt dafür, dass Benutzer:innen sich mühelos in die für sie relevanten Bereiche navigieren können.

Ebenfalls befindet sich in der linken oberen Ecke ein Hamburger Menü, welches dem/der Benutzer:in den Zugriff zur Navigation für die gesamte Applikation bietet. Hier wurde sich für ein innovatives, aber dennoch schlichtes Design entschieden. In der Navigation hat der/die Benutzer:in die Möglichkeit die Seiten:

- **Home**
- **Gruppen**
- **Die Einstellungen**
- **Abmelden**

zu öffnen bzw. zu erreichen. Um den/der Benutzer:in gut zu visualisieren auf welcher Seite er sich im Moment befindet, bekommt die gerade aktive Navigation einen leicht hellblauen Hintergrund.



Abbildung 26: Navigation der Unterseiten der Applikation

4.3.4 Lebenslauf

Die Unterseite “Lebenslauf” dient dem/der Benutzer:in als zentrale Anlaufstelle, um persönliche Informationen, Bildungsabschlüsse und berufliche Erfahrungen zu verwalten. Die Benutzeroberfläche ist klar strukturiert und in drei Hauptbereiche unterteilt: Allgemeine Informationen, Ausbildungen und Berufserfahrungen. Diese Abschnitte sind in separaten, visuell ansprechenden Formularen angeordnet, die eine intuitive Navigation und einfache Datenbearbeitung ermöglichen.

Willkommen zurück!
Max Mustermann

Lebenslauf Projekte Kompetenzen

Einstellungen ▾

Ausbildungen

Bildungseinrichtung/Ort
HTBLA Leonding

Von 02/09/2020 Bis 02/07/2025 Absolviert ☒

Allgemein

Anrede ▾

Vorname Max Nachname Mustermann

Email max.mustermann@gmail.com

Telefonnummer 1234567890

PLZ 4020 Stadt Linz

Adresse Sonnengasse 9

Speichern

Berufserfahrungen

Firma/Ort
Programmierfabrik

Von 02/07/2023 Bis 02/08/2023

Zusätzliche Informationen
Praktikum erfolgreich beendet

Abbildung 27: Unterseite Lebenslauf

Ausbildungen

Das linke Formular ermöglicht es dem/der Benutzer:in, seine bisherigen und aktuellen Bildungswege einzutragen. Hier können folgende Angaben gemacht werden:

- Bildungseinrichtung Ort: Der Name der Institution oder des Ausbildungsortes.
- Zeitraum: Start- und Enddatum der Ausbildung.
- Absolviert-Status: Eine Auswahloption, mit der der/die Benutzer:in angeben kann, ob die Ausbildung bereits abgeschlossen ist.

Ein wesentliches Feature ist die Möglichkeit, zusätzliche Ausbildungsstationen über eine Schaltfläche hinzuzufügen. Ebenso kann eine eingetragene Ausbildung bei Bedarf wieder gelöscht werden.

Allgemein

Im mittleren Formular kann der/die Benutzer:in grundlegende Angaben zu seiner Person eintragen. Dazu gehören:

- Anrede (z. B. Herr, Frau oder Divers)
- Vorname und Nachname
- E-Mail-Adresse
- Telefonnummer

- Postleitzahl und Stadt
- Wohnadresse

Alle Felder sind frei editierbar, sodass der/die Benutzer:in jederzeit Änderungen vornehmen und seine Daten in der Datenbank speichern kann.

Berufserfahrungen

Auf der rechten Seite befindet sich das Formular für die beruflichen Erfahrungen. Hier kann der/die Benutzer:in Informationen zu bisherigen Tätigkeiten, Praktika oder anderen relevanten Erfahrungen eingeben. Die Felder umfassen:

- Firma / Ort: Name des Unternehmens oder der Organisation.
- Zeitraum: Start- und Enddatum der Tätigkeit.
- Zusätzliche Informationen: Ein Textfeld für weitere Details, z. B. Aufgabenbereiche oder besondere Leistungen.

Auch hier gibt es die Möglichkeit, weitere Berufserfahrungen über eine Schaltfläche hinzuzufügen oder bestehende Einträge wieder zu entfernen.

Benutzerfreundlichkeit und Anpassungsmöglichkeiten

Die Unterseite "Lebenslauf" wurde mit besonderem Fokus auf eine einfache Handhabung gestaltet. Die klar strukturierten Formulare ermöglichen eine schnelle und intuitive Dateneingabe. Die Benutzer:innen können Einträge jederzeit bearbeiten, hinzufügen oder entfernen, was eine hohe Flexibilität gewährleistet.

Durch die klare optische Trennung der Abschnitte und die Verwendung interaktiver Elemente wie Drop-down-Menüs und Schaltflächen bietet die Unterseite eine benutzerfreundliche und übersichtliche Verwaltung der eigenen beruflichen und akademischen Laufbahn.

Die Drag-and-Drop Funktion

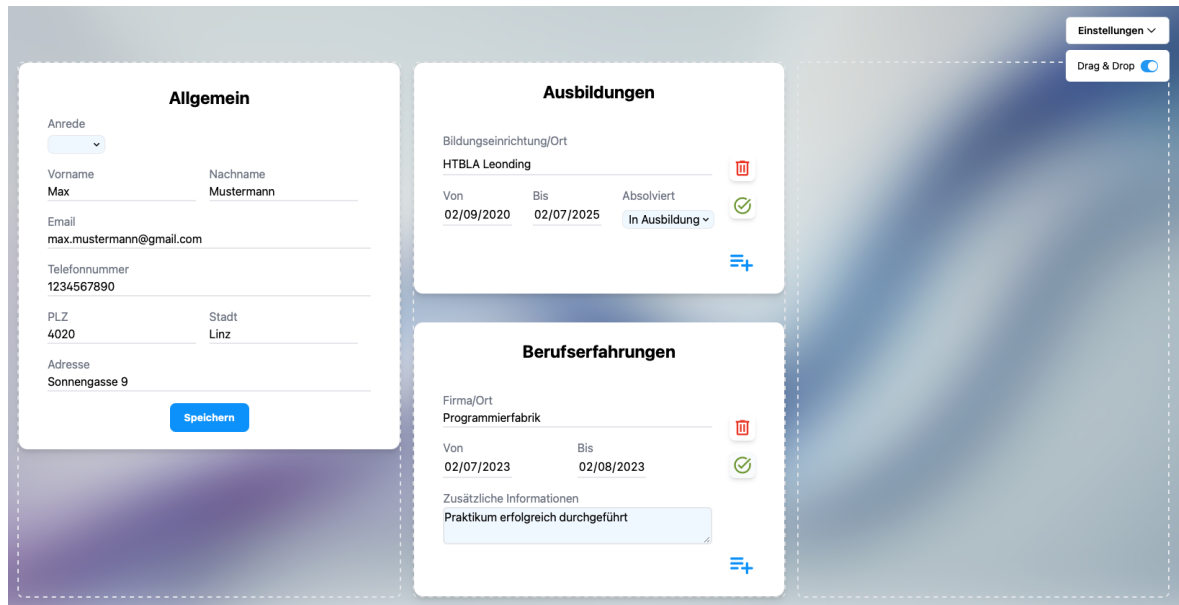


Abbildung 28: Verschiebung der Formulare mittels der Drag-and-Drop Funktion

Ein zentrales Merkmal der Website ist die Implementierung einer Drag-and-Drop-Funktion, welche es dem/der Benutzer:in ermöglicht, die Anordnung spezifischer Elemente wie Bilder und Formulare nach individuellen Präferenzen zu gestalten. Diese Funktion ist nicht nur auf der "Lebenslauf"-, sondern auch bei der "Projekte"- (Kunstwerken) sowie der "Kompetenzen"-Unterseite verfügbar. Ein besonderes Augenmerk wurde auf die Gewährleistung der Anpassungsfreiheit der Benutzeroberfläche gelegt, um den individuellen Bedürfnissen der Benutzer:innen gerecht zu werden.

Die Aktivierung der Drag-and-Drop-Funktion erfolgt über ein Dropdown-Menü. Bei Aktivierung werden weiße gestrichelte Hilfslinien um die verschiebbaren Elemente angezeigt, die die möglichen Positionen deutlich markieren. Gleichzeitig wird der Mauszeiger entsprechend angepasst, um anzuzeigen, dass die Formulare oder Elemente nun angeklickt und bewegt werden können.

Die visuelle Unterstützung erleichtert es dem/der Benutzer:in, die Anpassungsmöglichkeiten intuitiv zu erfassen und zu bedienen, ohne dass eine komplizierte Bedienung erforderlich ist. Die Drag-and-Drop-Funktion ermöglicht eine hohe Flexibilität und ein individuelles Nutzererlebnis, sei es beim Arrangieren von "Lebenslauf"-Abschnitten, der Neuordnung von Kunstwerken oder der Strukturierung von Kompetenzen.

4.3.5 Projekte

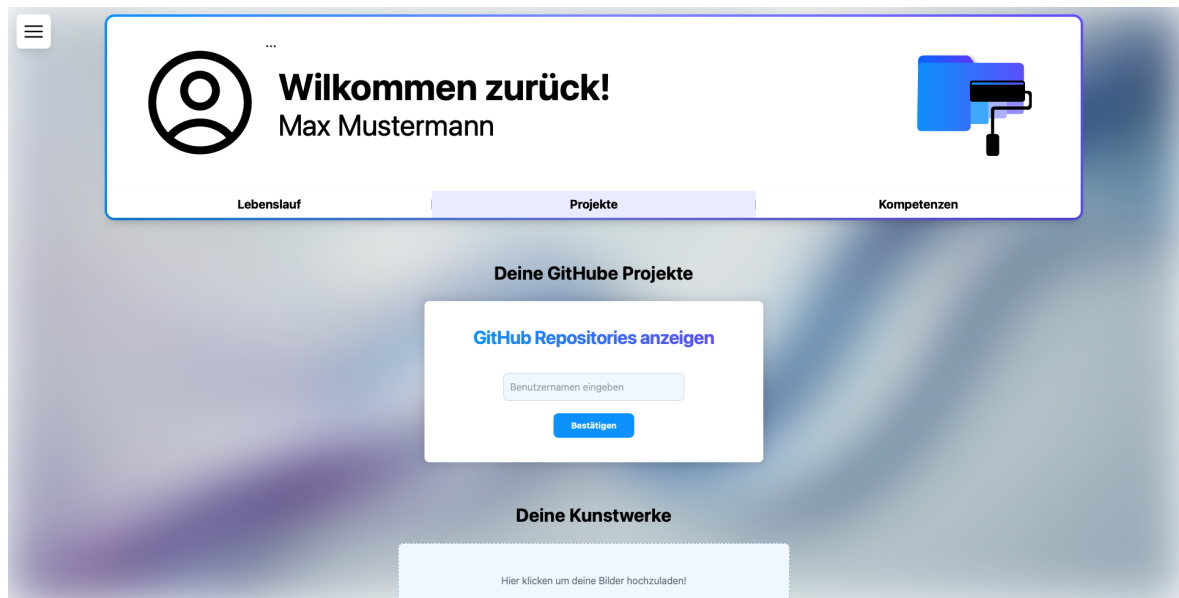


Abbildung 29: Unterseite Projekte

GitHub Projekte

Die "Projekte"-Seite von *Melius* ist ein Teilbereich, welche es ihren Nutzern ermöglicht, eigene technische und kreative Arbeiten auf einer persönlichen Webseite zu präsentieren. Die Seite ist in zwei Hauptbereiche unterteilt: die Integration von GitHub-Projekten und die Möglichkeit, multimediale Werke in Form von Bildern hochzuladen. GitHub-Projekte können über ein im oberen Bereich der Seite befindliches Formular hinzugefügt werden. Dazu ist die Eingabe des GitHub-Benutzernamens in ein Eingabefeld erforderlich, woraufhin die Schaltfläche "Bestätigen" zu betätigen ist. Nach erfolgter Eingabe wird eine Liste der öffentlichen Repositorys des jeweiligen GitHub-Kontos aufgerufen und auf der Seite dargestellt. Diese Funktionalität ermöglicht es, eigene technische Arbeiten direkt in das *Melius-Profil* zu integrieren. Dadurch entsteht ein nahtloser Übergang zwischen der persönlichen Webseite und den tatsächlichen Projekten auf GitHub.

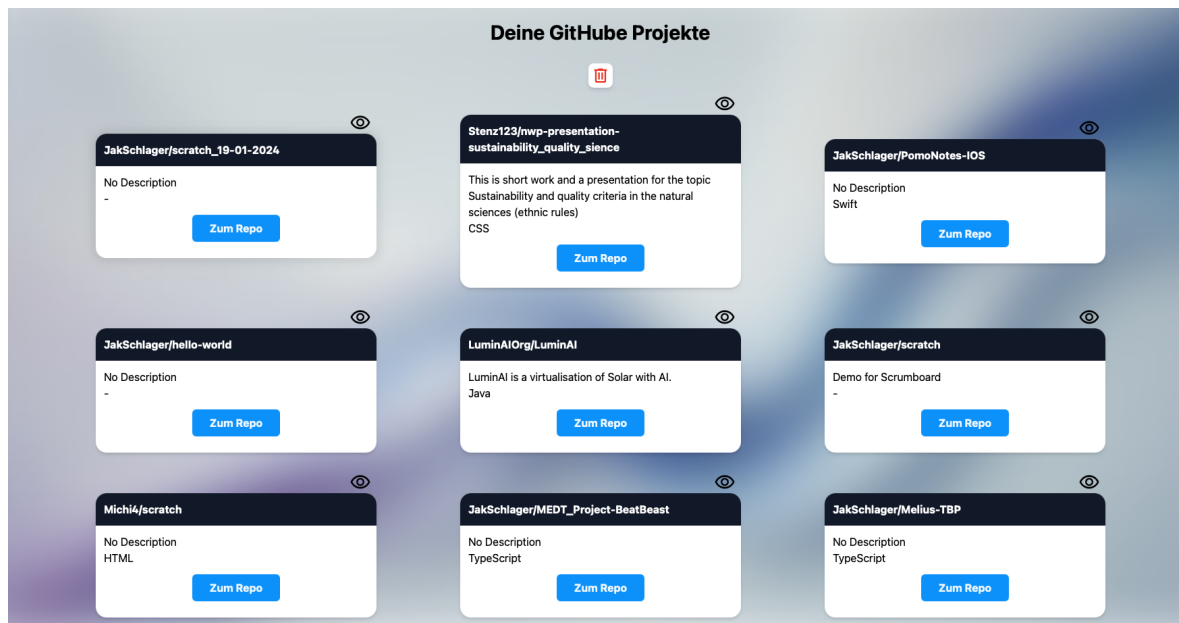


Abbildung 30: Gh Projekte angezeigt

Nach Eingabe des GitHub-Benutzernamens und Bestätigung der Eingabe erfolgt die übersichtliche Darstellung der zugehörigen Repositories in drei Spalten. Diese strukturierte Anordnung gewährleistet eine klare und aufgeräumte Präsentation der Projekte.

Am oberen Rand der Liste befindet sich ein schlichter “Remove”-Button mit einem Mülleimer-Symbol, mit dem die gesamte Liste der geladenen Repositories schnell und einfach entfernt werden kann.

Jedes einzelne Repository wird in einer eigenen Karte dargestellt, die folgende Informationen enthält:

- Eine Überschrift mit dem Repository-Namen, um das Projekt direkt zu identifizieren.
- Eine Beschreibung, die den Inhalt oder Zweck des Repositories näher erläutert.
- Ein Augen-Symbol dient als Steuerung für die Sichtbarkeit und kann durch Anklicken ausgeblendet oder bei erneutem Anklicken wieder eingeblendet werden.

Die durchdachte Struktur der Plattform ermöglicht eine einfache und flexible Verwaltung der eigenen GitHub-Projekte.

Präsentation künstlerischer Werke

Unterhalb der GitHub-Sektion besteht die Möglichkeit, Bilder hochzuladen, um künstlerische Arbeiten zu präsentieren. Dies bietet Nutzern:innen mit einem kreativen Hintergrund die Gelegenheit, ihre Werke in visuell ansprechender Weise darzustellen. Die

intuitive Bedienbarkeit, die durch eine Upload-Funktion gewährleistet wird, trägt dazu bei, dass die Schwelle für Nutzer:innen, ihre Inhalte auf *Melius* einzufügen, niedrig ist. Die Gestaltung und Benutzerführung der "Projekte"-Seite ist bewusst minimalistisch, um den Fokus auf die Inhalte der Nutzer zu legen. Überschriften und Buttons sind klar strukturiert, was eine intuitive Navigation ermöglicht. Die gewählten Schriftarten und Farben vermitteln eine moderne und professionelle Optik. Die Kombination aus technischer und kreativer Präsentation macht *Melius* zu einer Plattform, die für unterschiedlichste Nutzergruppen wie Studenten, Absolventen verschiedener Branchen, aber auch bereits Arbeitende attraktiv ist.

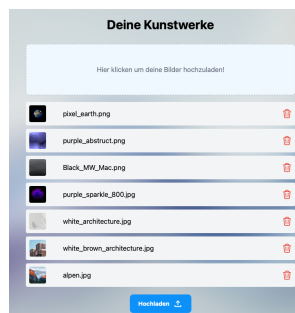


Abbildung 31: Liste der ausgewählten Bilder

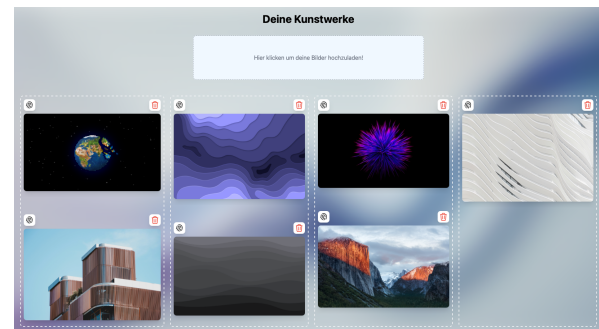


Abbildung 32: Ansicht der hochgeladenen Bilder

Nach dem Auswählen der gewünschten Bilder werden diese vor dem eigentlichen Upload in einer vorbereitenden Liste angezeigt. In dieser Liste wird jedes Bild in einer kleinen quadratischen Vorschau mit einer kurzen Beschreibung (Abbildung 31) dargestellt. Zusätzlich zu dieser Vorschaltanzeige ist ein "Remove"-Button integriert, mit dem Nutzer:innen unerwünschte Bilder noch vor dem Upload aus der Auswahl entfernen können. Diese Phase dient dazu, eine geordnete Auswahl zu ermöglichen, bevor die Bilder final hochgeladen werden.

Nach Abschluss des Hochladens erfolgt die Anzeige der Bilder in einem speziell dafür vorgesehenen Bereich, der als "Showcase" bezeichnet wird. Dieser besteht aus vier Spalten, in denen die hochgeladenen Bilder untereinander dargestellt werden. Das Design dieses Abschnitts ist so aufgebaut, dass eine flexible Anordnung der Werke möglich ist. Im "Showcase"-Bereich können Bilder per Drag-and-Drop zwischen den vier Spalten verschoben werden. Die Interaktion wird durch ein Fingerabdruck-Symbol initiiert, welches als Greifpunkt für die Drag-and-Drop-Funktion dient. Die Nutzerinnen und Nutzer haben dadurch die Möglichkeit, ihre Werke nachträglich umzuordnen und eine auf ihre Bedürfnisse zugeschnittene Präsentation zu erstellen. Zusätzlich ist in jeder Bildkarte ein Mülleimer-Symbol integriert, über welches hochgeladene Bilder bei Bedarf

wieder entfernt werden können. Die Anordnung dieser Bedienelemente gewährleistet, dass sowohl die Sortierung als auch das Löschen intuitiv und mit wenigen Klicks möglich ist. Die Kombination aus vorbereitender Bildliste und interaktivem Showcase resultiert in einem strukturierten und nutzerfreundlichen System zur Präsentation multimedialer Werke.

4.3.6 Kompetenzen

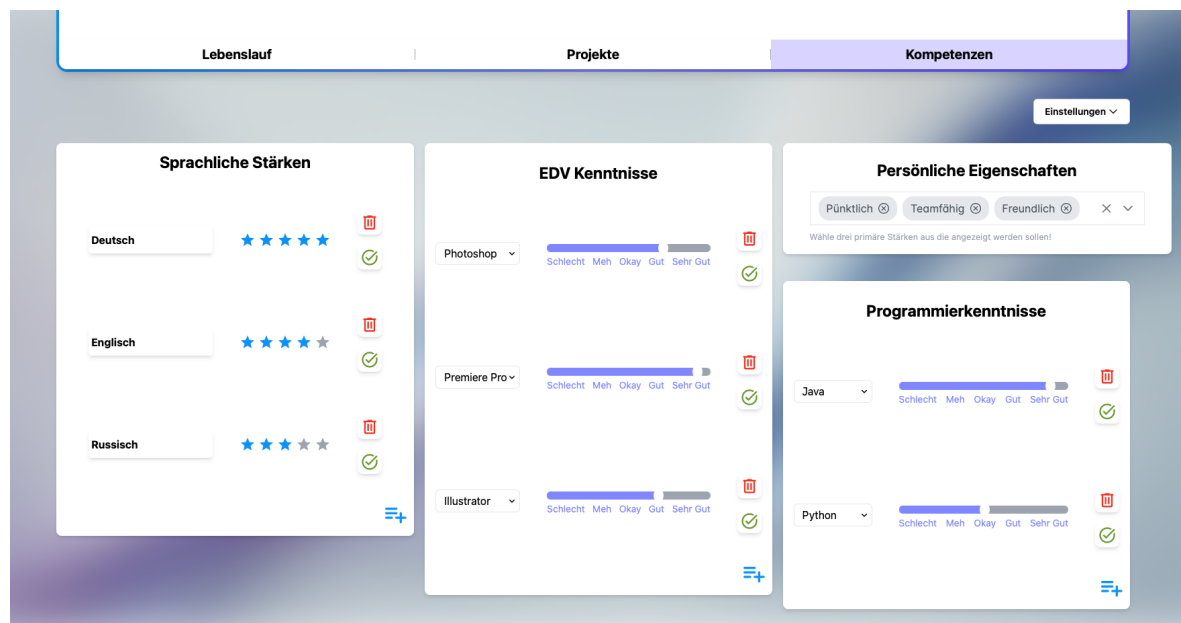


Abbildung 33: Unterseite Projekte

Die Kompetenzen sind in separaten Karten angeordnet, die verschiedene Fähigkeitsbereiche abdecken. Jede Karte besitzt eine eigene Überschrift und ermöglicht es den Nutzer:innen, ihre Kenntnisse und Stärken individuell anzupassen. Die Karten enthalten interaktive Elemente zur Auswahl, Bewertung und Verwaltung der Einträge.

Auch hier ist die Drag-and-Drop-Funktion ein zentrales Feature, mit der Nutzer:innen die Karten flexibel umsortieren können. Dies sorgt für eine personalisierte Darstellung der Kompetenzen und ermöglicht eine Anpassung der Reihenfolge je nach Relevanz.

Formularaufbau und Unterschiede

1. Sprachliche Stärken

- Nutzer:innen können verschiedene Sprachen bewerten. Jede Sprache wird als separater Eintrag dargestellt.
- Die Bewertung erfolgt über ein Sternesystem (1–5 Sterne), wobei mehr Sterne eine höhere Kompetenz anzeigen.

- Neben jeder Sprache befinden sich zwei interaktive Buttons:
 - Ein roter Papierkorb, um die Sprache zu löschen.
 - Ein grünes Häkchen, um die Eingabe zu bestätigen.
- Das Layout ist schlicht gehalten und konzentriert sich auf eine visuelle Bewertung durch Sterne, ohne weitere Details oder Textbeschreibungen.

2. EDV-Kenntnisse | Programmierkenntnisse

- Diese beiden Kategorien sind strukturell identisch, unterscheiden sich jedoch in der Art der wählbaren Einträge.
- Nutzer:innen wählen eine Software oder Programmiersprache über ein Dropdown-Menü (z. B. „Photoshop“, „Java“).
- Die Bewertung erfolgt über einen horizontalen Fortschrittsbalken mit fünf Stufen:
 - Schlecht – Meh – Okay – Gut – Sehr Gut
- Die Balken sind farblich abgestuft, um die unterschiedlichen Kompetenzniveaus optisch hervorzuheben.
- Jeder Eintrag verfügt über die gleichen interaktiven Buttons wie bei den Sprachkenntnissen (Löschen Bestätigung).

3. Persönliche Eigenschaften

- Statt einer Bewertungsskala wählen Nutzer:innen hier aus einer Liste vordefinierter Eigenschaften (z. B. „Pünktlich“, „Teamfähig“, „Freundlich“).
- Die gewählten Eigenschaften werden als Tags dargestellt, die per Klick entfernt werden können.
- Dieses Formular verzichtet auf Lösch- oder Bestätigungsbuttons für einzelne Einträge, da die Anpassung ausschließlich über die Auswahl der Tags erfolgt.

4.3.7 Die Gruppen-Seite

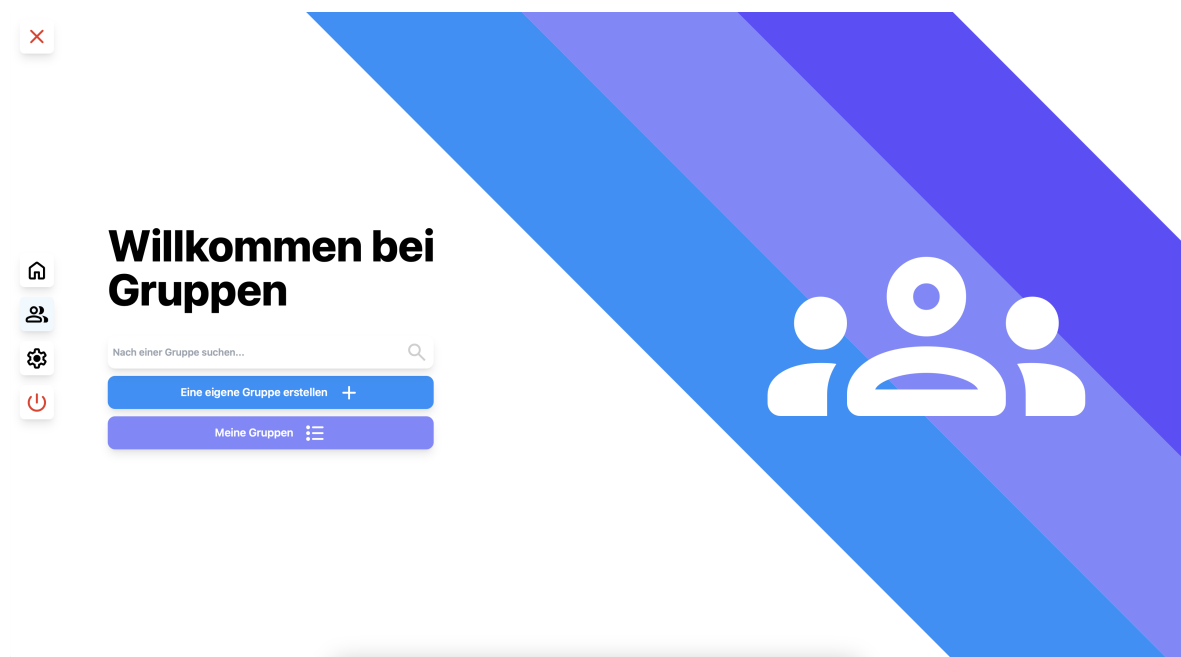


Abbildung 34: Landing Page der Gruppenseite

Die Gruppen-Landing-Page von *Melius* zeichnet sich durch eine intuitive und moderne Benutzererfahrung aus, die den Community-Gedanken der Plattform unterstreicht. Sie ermöglicht es Nutzer:innen, neue Gruppen effizient zu erstellen oder bestehenden Gruppen beizutreten. Hinsichtlich des Designs und der Struktur ist festzustellen, dass die linke Seite der Seite den Nutzer mit der klaren Botschaft “Willkommen bei Gruppen” begrüßt. Darunter befindet sich eine Suchleiste, mit der Nutzer gezielt nach Gruppen suchen können. Diese Funktion erleichtert den Einstieg und ermöglicht eine direkte Navigation.

Darunter befinden sich zwei zentrale **Call-to-Action-Buttons**:

- Der blaue Button mit der Beschriftung “Eine eigene Gruppe erstellen” ermöglicht den schnellen Aufbau einer neuen Gruppe innerhalb der Plattform.
- Der lila Button mit der Beschriftung "Meine Gruppen" führt den/die Nutzer:in zu seinen bereits erstellten oder beigetretenen Gruppen.

Die visuelle Gestaltung der Plattform ist auf eine harmonische Ästhetik ausgerichtet, die eine visuelle Wiedererkennung ermöglicht. Auf der rechten Seite der Startseite befindet sich ein großes, stilisiertes Gruppen-Icon, das die Idee der Vernetzung symbolisiert. Der Hintergrund ist mit diagonalen Farbverläufen in Blau- und Lilatönen gestaltet, die sich an den Primärfarben der Applikation orientieren.

Die Seite ist übersichtlich gestaltet, sodass sich Nutzer:innen intuitiv zurechtfinden. Dies ist ein Aspekt der Funktionalität. Die zentral platzierten Buttons ermöglichen eine schnelle und unkomplizierte Nutzung der Gruppenfunktionen.

Die Integration einer **Suchleiste** erleichtert es, gezielt nach bestehenden Gruppen zu suchen.

Suchleiste

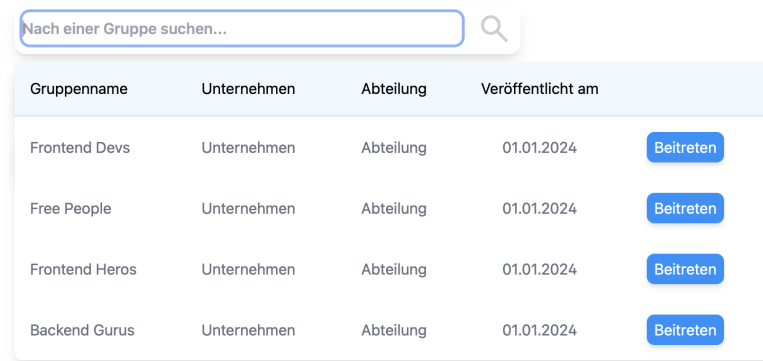


Abbildung 35: Suchergebnisse

Die Suchfunktion ermöglicht es den Nutzern, gezielt nach Gruppen zu suchen. Beim Eintippen eines Gruppennamens filtert das System automatisch die verfügbaren Gruppen und zeigt passende Ergebnisse an. Sollte keine Gruppe mit dem eingegebenen Namen existieren, wird eine entsprechende Meldung angezeigt. Die Ergebnisliste besteht aus einer Tabelle mit folgenden Spalten:

- der Gruppenname
- das dazugehörige Unternehmen
- die spezifische Abteilung innerhalb des Unternehmens
- das Erstellungsdatum der Gruppe

In der Nähe jeder Gruppe befindet sich ein blauer Button mit der Beschriftung "Beitreten". Dieser führt den/die Benutzer:in zur nächsten Ansicht, in der ein Beitritt zur Gruppe möglich ist.

The image shows a web interface for joining a group. At the top left is a back arrow. To its right is a circular profile picture placeholder. Further right, the group name 'Frontend Devs' is displayed in a bold, black font. Below the group name, a message states: 'Bevor du der Gruppe Frontend Devs beitreten kannst, musst du noch einen bestimmten Einschreibeschlüssel eingeben.' (Before you can join the group Frontend Devs, you still need to enter a specific sign-up key). Underneath this message is a text input field with the placeholder text 'Einschreibeschlüssel eingeben...'. Below the input field is a blue button with the text 'Gruppe beitreten' (Join group). The entire form is enclosed in a light gray border with blue decorative elements at the corners.

Abbildung 36: Eingabeformular zum beitreten einer Gruppe

Sobald eine Gruppe ausgewählt wurde, erfolgt eine Weiterleitung zur Beitrittsansicht. Hier wird er aufgefordert, einen Einschreibeschlüssel einzugeben, um der Gruppe beizutreten.

Diese Sicherheitsmaßnahme stellt sicher, dass nur autorisierte Personen Zugang zu bestimmten Gruppen erhalten. Dies kann besonders in geschlossenen Unternehmensgruppen oder spezifischen Fachgruppen wichtig sein, um eine kontrollierte Mitgliederstruktur zu gewährleisten.

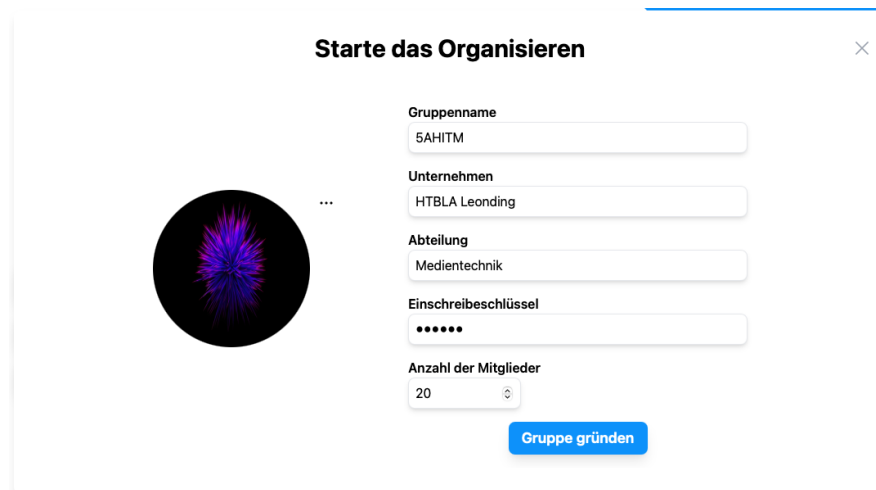
Das Design dieser Ansicht ist minimalistisch und auf eine klare Benutzerführung ausgerichtet:

- Der Gruppenname wird prominent in großer, fetter Schrift dargestellt, sodass der Nutzer eindeutig erkennen kann, welcher Gruppe er beitreten möchte.
- Direkt darunter befindet sich eine kurze Erklärung, dass ein Einschreibeschlüssel erforderlich ist. Diese Anleitung hilft, Verwirrung zu vermeiden und macht deutlich, dass ohne den Schlüssel kein Beitritt möglich ist.
- Ein Eingabefeld erlaubt es dem Nutzer, den Einschreibeschlüssel einzugeben. Das Feld ist dezent gestaltet und vermittelt durch den Platzhaltertext "Einschreibeschlüssel eingeben..." eine klare Anweisung.
- Der "Gruppe beitreten"-Button hebt sich durch seine kräftige blaue Farbe deutlich von der restlichen Oberfläche ab. Sobald der/die Nutzer:in den Schlüssel eingegeben hat, kann er diesen Button klicken, um der Gruppe beizutreten.

Falls der Einschreibeschlüssel korrekt ist, wird der/die Nutzer:in erfolgreich zur Gruppe hinzugefügt und erhält Zugriff auf deren Inhalte und Mitglieder. Sollte der Schlüssel falsch sein, kann das System eine Fehlermeldung ausgeben, um darauf hinzuweisen.

Diese Beitrittsfunktion stellt sicher dass die Gruppen geschützt bleiben, während gleichzeitig eine einfache Möglichkeit besteht, neue Mitglieder aufzunehmen.

Gruppen erstellen



The screenshot shows a modal window titled "Starte das Organisieren" with a close button (X) in the top right corner. On the left side of the modal is a circular profile picture placeholder containing a purple, starburst-like graphic. To the right of the image is an ellipsis menu icon (...). The right side of the modal contains a form with the following fields:

- Gruppenname**: A text input field containing "5AHITM".
- Unternehmen**: A text input field containing "HTBLA Leonding".
- Abteilung**: A text input field containing "Medientechnik".
- Einschreibeschlüssel**: A text input field containing six dots (••••••).
- Anzahl der Mitglieder**: A numeric input field with a spinner, currently set to "20".

At the bottom right of the form is a blue button labeled "Gruppe gründen".

Abbildung 37: Eigene Gruppen erstellen

Betätigt man den Button "Eine Gruppe erstellen +", so öffnet sich ein PopUp, das es dem/der Nutzer:in ermöglicht, eine eigene Gruppe zu erstellen. In dem Formular werden alle relevanten Informationen für die Gruppenerstellung erfasst. Gleichzeitig besteht die Möglichkeit, das Gruppenprofilbild visuell anzupassen.

Design visuelle Aufteilung

Das Formular präsentiert sich als zentriertes Popup-Fenster, das sich über der bestehenden Gruppen-Seite öffnet. Die Gestaltung ist in zwei Hauptbereiche unterteilt:

Visueller Bereich (links - Abbildung 37)

- Der visuelle Bereich auf der linken Seite des Formulars enthält ein rundes Gruppenprofilbild mit einem Platzhalterbild. Ähnlich aufgebaut wie bei dem Profilbild des/der Benutzer:in hat man auf Rechts neben dem Bild ein kleines Menü, das die Optionen zum Ändern, Löschen oder Hochladen eines neuen Profilbildes bietet. Die Bilddarstellung trägt zur klaren visuellen Differenzierung zwischen den verschiedenen Gruppen sowie zur einfachen Personalisierung bei.

Formular Bereich (rechts - Abbildung 37)

- Der Formularbereich auf der rechten Seite besteht aus mehreren logisch untereinander angeordneten Eingabefeldern. Die Eingabefelder sind schlicht und minimalistisch gehalten, mit abgerundeten Ecken und einer klaren Schrift für bessere Lesbarkeit. Beschriftungen über den Eingabefeldern stellen sicher, dass der Nutzer weiß, welche Information jeweils eingetragen werden muss.

Funktionale Komponenten des Formulars

- Gruppenname: Freitextfeld, in das der Nutzer den Namen seiner Gruppe eingeben kann.
- Unternehmen: Eingabefeld zur Angabe der zugehörigen Organisation oder Firma.
- Abteilung: Ermöglicht eine detailliertere Zuordnung innerhalb eines Unternehmens.
- Einschreibeschlüssel: Passwortfeld zur Eingabe eines Beitrittsschlüssels für User um der Gruppe später beitreten zu können. Wichtig hier ist dass das Passwort mindestens 6 Zeichen lang sein muss.
- Anzahl der Mitglieder: Drop-down-Menü, mit dem die maximale Mitgliederanzahl definiert werden kann. Auch hier ist es wieder wichtig zu wissen, dass man keine Gruppe mit weniger als 2 Personen erstellen kann.
- „Gruppe gründen“-Button: Zu guter letzt, ein in Blau deutlich hervorgehobener Button, damit die Gruppe final erstellt werden kann.

Das Popup-Fenster gewährleistet, dass Nutzer:innen nicht die aktuelle Seite verlassen müssen, sondern direkt zur Gruppenerstellung zurückkehren können. Das runde Profilbild und die optionale Personalisierung tragen zur visuellen Unterscheidbarkeit der Gruppen bei. Die Anordnung der Felder folgt einer logischen Reihenfolge, sodass Nutzer:innen ohne Umwege alle notwendigen Informationen eingeben können.

Meine Gruppen



Abbildung 38: Meine Gruppen

Wird eine Gruppe angelegt, so kann sie nach Betätigung des “Meine Gruppen”-Buttons in einer separaten Liste angezeigt werden. Es wird eine Auflistung aller von dem/der Benutzer:in erstellten Gruppen angezeigt, wie sie auch bei den separaten Suchergebnissen zu sehen ist. Diese können entweder wieder entfernt werden, wenn sie nicht mehr benötigt werden, oder durch Betätigung von **Ansehen** in der Admin-Ansicht der Gruppe nachträglich bearbeitet werden.

Admin Ansicht einer Gruppe

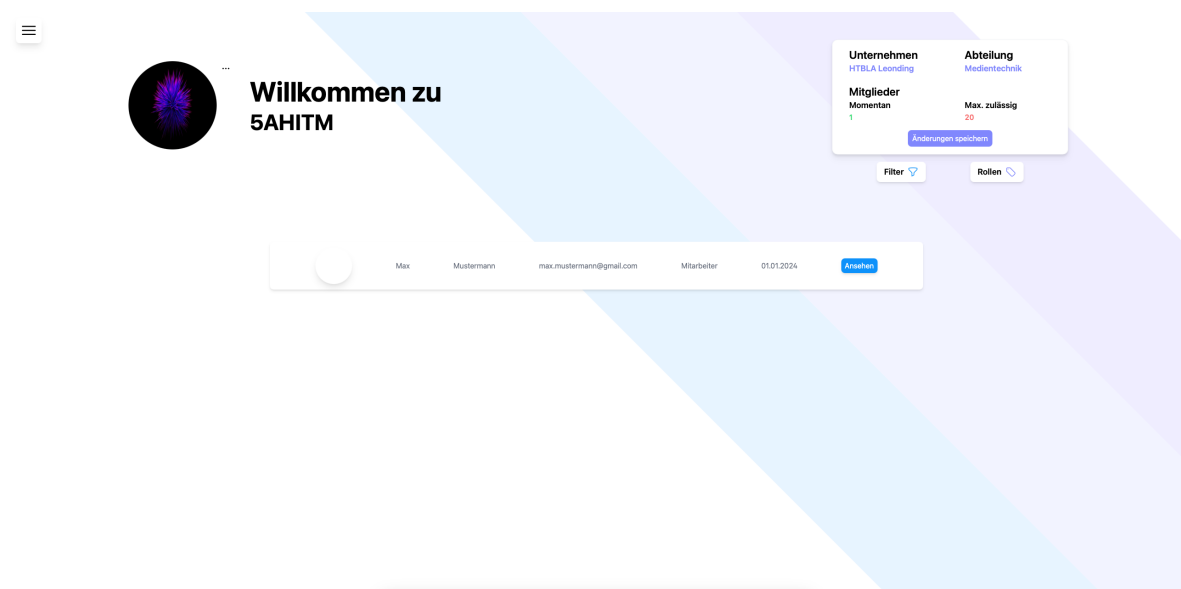


Abbildung 39: Admin Ansicht

Die Gruppen-Admin-Seite von *Melius* ist durch ein minimalistisches und strukturiertes Design gekennzeichnet, das eine intuitive Verwaltung von Gruppen und Mitgliedern

ermöglicht. Die klare Trennung von Inhalten, die durchdachte Farbgestaltung und die gezielten Interaktionsmöglichkeiten bieten Administratoren eine effiziente Möglichkeit, ihre Gruppen zu organisieren.

Das Interface präsentiert sich in einem klaren, hellen Stil, der durch dezente Farbverläufe in den Primärfarben von Melius im Hintergrund eine zeitgemäße Ästhetik aufbaut. Diese Farbgebung fördert nicht nur eine angenehme Lesbarkeit, sondern hebt auch verschiedene Bereiche der Seite subtil hervor.

Das Design der Admin-Ansicht folgt den Prinzipien der Klarheit und Benutzerfreundlichkeit. Die Farbpalette ist bewusst auf ein schlichtes, professionelles Weiß mit sanften blauen und violetten Akzenten reduziert, was für eine angenehme, nicht überladene Optik sorgt. Große, gut lesbare Schriften und ausreichend Abstände zwischen den einzelnen Elementen tragen dazu bei, dass alle Inhalte leicht erfassbar sind.

Gruppen Informationen

In der oberen rechten Ecke (Abbildung 39) befindet sich eine kompakte Box, die zentrale Informationen über die Gruppe zusammenfasst. Diese enthält:

- Den Namen des Unternehmens (in diesem Fall "HTBLA Leonding") mit einer farblichen Darstellung, die eine direkte Navigation zu relevanten Informationen ermöglicht.
- Die zugehörige Abteilung ("Medientechnik") ist ebenfalls farblich markiert.

Eine Übersicht über die Mitgliederzahl, wobei die aktuelle Anzahl der Mitglieder farblich hervorgehoben wird:

- Die aktuelle Anzahl ist in grün gehalten, um eine positive, aktive Darstellung zu gewährleisten.
- Die maximal zulässige Mitgliederzahl ist in rot formatiert, um Administratoren darauf hinzuweisen, falls eine Grenze erreicht oder überschritten wird.

Direkt darunter befindet sich ein gut sichtbarer "Änderungen speichern"-Button, mit dem Administratoren Modifikationen an der Gruppe sichern können. Der Button ist in einem klaren, auffälligen Blau gehalten, sodass er als primäre Handlungsaufforderung wahrgenommen wird.

Mitgliederverwaltung

Ein zentrales Element der Seite ist die übersichtliche Liste der Gruppenmitglieder. Diese

wird in einer horizontalen Zeile dargestellt und bietet alle relevanten Informationen in einer aufgeräumten Form. Für jedes Mitglied werden folgende Daten angezeigt:

- Ein rundes Profilbild (bei fehlendem Bild ein Platzhalter-Icon), das für eine persönliche Identifikation sorgt.
- Der Vor- und Nachname des Mitglieds in einer klaren Typografie.
- Die hinterlegte E-Mail-Adresse, die eine direkte Kontaktaufnahme ermöglicht.
- Die Rolle des Nutzers innerhalb der Gruppe (z. B. „Mitarbeiter“), was eine schnelle Einordnung der Zuständigkeiten erleichtert.
- Das Beitrittsdatum, das den Zeitpunkt der Mitgliedschaft anzeigt.
- Ein „Ansehen“-Button, der in einem kräftigen Blau gehalten ist, um auf zusätzliche Details des Nutzers zuzugreifen.

Filter Methoden und Rollen

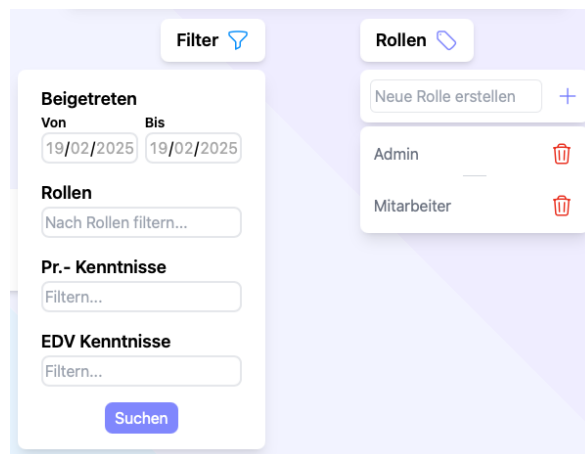


Abbildung 40: Filter-Methode und Rollen

Filter-Bereich (links - Abbildung 40) Dieser Bereich ist als Dropdown-Menü gestaltet, das sich nach einem Klick auf den „Filter“-Button mit einem Trichter-Symbol öffnet. Das Design nutzt eine klare Struktur mit Eingabefeldern, um verschiedene Filterkriterien festzulegen.

Enthält folgende interaktive Elemente:

- Beitrittsdatum (Von/Bis): Auswahlfelder mit einem deaktivierten Zustand, die derzeit auf den 19.02.2025 gesetzt sind.
- Rollen-Filter: Ein Suchfeld zur Eingabe spezifischer Rollen.

- Pr.-Kenntnisse EDV-Kenntnisse: Jeweils eigene Filterfelder zur gezielten Suche nach Mitarbeitern mit bestimmten Fähigkeiten.
- „Suchen“-Button: Auffällig in Lila, um die Filteraktion klar erkennbar zu machen.

Rollen-Verwaltung (rechts - Abbildung 40) Ebenfalls als Dropdown-Menü gestaltet, das nach einem Klick auf den „Rollen“-Button mit einem Tag-Symbol erscheint.

Der Bereich enthält:

- Ein Eingabefeld zur Erstellung neuer Rollen, das in einem hellgrauen Zustand ist.
- Ein „+“-Button, um eine neue Rolle hinzuzufügen.
- Liste bestehender Rollen (z. B. „Admin“ und „Mitarbeiter“), die in einem schlichten, zeilenweisen Format angezeigt werden.
- Rote Papierkorb-Symbole, die klar als Lösch-Icons erkennbar sind.

Die Interaktionselemente sind konsistent gestaltet: Primäre Aktionen werden in Blau hervorgehoben, während informative Elemente in Schwarz oder Grau dargestellt werden. Diese Farbkodierung erzeugt eine klare visuelle Hierarchie und erleichtert die Orientierung.

Gast Ansicht einer Gruppe

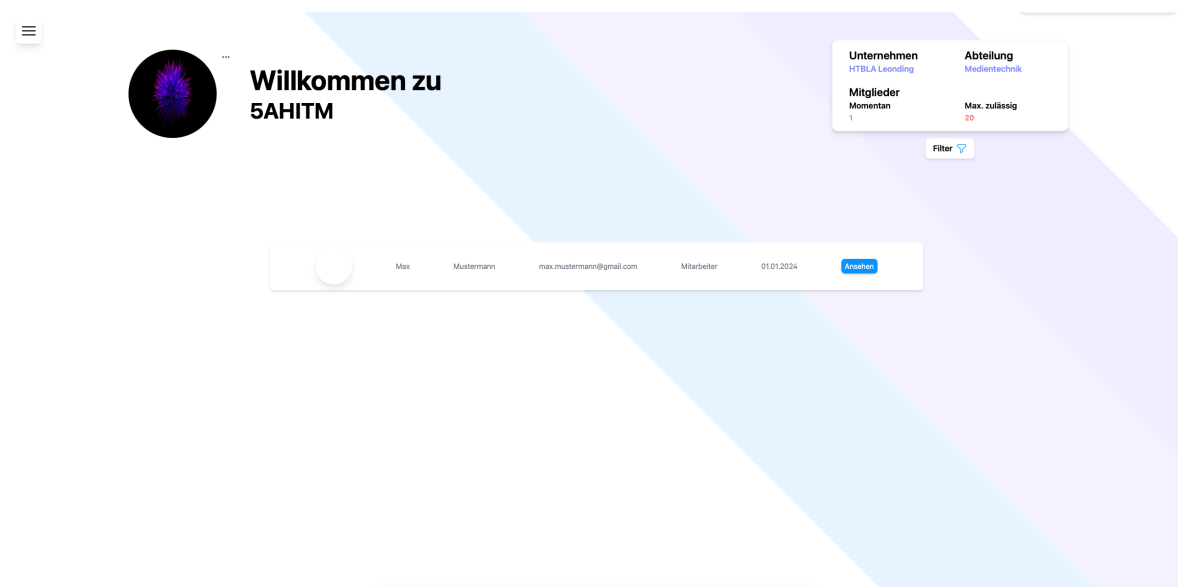


Abbildung 41: Gast Ansicht

Im Falle einer Aufnahme in eine Gruppe ist die Ansicht des Gastes in den meisten Fällen deckungsgleich mit der des/der Administrators:in. Die einzigen Abweichungen zur Admin-Ansicht sind darin begründet, dass folgende Veränderungen nicht vorgenommen werden können:

- Änderungen an der maximalen Gruppenmitglieder
- Änderungen am Profilbild
- Rollen erstellen oder löschen

Dennoch besteht für jeden Gast die Möglichkeit, eine Filterung nach verschiedenen Benutzern:innen vorzunehmen.

4.3.8 Die Einstellungen

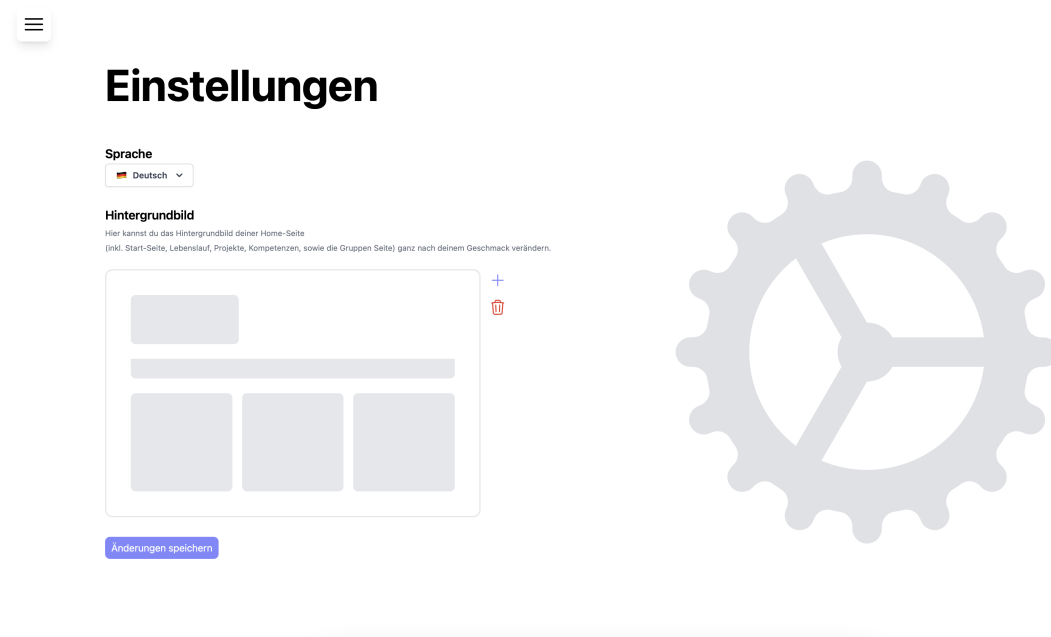


Abbildung 42: Einstellungen

Bei der erstmaligen Öffnung der Einstellungs-Seite von *Melius* fällt die aufgeräumte und übersichtliche Gestaltung des Layouts ins Auge, die eine klare Struktur vermittelt. Die Benutzer:innen werden somit in die Lage versetzt, die verfügbaren Einstellungsmöglichkeiten unmittelbar zu erfassen. Die Farbgestaltung ist angenehm dezent gehalten und lenkt den Blick auf die wesentlichen Elemente, insbesondere die Überschriften, die Auswahlfelder und den großen Button zum Speichern. Der Seitentitel “Einstellungen” ist oben in einer großen Schrift zu sehen, was sofort klar macht, dass es sich um den Bereich der persönlichen Anpassungen handelt. Das Hauptmenü kann über ein in der oberen

linken Ecke platziertes Icon, das die Form eines “Hamburger”-Symbols aufweist, ein- und ausgeblendet werden, sodass die Navigation auf der Seite stets leicht erreichbar bleibt. Die gewählte Schriftart (*Geist*) sorgt für klaren Kontrast und eine gute Lesbarkeit, was zu einer möglichst einfachen Bedienung in der gesamten Applikation beiträgt.

Auswahl der Sprache

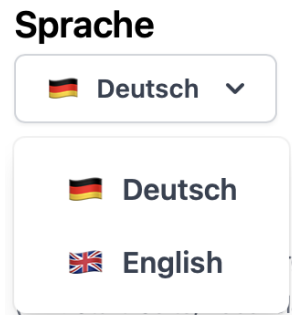


Abbildung 43: Einstellungen

In der oberen linken Ecke des Hauptbereichs, der sich direkt unter der Überschrift befindet, ist ein Dropdown-Menü, in dem die derzeit verfügbaren Sprachen Deutsch und Englisch präsentiert werden. Neben den jeweiligen Sprachbezeichnungen werden kleine Flaggen angezeigt, sodass die Auswahl nicht nur sprachlich, sondern auch visuell unterstützt wird. Dies trägt zu einer schnellen Orientierung bei, insbesondere für Nutzer:innen, die sich mit fremdsprachlichen Texten weniger sicher fühlen.

Durch Anklicken des Feldes, in dem standardmäßig “Deutsch” ausgewählt ist, öffnet sich das Dropdown-Menü. Sofort werden die verfügbaren Optionen sichtbar, in diesem Fall “Deutsch” und “English”. Das Design zeichnet sich durch einen weißen Hintergrund, eine klare Typografie und ausreichend Weißraum aus, sodass die einzelnen Optionen deutlich voneinander getrennt sind.

Ein Wechsel der Spracheinstellung wird durch Betätigung der Schaltfläche “Änderungen speichern” initiiert, woraufhin die Seite kurz neu lädt und die ausgewählte Spracheinstellung auf der gesamten Website übernommen wird. Aufgrund der schlanken und modernen Gestaltung des Designs wird dieser Prozess als angenehm flüssig wahrgenommen.

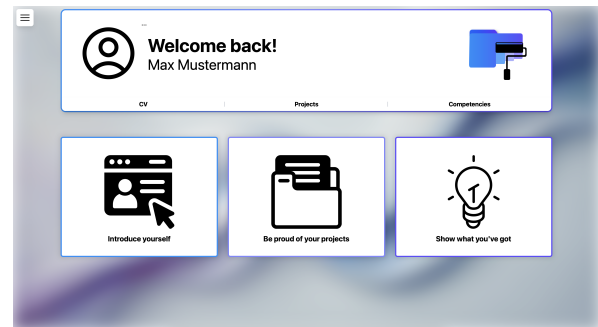
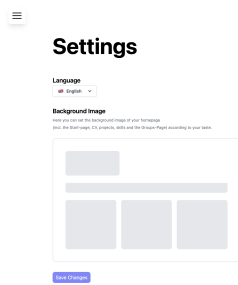


Abbildung 44: Einstellungen in Englisch

Abbildung 45: Startseite in Englisch

Auswahl des Hintergrundes

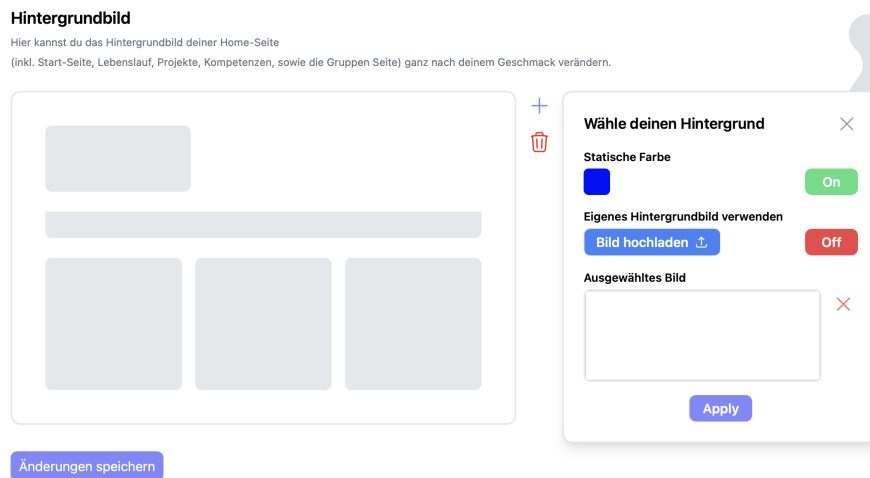


Abbildung 46: Formular zum Auswählen der Hintergrundfarbe oder -bild

Ein besonders auffälliger Teil der Einstellungsseite ist der Bereich "Hintergrundbild". In einem großzügig dimensionierten Kasten, der sich auf der linken Seite (Abbildung 46) der Seite befindet, wird eine vereinfachte Vorschau der Startseite angezeigt. Dadurch wird es den Nutzer:innen ermöglicht, sich ein Bild davon zu machen, wie das Hintergrundbild auf ihrer Seite wirken wird.

Die Nutzer:innen haben die Auswahlmöglichkeit zwischen zwei Hauptoptionen:

1. Statische Farbe

- Über einen Schalter, der zwischen "an" und "aus" schaltet, lässt sich eine einfarbige Hinterlegung für die Seite festlegen. Dies kann zum Beispiel ein neutrales Grau, ein dezentes Pastell oder jede andere Wunschfarbe sein.

2. Eigenes Hintergrundbild

- Auch hier findet sich ein Schalter, der bei Bedarf aktiviert werden kann, um ein individuelles Bild hochzuladen.

Die Trennung der Optionen in zwei Schalter (Toggles) gewährleistet eine intuitive Benutzerführung, da Nutzer:innen unmittelbar erkennen können, welche Option aktiv ist und welche Funktion verfügbar ist. Es ist jedoch zu beachten, dass stets nur eine Option ausgewählt sein kann.

Wird die Option “Eigenes Hintergrundbild verwenden” aktiviert und anschließend auf den Button mit der Beschriftung "Bild hochladen" geklickt, öffnet sich ein Dialog- oder Datei-Auswahlfenster, in dem die Nutzer:innen das gewünschte Bild von ihrem Gerät auswählen können. Nach dem Upload wird das “Ausgewähltes Bild”-Feld sichtbar bzw. aktualisiert. In diesem Bereich wird dem/der Nutzer:in zudem eine Vorschau des ausgewählten Hintergrunds angezeigt, um sicherzustellen, dass das intendierte Motiv korrekt ausgewählt wurde. Sollte dies nicht der Fall sein, besteht die Möglichkeit, das Bild wieder zu entfernen oder eine andere Option zu wählen. Hierzu steht ein “X”-Button zur Verfügung, mit dem das ausgewählte Bild entfernt werden kann.

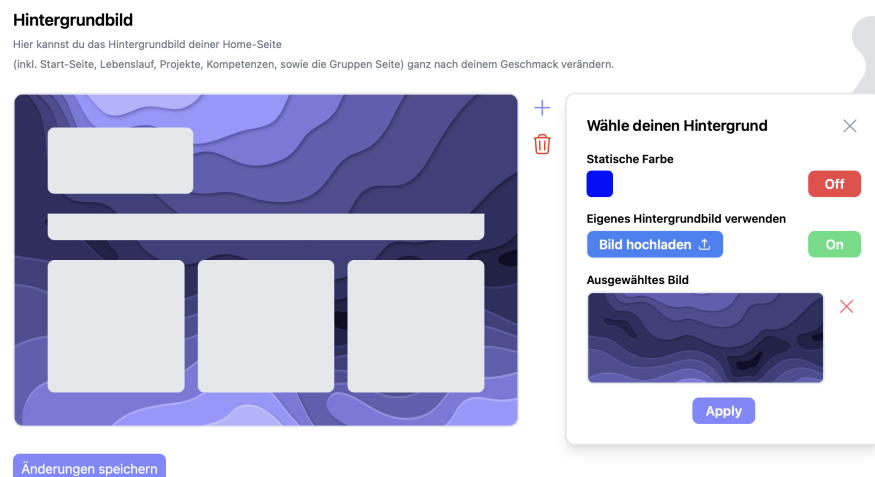


Abbildung 47: Hintergrund wurde ausgewählt

Unterhalb der Optionen befindet sich ein Button mit der Beschriftung “Apply”, der eine unmittelbare Anwendung der getroffenen Auswahl (statische Farbe oder eigenes Bild) auf der Vorschau ermöglicht. Das unmittelbare Feedback im Interface trägt dazu bei, das Design des eigenen Profils oder der Startseite zügig und unkompliziert zu überprüfen. Auf diese Weise können:innen rasch entscheiden, ob sie das hochgeladene Bild behalten oder eine andere Farbe oder ein anderes Foto ausprobieren möchten.

5 Umsetzung

5.1 Frontend - Jakob Schlager

5.1.1 Entwicklungsumgebung



Abbildung 48: Webstorm - JetBrains

Für die Realisierung des Frontends wurde WebStorm als primäre Entwicklungsumgebung ausgewählt, da es eine spezifische Anpassung an JavaScript, TypeScript und moderne Frontend-Frameworks aufweist. Die Fokussierung auf Web-Technologien gewährleistet eine nahtlose Integration zentraler Funktionen wie Code-Vervollständigung, Linter-Integration und automatische Projektkonfigurationen. Insbesondere bei spezifischeren Anwendungen, die verschiedene Frameworks und Bibliotheken umfassen, demonstriert WebStorm seine Stärken und beschleunigt den Entwicklungsprozess signifikant.

Erwähnenswert sind zudem die integrierten Debugging- und Testfunktionen, die das Ausführen und Überprüfen von Frontend-Code sowohl im Browser als auch in Node.js-Umgebungen erleichtern. Darüber hinaus bietet WebStorm eine umfassende Unterstützung für gängige Tools wie Prettier, ESLint oder Jest, sodass sich Qualitäts- und Stilvorgaben direkt in den Entwicklungsprozess einbinden lassen. Dies gewährleistet eine durchgängig hohe Qualität des Codes und dessen Lesbarkeit. Ein weiterer entscheidender Aspekt ist die Integration der Versionskontrolle. Die direkte Anbindung an Git und andere Versionsverwaltungssysteme ermöglicht dem gesamten Team eine schnelle Nachverfolgung, Merge und Rückgängigmachung von Änderungen im Code. Durch das reibungslose Zusammenspiel von Code-Editor, Debugger, Versionskontrolle

und Projektkonfiguration bietet WebStorm eine umfassende All-in-One-Lösung, die den Bedarf an zusätzlichen Tools stark reduziert.

Ein weiterer Vorteil von WebStorm ist seine stabile Performance, die auch bei großen Projekten eine zügige und übersichtliche Bearbeitung ermöglicht. Zudem bietet WebStorm die Möglichkeit, neue Plugins oder Erweiterungen einzubinden, die die IDE um zusätzliche Funktionen ergänzen. Die automatischen Vorschläge für Refactorings oder Code-Optimierungen unterstützen das Team dabei, den Code kontinuierlich zu verbessern und eine saubere Projektstruktur beizubehalten.

Die Entscheidung beruhte auf zahlreichen Vorteilen, die WebStorm speziell für die Frontend-Entwicklung bietet. Hier sind die wesentlichen Gründe, die diese IDE zur idealen Wahl machen:

Intelligente Code-Vervollständigung und Refactoring:

- Die IDE bietet fortschrittliche Funktionen zur Code-Vervollständigung, automatischen Fehlererkennung und Refactoring-Tools. Dadurch wird nicht nur die Produktivität gesteigert, sondern auch die Code-Qualität nachhaltig verbessert.

Nahtlose Versionskontrolle:

- Die direkte Integration von Git und anderen Versionsverwaltungssystemen erleichtert das Verwalten von Code-Änderungen und unterstützt kollaboratives Arbeiten im Team.

Anpassungsfähigkeit und Erweiterbarkeit:

- Dank einer Vielzahl von Plugins und Erweiterungen lässt sich WebStorm individuell an die spezifischen Bedürfnisse eines Projekts anpassen. Dies schafft eine maßgeschneiderte Entwicklungsumgebung.

Vertrautheit durch die JetBrains Toolbox:

- Das Team ist bereits mit anderen JetBrains-Produkten vertraut, was den Einstieg in WebStorm erleichtert und für einen reibungslosen Arbeitsablauf sorgt.

5.1.2 Anmeldung und Registrierung - Form Validation

register-form.html



Abbildung 49: Eingabefelder der FormGroup

In diesem Ausschnitt sieht man das HTML-Template für das Formular. Das saveForm wird über `[formGroup]="saveForm"` eingebunden. Jedes Eingabefeld ist mit `formControlName` an ein entsprechendes FormControl in der TypeScript-Datei gebunden. Zusätzlich besitzt jedes Feld, das validiert wird, ein entsprechendes `*ngIf`, das nur dann eine Fehlermeldung anzeigt, wenn die jeweilige Validierung fehlschlägt (z. B. bei `Validators.required`, `Validators.email` usw.). Auch die eigenen erstellen Validations wie beispielsweise für das Überprüfen ob das Passwort erneut richtig eingegeben wurde, wird hier mittels `*ngIf` überprüft. Hier wird dann ein einfaches `saveForm.hasError('passwordDoesNotMatch')` zusätzlich dazu geschrieben. Der Submit-Button wird mittels `[disabled]="!saveForm.valid"` solange deaktiviert, bis alle Validierungen erfolgreich sind. Sobald das Formular korrekt ausgefüllt ist, wird der Button aktiviert und das `(click)="onRegister()"`-Event kann ausgeführt werden, um das Formular abzusenden.

register-form.ts

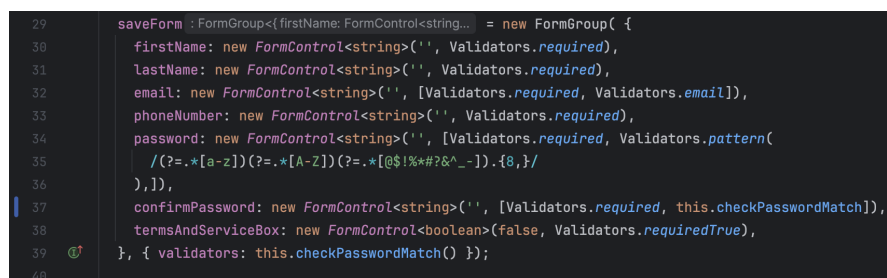


Abbildung 50: Aufbau und Validation der FormGroup

In diesem Code wird das FormGroup namens saveForm erstellt und konfiguriert. Jedes Feld wie beispielsweise "firstName", "lastName", "email", "phoneNumber",

“password” usw. erhält ein eigenes FormControl mit passenden Validierungen. Validators.required beispielsweise, sorgt dafür, dass ein Feld nicht leer sein darf. Validators.email prüft das E-Mail-Format.

```

89     checkPasswordMatch(): ValidatorFn { Show usages  ▸ Jakob Schlager
90     return (control: AbstractControl): { [key: string]: boolean } | null => {
91         const password : AbstractControl<any, any> | null = control.get('password');
92         const confirmPassword : AbstractControl<any, any> | null = control.get('confirmPassword');
93
94         if (password && confirmPassword && password.value !== confirmPassword.value) {
95             return { 'passwordDoesNotMatch': true }
96         }
97
98         return null;
99     };
100 }

```

Abbildung 51: Validation für die erneute Eingabe des Passwords

Darüber hinaus wird der benutzerdefinierte Validator this.checkPasswordMatch() dem Feld “confirmPassword” hinzugefügt, sodass die Passwörter als Kombination validiert werden können. Damit wird sicher gestellt, dass “password” und “confirmPassword” übereinstimmen, bevor das Formular abgesendet werden kann.

Listing 1: Einbindung eines benutzerdefinierten Validators

```

1      confirmPassword: new FormControl<string>('', [Validators.required,
      this.checkPasswordMatch]),

```

5.1.3 Drag und Drop Funktion

Die Drag und Drop Funktion ist auf der *Melius* Website eine essentielle und wichtige Funktion die vor allem für die Personalisierung eine wichtige Rolle spielt.

Möglichkeiten für Interaktion des/der Benutzer:in

```

16
17     <!-- CV Column Two -->
18     <div [ngClass]="showBorders" class="col-start-2 space-y-10" id="middleSkillCol"
19         (dragover)="onDragOver($event)"
20         (drop)="onDrop($event, targetContainerId: 'middleSkillCol')">
21
22     <!-- Allgemeine Informationen -->
23     <div id="generalInfoBox" [ngClass]="dragBox" [draggable]="checkDraggable()" (dragstart)="onDragStart($event, $event.target)"

```

Abbildung 52: Benutzer:in

Die Spalten

In der vorliegenden Datei (Abbildung 52) werden die Klassen sowie die Attribute des HTML-Formulars dargestellt. Die Interaktion mit den Formularen und den drei Spalten

erfolgt durch den/die Benutzer:in. Die Spalten ermöglichen dem/der Benutzer:in das Hin- und Herschieben der Formulare.

Jede Spalte ist mit einer der folgenden Klassen assoziiert:

Listing 2: Klassen die von den Spalten benutzt werden

```
1      [ngClass]="showBorder"
2      (dragover)="onDragOver($event)"
3      (drop)="onDrop($event, 'columnId')"
```

[ngClass]="showBorder"

- Wird verwendet um den/der Benutzer:in haptisches bzw. visuelles Feedback mittels gestrichelten Linien zu geben, so dass er erkennt, wo genau sich die Spalten befinden und wo er die Formulare hinverschieben kann

(dragover)="onDragOver(\$event)"

- Ruft bei jedem dragover-Ereignis (wenn ein Element über das Ziel bewegt wird) die Methode **onDragOver()** auf.

(drop)="onDrop(\$event, 'columnId')"

- Reagiert auf das drop-Ereignis (wenn ein Element tatsächlich losgelassen wird).
- Ruft die Methode **onDrop()** auf und übergibt das Ereignis sowie eine zusätzliche Information (z. B. 'columnId').

Die Formulare

Alle Formulare sind mit den Klassen:

- **[ngClass]="dragBox "**
- **[draggable]="checkDraggable() "**
- **[dragstart]="onDragStart(\$event, \$event.target) "**

ausgestattet.

Was im Hintergrund passiert

In diesem Code werden drei zentrale Methoden zur Implementierung der Drag-and-Drop-Funktionalität genutzt:

```

263 // Wird ausgelöst, wenn das Ziehen beginnt
264 onDragStart(event: DragEvent, item: any) :void { Show usages  Jakob Schlager
265     this.draggedItem = item;
266     event.dataTransfer?.setData('text/plain', event.target?.toString() || '');
267 }

```

Abbildung 53: DragStart Funktion der Funktionalität

Diese Methode wird aufgerufen, sobald der/die Benutzer:in beginnt, ein Element zu ziehen. Durch `this.draggedItem = item;` wird das aktuell gezogene Element in einer Variable gespeichert, damit später – beispielsweise beim Ablegen – bekannt ist, welches Element bewegt wurde. Mit `event.dataTransfer?.setData('text/plain', event.target?.toString() || '')`; wird zusätzlich ein Text (hier in Form einer Zeichenkette) im dataTransfer-Objekt hinterlegt. Dadurch kann das Ziel (Drop-Ziel) später erkennen, welches Element gezogen wurde.

```

269 // Wird ausgelöst, wenn das Element über ein gültiges Drop-Ziel gezogen wird
270 onDragOver(event: DragEvent) :void { Show usages  Jakob Schlager
271     event.preventDefault(); // Muss aufgerufen werden, damit ein Drop möglich ist
272 }

```

Abbildung 54: DragOver Funktion der Funktionalität

Diese Methode wird jedes Mal aufgerufen, wenn ein gezogenes Element über ein potenzielles Ziel bewegt wird. Durch `event.preventDefault();` wird verhindert, dass der Browser die Standardaktion ausführt (die das Ablegen in vielen Fällen blockieren würde). Nur wenn `preventDefault()` aufgerufen wird, ist ein Drop-Event überhaupt möglich.

```

274 // Wird ausgelöst, wenn das Element fallen gelassen wird
275 onDrop(event: DragEvent, targetContainerId: string) :void { Show usages  sStieg +1
276     event.preventDefault();
277     const targetElement :HTMLInputElement | null = document.getElementById(targetContainerId);
278     if (targetElement && this.draggedItem) {
279         // Füge das gezogene Element dem Ziel hinzu
280         targetElement.appendChild(this.draggedItem);
281
282         switch (this.draggedItem) {
283             case this.generalInfoBox:
284                 this.portfolioService.currPortfolio!.generalInfoPosition = targetContainerId;
285                 break;
286             case this.educationsBox:
287                 this.portfolioService.currPortfolio!.educationsPosition = targetContainerId;
288                 break;
289             case this.workExperienceBox:
290                 this.portfolioService.currPortfolio!.workExperiencesPosition = targetContainerId;
291                 break;
292         }
293         this.portfolioService.updatePortfolio(this.portfolioService.currPortfolio!).subscribe();
294
295         this.draggedItem = null;
296     }
297 }

```

Abbildung 55: Drop Funktion der Funktionalität

Ereignis auslösen und Standardaktion verhindern

Sobald ein gezogenes Element über einem Drop-Ziel losgelassen wird, ruft der Browser die Methode `onDrop()` auf. Mit `event.preventDefault();` wird die Standardaktion verhindert, damit das Ablegen (Drop) im gewünschten Bereich funktioniert.

Ziel-Container ermitteln

Über `document.getElementById(targetContainerId)` wird das DOM-Element gesucht, in das das gezogene Element eingefügt werden soll. So weiß der Code, an welcher Stelle das Element abgelegt werden soll.

Element in Ziel-Container einfügen

Wenn sowohl das Ziel-Element als auch das gezogene Element (`this.draggedItem`) existieren, wird das gezogene Element via `appendChild()` dem Ziel-Container hinzugefügt. Damit wird es optisch und strukturell in die neue Position verschoben.

Aktualisierung der Positionsdaten In der anschließenden switch-Anweisung wird geprüft, ob das gezogene Element eines der speziellen Box-Elemente (`generalInfoBox`, `educationsBox`, `workExperienceBox`) ist. Je nach Fall wird die entsprechende Position in `this.portfolioService.currPortfolio!` auf die neue Ziel-ID (`targetContainerId`) gesetzt. Das bedeutet, dass das Portfolio-Objekt weiß, wo sich das jeweilige Element nun befindet.

Speichern der Änderungen Mit `this.portfolioService.updatePortfolio(...)` werden die aktualisierten Daten an den Server oder eine Datenbank geschickt (bzw. gespeichert), damit die Änderungen auch nach einem Neuladen der Seite bestehen bleiben.

Zurücksetzen des gezogenen Elements

Abschließend wird `this.draggedItem = null;` gesetzt, um anzuzeigen, dass momentan kein Element mehr gezogen wird. Damit ist der Drag-and-Drop-Vorgang abgeschlossen.

5.1.4 Auswahl der Sprache bzw. die Mehrsprachigkeit

Auswahl der Sprache

Settings Component

```
19 </div>
20 <app-language-select (eventEmitter)="this.selectedLanguage = $event" ></app-language-select>
21 </div>
```

Abbildung 56: Auswahl der Sprache

Auf der Einstellungen Seite wird auf eine benutzerdefinierte Angular-Komponente für die Sprachauswahl zugegriffen.

(eventEmitter)="this.selectedLanguage = \$event":

- Hier wird ein Event-Emitter verwendet, um die ausgewählte Sprache (\$event) in der Komponente zu speichern.
- Das bedeutet, dass die selectedLanguage-Variable in der TypeScript-Datei bei jeder Änderung automatisch aktualisiert wird.

```
58   confirmSettings() : void { Show usages  sStieg +1
59       let portfolio : Portfolio = this.portfolioService.currPortfolio!;
60       if (this.selectedBackgroundImageUrl != null) {
61
62       }
63
64       if (this.selectedBackgroundColor != '') {
65           portfolio.color = this.selectedBackgroundColor;
66       }
67
68       portfolio.languageCode = this.selectedLanguage;
69       console.log(portfolio);
70       this.portfolioService.updatePortfolio(portfolio).subscribe();
71
72       this.reloadPage()
73   }
```

Abbildung 57: Auswahl der Sprache

Die Datei **Language-settings-comp.ts** beinhaltet die Logik zur Verwaltung der Spracheinstellungen innerhalb der Anwendung. Im Fokus steht hierbei die Methode **confirmSettings()**, deren Funktion darin besteht, den Hintergrund des/der Benutzer:in, sowie die von dem/der Nutzer:in ausgewählte Sprache im Portfolio zu speichern und an das Backend zu übermitteln. Zu diesem Zweck wird zunächst das aktuelle Portfolio-Objekt aus dem "portfolioService" geladen. Im Anschluss wird die ausgewählte Sprache durch die Zeile **portfolio.languageCode = this.selectedLanguage** in das Objekt übernommen. Es wird sichergestellt, dass die getroffene Auswahl eines User korrekt im Portfolio hinterlegt wird. Um die geänderten Einstellungen zu speichern, wird die Methode **updatePortfolio(portfolio).subscribe()** auf dem **portfolioService** aufgerufen. Dadurch wird das aktualisierte Portfolio-Objekt an das Backend gesendet und die neue Sprache dauerhaft gespeichert. Zum Abschluss wird mit **this.reloadPage()** die Seite neu geladen, um die Änderungen sofort sichtbar zu machen.

Diese Datei stellt eine essenzielle Verbindung zwischen der Benutzeroberfläche und dem Backend dar, indem sie die Sprachauswahl des/der Nutzers:in entgegennimmt, speichert und sicherstellt, dass die Anwendung stets die korrekte Sprache anzeigt.

Lanugage Select Component

```

38 <button
39   *ngFor="let language of languages"
40   class="flex items-center px-6 py-2 rounded-md font-semibold text-gray-700 hover:bg-accent-blue duration-150 w-full"
41   (click)="selectLanguage(language)"
42   role="menuitem"
43 >
44   <span class="mr-2">{{ language.flag }}</span>
45   {{ language.name }}
46 </button>

```

Abbildung 58: Liste and Sprachen

In diesem Kontext wird mit `*ngFor` über das `languages`-Array iteriert, um für jede verfügbare Sprache einen Button zu erstellen. Jeder Button repräsentiert eine Sprache und zeigt sowohl ein Flaggen-Symbol (`language.flag`) als auch den Namen der Sprache (`language.name`) an. Beim Klicken auf einen Button wird die Methode `selectLanguage(language)` aufgerufen, wodurch die ausgewählte Sprache an die übergeordnete Komponente weitergegeben wird.

Diese Struktur gestattet es dem/der Nutzer:in, eine Sprache durch einfachen Klick auszuwählen, wodurch die Auswahl direkt übernommen und verarbeitet werden kann.

```

37 selectLanguage(language: any) : void { Show usages 1 Jakob Schlager +1
38   this.selectedLanguage = language;
39   this.eventEmitter.emit(language.code);
40 }

```

Abbildung 59: Bestätigung der Auswahl sowie Weiterleitung an Settings Component

Die in dem TypeScript-Code enthaltene Methode (Abbildung 59) `selectLanguage(language: any)` ist für die Speicherung und Weitergabe der ausgewählten Sprache an die übergeordnete Komponente verantwortlich. Die Funktionsweise ist wie folgt:

Speichern der ausgewählten Sprache:

- Die Zeile `this.selectedLanguage = language;` weist der `selectedLanguage`-Eigenschaft das übergebene `language`-Objekt zu, wodurch die gewählte Sprache in der Komponente gespeichert wird.

Event-Emitter zur Weitergabe der Sprache:

- Mittels `this.eventEmitter.emit(language.code);` wird das Sprachcode-Attribut der ausgewählten Sprache (`language.code`) über den Event-Emitter an die überge-

ordnete Komponente gesendet. Dies ermöglicht es anderen Komponenten, auf die Änderung der Sprache zu reagieren und die Sprache gegebenenfalls anzupassen.

App Component

```

27     constructor(private router: Router, private translate: TranslateService) {
28         this.translate.addLangs(['de', 'en']);
29         this.translate.setDefaultLang('de');
30     }

```

Abbildung 60: Setzen der Auswahlmöglichkeiten im Konstruktor

Beim Erstellen der AppComponent wird die Sprachverwaltung über den TranslateService eingerichtet.

- **this.translate.addLangs(['de', 'en']);** definiert die verfügbaren Sprachen – in diesem Fall Deutsch (de) und Englisch (en).
- **this.translate.setDefaultLang('de');** legt Deutsch als Standardsprache fest, falls keine andere Sprache gewählt wurde.

Damit wird sichergestellt, dass die Anwendung von Beginn an Mehrsprachigkeit unterstützt.

```

41         this.portfolioService.getPortfolioById(user.id).subscribe(portfolio : Portfolio => {
42             this.portfolioService.currPortfolio = portfolio;
43             this.translate.use(portfolio.languageCode);
44             this.portfolioService.backgroundColorSubject.next(portfolio.color);
45             console.log(portfolio);
46         })

```

Abbildung 61: Übernehmen der ausgewählten Sprache im System

Hier wird beim Laden der Anwendung das Portfolio des/der aktuellen Nutzers:in abgerufen:

- **this.portfolioService.getPortfolioById(user.id).subscribe(portfolio => ...)** ruft die gespeicherten Daten des/der Nutzers:in aus dem Backend ab.
- **this.portfolioService.currPortfolio = portfolio;** speichert das Portfolio in der portfolioService-Instanz für die weitere Nutzung.
- **this.translate.use(portfolio.languageCode);** setzt die Sprache der Anwendung auf die im Portfolio gespeicherte Sprache (portfolio.languageCode). Dadurch wird sichergestellt, dass die Benutzeroberfläche sofort in der bevorzugten Sprache des Nutzers geladen wird.

- `this.portfolioService.backgroundColorSubject.next(portfolio.color);` aktualisiert die Hintergrundfarbe basierend auf den gespeicherten Nutzereinstellungen.
- `console.log(portfolio);` gibt die geladenen Portfolio-Daten zur Überprüfung in der Konsole aus.

Übersetzungsdateien

```

1  {
2    "landing": {
3      "registerBtn": "Jetzt Loslegen",
4      "loginBtn": "Anmelden"
5    },
6    "home": {
7      "avatar": {
8        "add": "Hinzufügen",
9        "remove": "Entfernen"
10     },
11     "header": "Willkommen zurück!",
12     "nav": {
13       "cv": "Lebenslauf",
14       "projects": "Projekte",
15       "comps": "Kompetenzen"
16     },
17     "start": {
18       "landingBoxCV": "Stell dich vor",
19       "landingBoxProjects": "Sei stolz auf deine Projekte",
20       "landingBoxComp": "Zeig was du drauf hast"
21     },
22     "settingsButton": {
23       "title": "Einstellungen",
24       "dad": "Drag & Drop"
25     },
26   },
27 }

```

```

1  {
2    "landing": {
3      "registerBtn": "Start now!",
4      "loginBtn": "Login"
5    },
6    "home": {
7      "avatar": {
8        "add": "Add",
9        "remove": "Remove"
10     },
11     "header": "Welcome back!",
12     "nav": {
13       "cv": "CV",
14       "projects": "Projects",
15       "comps": "Competencies"
16     },
17     "start": {
18       "landingBoxCV": "Introduce yourself",
19       "landingBoxProjects": "Be proud of your projects",
20       "landingBoxComp": "Show what you've got"
21     },
22     "settingsButton": {
23       "title": "Settings",
24       "dad": "Drag & Drop"
25     },
26   },
27 }

```

Abbildung 62: Übersetzungsdatei - Deutsch

Abbildung 63: Übersetzungsdatei - Englisch

Für die Gewährleistung der Mehrsprachigkeit sind zwei .json-Dateien von essentieller Relevanz, in welchen sämtliche statischen Texte übersetzt werden. Im vorliegenden Fall von *Melius* befinden sich in diesen Dateien die Texte jeweils in deutscher und englischer Sprache. Die Pfade zu den Texten müssen schließlich lediglich in die .html-Datei übernommen werden, um die Übersetzung des Textes abzuschließen.

Nutzung der translate-pipe in den .html Dateien

```

12  <div class="w-fit text-center font-bold pt-10 space-y-2.5">
13    <div class="bg-primary-color text-white rounded-lg hover:bg-accent-blue hover:text-primary-color duration-150 ease-in-out">
14      <button class="text-xl px-8 py-1.5" (click)="clickedRegister()">{{ 'landing.registerBtn' | translate }}</button>
15    </div>
16
17    <div class="bg-primary-color text-white rounded-lg hover:bg-accent-blue hover:text-primary-color duration-150 ease-in-out">
18      <button class="text-xl w-full py-1.5" (click)="clickedLogin()">{{ 'landing.loginBtn' | translate }}</button>
19    </div>
20  </div>

```

Abbildung 64: Beispiel für die Verwendung der Sprachübersetzung

Abschließend wird lediglich der statische Text mit den Pfaden der Übersetzungsdatei ausgetauscht und die **Translate Pipe** hinzugefügt. In der Folge wird der Text je nach Auswahl des/der Benutzers:in in die jeweilige Sprache übersetzt und angezeigt.

5.2 Backend - Josef Stieg

5.2.1 Entwicklungsumgebung

Für die Entwicklung des Backends wurde die Entwicklungsumgebung **IntelliJ IDEA** aus der **JetBrains Toolbox** verwendet. Die Wahl fiel auf IntelliJ aus mehreren Gründen, die sowohl auf der Erfahrung des Entwicklerteams als auch auf den zahlreichen Vorteilen dieser integrierten Entwicklungsumgebung (IDE) beruhen.

Ein entscheidender Faktor für die Nutzung von IntelliJ ist, dass das Entwicklerteam die JetBrains Toolbox, und damit auch IntelliJ IDEA, regelmäßig im Schulalltag verwendet. Durch diese häufige Nutzung war das Team bereits mit der Benutzeroberfläche, den Funktionen und der Konfiguration der IDE vertraut, was den Einstieg in die Entwicklung deutlich erleichterte. Diese Vertrautheit reduzierte die Lernkurve und ermöglichte eine produktive Arbeitsweise von Beginn an.

Darüber hinaus bietet IntelliJ IDEA zahlreiche Vorteile, die es zu einer ausgezeichneten Wahl für die Backend-Entwicklung machen:

- **Umfassende Unterstützung für Java und Frameworks:** IntelliJ IDEA ist speziell auf die Java-Entwicklung zugeschnitten und bietet erstklassige Unterstützung für Frameworks wie Quarkus, Spring und *Jakarta EE*. Dadurch werden Konfigurationsaufgaben erleichtert, und die Arbeit mit modernen Frameworks wird durch eingebaute Tools und Assistenten optimiert.
- **Intelligente Code-Navigation:** Mit Funktionen wie Code-Vervollständigung, Refactoring-Tools und schneller Navigation durch Projekte bietet IntelliJ eine hochproduktive Umgebung, die den Entwicklungsprozess beschleunigt und Fehler reduziert.
- **Einfache Integration von Plugins:** IntelliJ unterstützt eine Vielzahl von Plugins, die sich nahtlos in die Entwicklungsumgebung integrieren lassen. Dadurch können zusätzliche Tools und Technologien, die im Projekt benötigt werden, schnell eingebunden werden.

- **Integrierte Tools:** IntelliJ bietet zahlreiche integrierte Werkzeuge wie einen Version-Control-Client (z. B. für Git), eine Datenbank-Integration und ein leistungsstarkes Debugging-System. Diese Funktionen machen externe Tools oft überflüssig und ermöglichen ein konzentriertes Arbeiten innerhalb der IDE.
- **Benutzerfreundlichkeit:** Die intuitive Benutzeroberfläche, anpassbare Einstellungen und umfangreiche Dokumentation machen IntelliJ IDEA zu einer leicht zugänglichen Entwicklungsumgebung – sowohl für Einsteiger als auch für erfahrene Entwickler.
- **Leistungsfähigkeit und Stabilität:** IntelliJ IDEA ist bekannt für seine zuverlässige Leistung, selbst bei großen Projekten mit komplexer Codebasis.

Zusätzlich bietet die JetBrains Toolbox den Vorteil, dass alle relevanten JetBrains-Tools zentral verwaltet werden können. So bleibt die Entwicklungsumgebung stets aktuell, und der Zugriff auf andere Tools wie DataGrip oder PhpStorm wird erleichtert, falls diese im Projekt benötigt werden.

Die Kombination aus der Vertrautheit des Teams mit der Umgebung, der leistungsstarken Unterstützung für Java und modernen Frameworks sowie den zahlreichen integrierten Werkzeugen machte IntelliJ IDEA zur idealen Wahl für die Entwicklung des Backends.



Abbildung 65: IntelliJ IDEA - JetBrains

5.2.2 Model

Im *Melius*-Backend erfolgt die Erstellung und Verwaltung aller Datenbanktabellen unter Zuhilfenahme der *Jakarta Persistence API (JPA)* in Kombination mit *Hibernate* als Object-Relational Mapping (ORM)-Framework. Dies impliziert, dass jede in der Anwendung definierte Java-Entitätsklasse einer eigenen Tabelle in der Datenbank entspricht.

Dies hat den Vorteil, dass die Verwaltung der Daten erheblich vereinfacht wird, da Entwickler nicht direkt mit SQL-Abfragen arbeiten müssen, sondern auf objektorientierte Weise mit den Daten interagieren können.

Durch diese ORM-Technologie können Datenbankoperationen wie Erstellen, Lesen, Aktualisieren und Löschen (CRUD-Operationen) direkt über Java-Klassen durchgeführt werden. *Hibernate* übernimmt dabei automatisch die Zuordnung der Java-Objekte zu den entsprechenden Tabellen und Spalten, sodass eine enge Integration zwischen der Anwendung und der Datenbank entsteht.

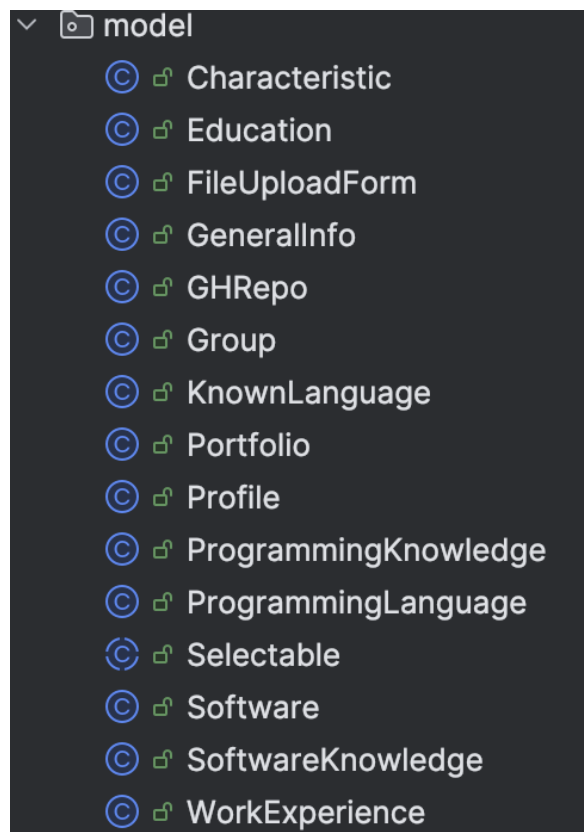


Abbildung 66: Entity - Klassen

Beispiele

Portfolio - Klasse

Die Portfolio-Klasse konstituiert das zentrale Element der Datenstruktur im Backend. Sie fungiert als Hauptentität und verweist auf nahezu alle anderen Entitäten, die auf der Portfolio-Seite dargestellt werden. In dieser Funktion verknüpft sie verschiedene Datenbereiche miteinander und ermöglicht eine strukturierte Verwaltung sowie eine effiziente Abfrage der relevanten Informationen.

Annotationen

@Id

Diese Annotation kennzeichnet das Primärschlüsselfeld der Entität. In dieser Klasse fehlt jedoch die @GeneratedValue-Annotation, die für die automatische Generierung eines Primärschlüssels sorgen könnte.

@OneToOne

Diese Annotation beschreibt eine 1:1-Beziehung zwischen der Portfolio-Entität und einer anderen Entität.

```
@Id
@OneToOne
@JoinColumn(name="profile_id", referencedColumnName = "id")
@JsonIgnoreProperties(value = {"portfolio"}, allowSetters = true)
private Profile profile;
```

Abbildung 67: OneToOne

- Diese Beziehung bedeutet, dass jedes Portfolio genau ein Profile-Objekt besitzt.
- @JoinColumn(name="profile_id", referencedColumnName = "id"):
profileId ist der Fremdschlüssel in der Portfolio-Tabelle, der auf die id-Spalte der Profile-Tabelle verweist.
- @JsonIgnoreProperties(value = {"portfolio"}, allowSetters = true):
stellt sicher, dass bei JSON-Serialisierung rekursive Referenzen zwischen Portfolio und Profile vermieden werden.

```
@OneToOne 2 usages
@JoinColumn(name="generalInfo_id", referencedColumnName = "id")
@JsonIgnoreProperties(value = {"portfolio"}, allowSetters = true)
private GeneralInfo generalInfo;
```

Abbildung 68: OneToOne

@OneToMany

Diese Annotation beschreibt eine 1:n-Beziehung, bei der ein Portfolio mehrere abhängige Entitäten besitzt.

```
@OneToMany(mappedBy = "portfolio") 2 usages
@JsonIgnoreProperties({"portfolio"})
private Set<Education> educations;
```

Abbildung 69: OneToMany

- Ein Portfolio kann mehrere Education-Einträge haben.
- **mappedBy = "portfolio"** zeigt, dass die Education-Entität eine portfolio-Referenz besitzt.
- **@JsonIgnoreProperties({"portfolio"})**:
verhindert rekursive JSON-Serialisierung.

Weitere 1:n-Beziehungen:

- Set<WorkExperience> workExperiences
- Set<KnownLanguage> knownLanguages
- Set<ProgrammingKnowledge> programmingKnowledges
- Set<SoftwareKnowledge> softwareKnowledges

Diese Beziehungen ermöglichen, dass ein Portfolio mehrere berufliche Erfahrungen, Sprachen, Programmierkenntnisse oder Softwarekenntnisse speichern kann.

@ManyToMany

Diese Annotation beschreibt eine n:m-Beziehung, bei der mehrere Portfolios mehrere Eigenschaften (Characteristic) haben können.

```
@ManyToMany 2 usages
@JoinTable(
    name="portfolio_characteristic",
    joinColumns = @JoinColumn(name="portfolio_id"),
    inverseJoinColumns = @JoinColumn(name="characteristic_id")
)
@JsonIgnoreProperties({"portfolios"})
private Set<Characteristic> characteristics;
```

Abbildung 70: ManyToMany

- Dies bedeutet, dass eine separate Join-Tabelle (portfolio_characteristic) existiert, die die Verknüpfung zwischen Portfolio und Characteristic speichert.

- `joinColumns = @JoinColumn(name="portfolio_id"):`

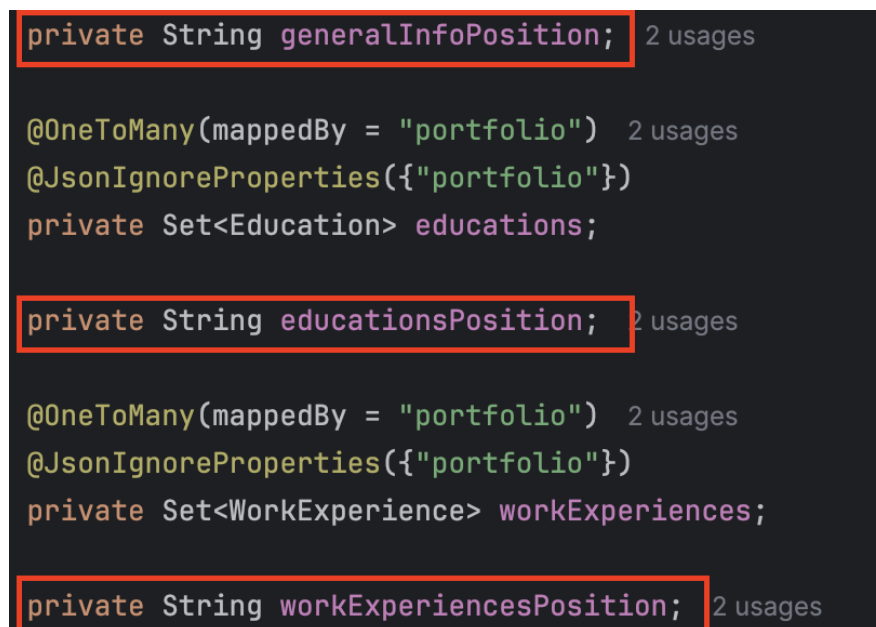
Verweist auf die `portfolio_id` – Spalte in der Join – Tabelle.

- `inverseJoinColumns = @JoinColumn(name="characteristic_id"):`

Verweist auf die `characteristic_id`-Spalte.

Zusätzliche String-Felder

Die Klasse enthält mehrere String-Attribute für Positionen wie: `private String generalInfoPosition;` `private String educationsPosition;` `private String workExperiencesPosition;` Diese Felder werden verwendet, um die Spalten - Positionen der einzelnen Boxen im Portfolio zu speichern.



```

private String generalInfoPosition; 2 usages

@OneToMany(mappedBy = "portfolio") 2 usages
@JsonIgnoreProperties({"portfolio"})
private Set<Education> educations;

private String educationsPosition; 2 usages

@OneToMany(mappedBy = "portfolio") 2 usages
@JsonIgnoreProperties({"portfolio"})
private Set<WorkExperience> workExperiences;

private String workExperiencesPosition; 2 usages

```

Abbildung 71: Portfolio - Strings

5.2.3 Repository

Zur Gewährleistung einer effizienten Verwaltung und Interaktion mit den verschiedenen Entity-Klassen wurde für nahezu jede Entitätsklasse eine entsprechende Repository-Klasse implementiert. Die Repository-Klassen fungieren als Schnittstelle zwischen der Anwendung und der Datenbank und ermöglichen grundlegende CRUD-Operationen (Create, Read, Update, Delete) sowie komplexe Datenabfragen.

Die Verwendung von Spring Data JPA ermöglicht die Durchführung vieler dieser Abfragen ohne umfangreiche manuelle SQL-Statements, da Spring Data automatisch

Methoden zur Datenverarbeitung bereitstellt. Zudem lassen sich individuelle Abfragen mithilfe von JPQL (Java Persistence Query Language) oder der Criteria API definieren, um spezifische Datenanforderungen effizient umzusetzen.

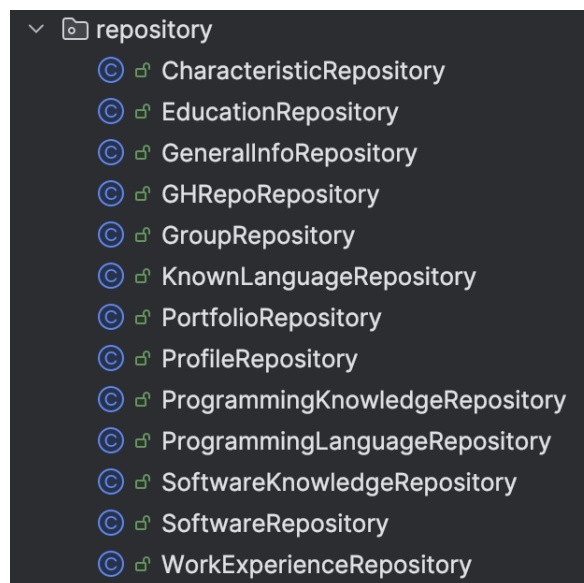


Abbildung 72: Repository - Klassen

Beispiele

ProfileRepository - Klasse

Die ProfileRepository-Klasse spielt eine zentrale Rolle in der Verwaltung von Benutzerprofilen innerhalb der Anwendung. Sie ist für grundlegende CRUD-Operationen (Erstellen, Lesen, Aktualisieren und Löschen) von Profilen verantwortlich und stellt sicher, dass diese effizient in der Datenbank gespeichert und verwaltet werden.

Neben diesen Standardfunktionen übernimmt die ProfileRepository-Klasse auch wichtige Überprüfungen und Sicherheitsmaßnahmen. Beispielsweise wird beim Login das eingegebene Passwort mit dem in der Datenbank gespeicherten Hash-Wert verglichen, um die Identität zu bestätigen. Darüber hinaus wird bei der Registrierung neuer Nutzer:innen überprüft, ob bereits ein Profil mit der gleichen E-Mail-Adresse existiert, um Doppelregistrierungen zu verhindern und die Datenintegrität zu gewährleisten.

Dank dieser Funktionen trägt das Repository maßgeblich zur Sicherheit, Datenkonsistenz und Benutzerverwaltung innerhalb der Anwendung bei. Es ermöglicht eine reibungslose Interaktion mit der Datenbank und bildet eine essenzielle Schnittstelle zwischen Backend-Logik und Datenpersistenz.

Annotationen

@ApplicationScoped

Die Annotation `@ApplicationScoped`, die aus *Jakarta CDI (Contexts and Dependency Injection)* stammt, gewährleistet, dass eine Klasse während der gesamten Laufzeit der Anwendung lediglich einmal instanziiert wird. Dies impliziert, dass sämtliche Beans, welche diese Klasse verwenden, auf die identische Instanz zugreifen und folglich ein Singleton-Verhalten erzielen. In Bezug auf die `ProfileRepository`-Klasse resultiert daraus eine Vielzahl an Vorteilen. Da das Repository für den Zugriff auf die Datenbank zuständig ist, sorgt `@ApplicationScoped` dafür, dass nicht bei jeder Anfrage eine neue Instanz erstellt wird, sondern alle Datenbankoperationen über eine einzige, wiederverwendbare Instanz laufen. Dies verbessert die Effizienz, da weniger Ressourcen für das Erstellen neuer Objekte verbraucht werden. Darüber hinaus bleibt der Zustand der Bean während der gesamten Laufzeit der Anwendung erhalten. Dies kann sich beispielsweise positiv auf Caching-Mechanismen oder optimierte Datenbankverbindungen auswirken.

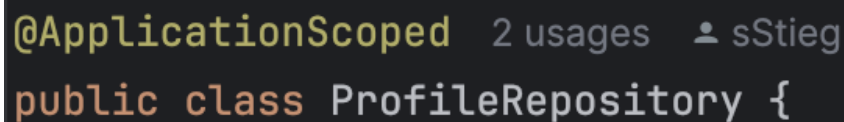
A screenshot of an IDE showing the `@ApplicationScoped` annotation in green text, followed by '2 usages' and a user icon labeled 'sStieg'. Below this, the start of a Java class is visible: `public class ProfileRepository {` in orange and white text on a dark background.

Abbildung 73: ApplicationScoped

@Transactional

Der Einsatz von `@Transactional` in den Methoden `addProfile` und `updateProfile` gewährleistet folglich die Aufrechterhaltung der Datenkonsistenz und verhindert das Auftreten inkonsistenter oder unvollständiger Datenbankzustände. Im Falle eines Fehlers während der Ausführungsphase der Methoden wird die gesamte Transaktion automatisch zurückgesetzt, wodurch fehlerhafte oder unvollständige Änderungen in der Datenbank verhindert werden.

- **addProfile(Profile profile)** Diese Methode prüft zunächst, ob bereits ein Profil mit der angegebenen E-Mail existiert. Falls nicht, wird das neue profile mit `entityManager.persist(profile)` in die Datenbank eingefügt. Sollte jedoch bereits ein Profil existieren, wird eine `BadRequestException` ausgelöst. Durch `@Transactional` wird sichergestellt, dass die gesamte Methode entweder komplett erfolgreich durchläuft oder im Fehlerfall keine Teildaten gespeichert werden.

- **updateProfile(Profile profile)** Hier wird das übergebene profile mit entity-Manager.merge(profile) aktualisiert. Auch diese Methode ist mit @Transactional versehen, wodurch sichergestellt wird, dass alle Änderungen an der Datenbank entweder vollständig übernommen oder – falls ein Fehler auftritt – zurückgesetzt werden.

```
@Transactional 1 usage  sStieg
public Profile addProfile(Profile profile) {
    if(getProfileByEmail(profile.getEmail()) == null) {
        this.entityManager.persist(profile);
        return profile;
    }

    throw new BadRequestException();
}

@Transactional 2 usages  sStieg
public Profile updateProfile(Profile profile) {
    this.entityManager.merge(profile);
    return profile;
}
```

Abbildung 74: Transactional

Wichtige Funktionen

addProfile

Die Methode ‘addProfile()‘ dient dazu, ein neues Profil in die Datenbank zu speichern. Bevor das Profil jedoch eingefügt wird, erfolgt eine Überprüfung, ob bereits ein anderes Profil mit derselben E-Mail-Adresse existiert. Dadurch wird verhindert, dass sich mehrere Benutzer:innen mit identischen E-Mail-Adressen registrieren, was die Datenkonsistenz und Eindeutigkeit der Benutzerprofile sicherstellt.

```
@Transactional 1 usage  sStieg
public Profile addProfile(Profile profile) {
    if(getProfileByEmail(profile.getEmail()) == null) {
        this.entityManager.persist(profile);
        return profile;
    }

    throw new BadRequestException();
}
```

Abbildung 75: addProfile

checkProfile

Die Methode ‘checkProfile()’ findet Anwendung im Rahmen des Autorisierungsprozesses für ein Login. Im ersten Schritt wird das Profil des/der Nutzers/in anhand der von ihm eingegebenen E-Mail-Adresse aus der Datenbank abgerufen. Daraufhin erfolgt ein Abgleich zwischen dem von dem/der Nutzer/in eingegebenen Passwort und dem im Profil gespeicherten Passwort. Diese Methode wird nicht nur beim eigentlichen Login-Prozess verwendet, sondern auch bei jedem Neuladen der Seite, um die Authentifizierung des/der Nutzers/in sicherzustellen.

```
public boolean checkProfile(String email, String password) { 1 usage  sStieg
    Profile profile = getProfileByEmail(email);
    return profile != null && profile.getPassword().equals(password);
}
```

Abbildung 76: checkProfile

getProfileByEmail

Die Methode ‘getProfileByEmail()’ nimmt eine signifikante Rolle im Kontext der Verwaltung von Benutzerprofilen ein. Sie wird unter anderem bei der Erstellung eines neuen Profils eingesetzt, um die Existenz doppelter Profile mit identischer E-Mail-Adresse zu verhindern. Dies dient dazu, die Vergabe identischer Anmeldedaten an mehrere Benutzer:innen zu unterbinden.

Darüber hinaus ist diese Funktion von entscheidender Bedeutung für den Login-Prozess. Da beim Einloggen die eindeutige ID des Profils nicht bekannt ist, sondern lediglich die E-Mail-Adresse des/der Nutzer/in vorliegt, kann das zugehörige Profil nicht direkt über den Primärschlüssel mit entityManager.find() ermittelt werden. Stattdessen durchsucht

getProfileByEmail() die gespeicherten Profile nach der angegebenen E-Mail-Adresse und gibt das entsprechende Profil zurück. Diese Vorgehensweise gewährleistet, dass die Anmeldeinformationen korrekt sind und die Authentifizierung des korrekten Users sichergestellt werden kann.

```
public Profile getProfileByEmail(String email) { 3 usages  ⚙ sStieg
    List<Profile> profiles = getAllProfiles();

    for(Profile currProfile: profiles) {
        if(currProfile.getEmail().equals(email)) {
            return currProfile;
        }
    }

    return null;
}
```

Abbildung 77: getProfileByEmail

5.2.4 Resource

Die Resource-Klassen nehmen eine zentrale Rolle in der Kommunikation zwischen dem Frontend und dem Backend ein. Sie fungieren als Vermittler, indem sie über REST-Endpunkte Anfragen aus dem Frontend entgegennehmen und die entsprechenden Daten aus der Datenbank abrufen oder speichern.

Jede Resource-Klasse ist dabei speziell für eine bestimmte Entität zuständig und bildet eine Schnittstelle zu einem zugehörigen Repository. Dies gewährleistet, dass sämtliche Datenbankoperationen effizient und strukturiert durchgeführt werden. Das Frontend kann über die definierten Endpunkte beispielsweise neue Einträge erstellen, bestehende Daten abrufen, aktualisieren oder löschen.

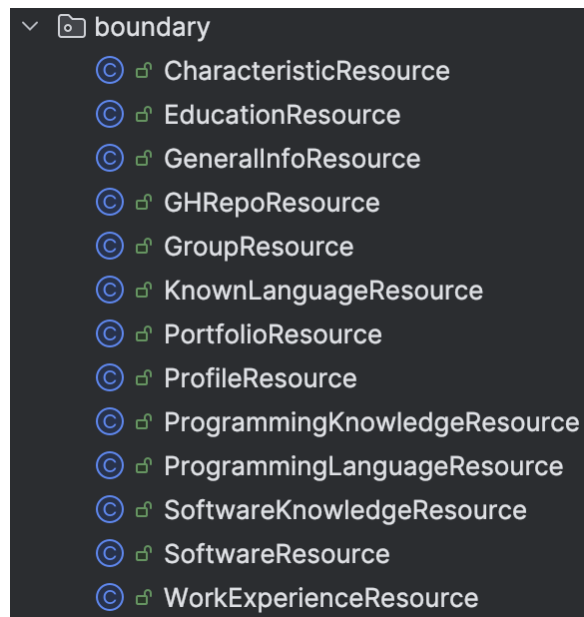


Abbildung 78: Resource - Klassen

Beispiele

ProfileResource - Klasse

Die ProfileResource-Klasse fungiert als zentrale Schnittstelle zwischen dem Frontend und den Datenbankoperationen, die mit den Profilen assoziiert sind. Sie stellt verschiedene REST-Endpunkte bereit, über die das Frontend mit dem Backend kommunizieren kann.

Mittels dieser Endpunkte können unterschiedliche Aktionen ausgeführt werden, wie beispielsweise das Erstellen, Abrufen und Aktualisieren von Profilen in der Datenbank. Die Klasse nutzt dazu Methoden des ProfileRepositorys, um die gewünschten Datenbankabfragen effizient durchzuführen.

Die ProfileResource-Klasse gewährleistet eine klare Trennung zwischen der Präsentations- und der Datenzugriffsschicht und ermöglicht eine strukturierte und sichere Verwaltung der Profildaten.

Annotationen

@Path

Die @Path-Annotation stellt eine zentrale Annotation in *Jakarta RESTful Web Services* (JAX-RS) dar. Ihre Funktion besteht in der Festlegung einer URI-Ressource (Uniform Resource Identifier) für eine bestimmte Klasse oder Methode. Somit definiert sie den

Endpunkt, unter dem eine bestimmte Ressource oder Funktionalität über eine HTTP-Anfrage erreichbar ist.

```
@Path("/profile")
public class ProfileResource {
```

Abbildung 79: Path - Klasse

```
@Path("/login")
public Profile loginProfile(Profile profile) {
    if(this.profileRepository.checkProfile(profile)) {
        return this.profileRepository.getProfile(profile);
    }
}
```

Abbildung 80: Path - Methode

@Produces

Die @Produces-Annotation in *Jakarta RESTful Web Services (JAX-RS)* wird verwendet, um anzugeben, in welchem Format eine Methode eine Antwort zurückgibt. Sie definiert also den MIME-Typ (z. B. JSON oder XML), den der Server an den Client sendet. Wird diese Annotation in einer REST-API-Methode verwendet, stellt sie sicher, dass die zurückgegebenen Daten in einem bestimmten Format vorliegen. Beispielsweise kann @Produces(MediaType.APPLICATION_JSON) verwendet werden, um sicherzustellen, dass die Antwort als JSON formatiert ist. Dies ist besonders nützlich, da moderne Webanwendungen oft JSON als Standardformat für den Datenaustausch nutzen. Wenn keine @Produces-Annotation angegeben ist, entscheidet der Server anhand der Standardkonfiguration oder der Anfrage des Clients, welches Format zurückgegeben wird.

```
@Produces(MediaType.APPLICATION_JSON)
@Path("/register")
public Profile registerProfile(Profile newProfile) { return this.
```

Abbildung 81: Produces

@Consumes

Die @Consumes-Annotation in *Jakarta RESTful Web Services (JAX-RS)* spezifiziert das Datenformat, welches von einer REST-API-Methode unterstützt wird. Demnach definiert sie die MIME-Typen, die von der Methode verarbeitet werden können. So

impliziert `@Consumes(MediaType.APPLICATION_JSON)` die Akzeptanz von Anfragen, die mit JSON-Daten angereichert sind. Diese Annotation erweist sich insbesondere dann als vorteilhaft, wenn der Server strukturierte Daten, beispielsweise aus einer Webanwendung oder einer mobilen Applikation, erhält und diese verarbeitet, bevor sie in die Datenbank gespeichert oder weiterverarbeitet werden. Wird `@Consumes` nicht angegeben, entscheidet der Server anhand der Standardkonfiguration oder der Anfrage des Clients, welches Format akzeptiert wird.

```
@Consumes(MediaType.APPLICATION_JSON)
@Produces(MediaType.APPLICATION_JSON)
@Path("/register")
public Profile registerProfile(Profile newProfile) { return this.
```

Abbildung 82: Consumes

5.2.5 Docker-Compose

Um die Datenbank für *Melius* zu starten, wird ein `docker-compose.yml`-File verwendet, das es ermöglicht, die benötigten Dienste einfach zu konfigurieren und zu starten. In diesem Fall definiert die Datei zwei Hauptservices: Adminer und PostgreSQL.

Der Adminer-Service verwendet das Docker-Image `adminer` und stellt eine Weboberfläche bereit, über die die PostgreSQL-Datenbank verwaltet werden kann. Der Service wird so konfiguriert, dass er immer neu startet, falls er ausfällt, und ist unter dem Port 5431 zugänglich, der auf den internen Port 8080 des Adminer-Containers weitergeleitet wird.

Der PostgreSQL-Service verwendet das Docker-Image `postgres:16-alpine` und stellt die tatsächliche Datenbank für *Melius* bereit. Auch dieser Service wird so konfiguriert, dass er im Falle eines Ausfalls immer neu startet. Die PostgreSQL-Datenbank wird unter dem Standardport 5432 betrieben, das auf den gleichen Port des Hosts weitergeleitet wird. Die `volumes`-Direktive sorgt dafür, dass die Daten persistent gespeichert werden, indem sie ein Verzeichnis auf dem Host (`./data/postgres`) mit dem internen Verzeichnis der Datenbank verbindet. Dadurch bleiben die Daten auch nach dem Neustart des Containers erhalten.

Zudem werden Umgebungsvariablen gesetzt, darunter `POSTGRES_PASSWORD`, das für den Zugang zur Datenbank verwendet wird, sowie `PGDATA`, das den Speicherort der Datenbankdateien innerhalb des Containers festlegt. Schließlich wird der Container so konfiguriert, dass er mit den vom System definierten UID und GID läuft, um

Berechtigungsprobleme zu vermeiden, was besonders in Teamumgebungen wichtig sein kann, in denen unterschiedliche Benutzer:innen auf die Docker-Container zugreifen.

```
services:
  adminer:
    image: adminer
    restart: always
    ports:
      - 5431:8080

  postgres:
    image: postgres:16-alpine
    restart: always
    ports:
      - 5432:5432
    volumes:
      - ./data/postgres:/var/lib/postgresql/data
    environment:
      POSTGRES_PASSWORD: 
      PGDATA: /var/lib/postgresql/data/pgdata
    user: ${UID:-501}:${GID:-20}
```

Abbildung 83: Docker - Compose

5.2.6 Implementierung im Frontend

Das Frontend wurde mit *Angular* entwickelt, weshalb für alle Backend-Anfragen der `HttpClient` genutzt wird. Ähnlich wie bei den Resource-Klassen im Backend, wird für jede Entität ein eigener Service im Frontend verwendet, um die entsprechenden Daten zu verwalten und mit dem Backend zu kommunizieren.

HttpClient

Der `HttpClient` von *Angular* ist ein Service, der es ermöglicht, HTTP-Anfragen zu stellen, um mit externen APIs oder Backend-Servern zu kommunizieren. Er basiert auf der `RxJS`-Bibliothek und bietet eine einfache Möglichkeit, asynchrone Anfragen zu stellen, Daten zu empfangen und darauf zu reagieren.

Der `HttpClient` unterstützt verschiedene HTTP-Methoden wie GET, POST, PUT, DELETE und PATCH und verarbeitet die Antworten in Form von Observables, was bedeutet, dass er auf die Antwort warten kann, ohne den UI-Thread zu blockieren. Durch

die Verwendung von Observables können Entwickler einfach mit den zurückgegebenen Daten arbeiten, Fehler behandeln und auf den Status der Anfrage reagieren (z. B. Erfolg oder Fehler).

```
@Injectable({ Show usages  ⓘ sStieg
  providedIn: 'root'
})
export class PortfolioService {
  currPortfolio: Portfolio | undefined;
  httpClient: HttpClient = inject(HttpClient);
  backgroundColorSubject: Subject<string> = new Subject<string>()

  private readonly url : 'http://localhost:8080/portfol...  = "http://localhost:8080/portfol..."

  addPortfolio(portfolio: Portfolio) : Observable<Portfolio> { Show usages  ⓘ
    return this.httpClient.post<Portfolio>(this.url, portfolio);
  }
}
```

Abbildung 84: HttpClient

LocalStorage

LocalStorage ist eine Web-Storage-API, die es ermöglicht, Daten dauerhaft im Browser zu speichern. Im Gegensatz zum sessionStorage, bei dem die gespeicherten Daten nach dem Schließen des Browser-Tabs oder -Fensters gelöscht werden, bleiben die Daten im LocalStorage erhalten, bis sie explizit gelöscht werden oder der/die Benutzer/in den Browser-Cache leert.

LocalStorage eignet sich besonders für persistente Benutzereinstellungen, Sitzungsdaten oder die Zwischenspeicherung von Formulardaten, da diese nicht durch ein einfaches Neuladen der Seite verloren gehen. Die gespeicherten Informationen sind domänenspezifisch, d.h. sie können nur von der Website gelesen werden, von der sie gespeichert wurden (aus Sicherheitsgründen, um Cross-Site-Scripting-Angriffe zu verhindern).

Die Daten werden im Key-Value-Format als Strings gespeichert. Um komplexere Daten wie Objekte oder Arrays speichern zu können, müssen diese zunächst mit `JSON.stringify()` in einen String umgewandelt und beim Abruf mit `JSON.parse()` wieder in ihre ursprüngliche Form zurückverwandelt werden.

Da LocalStorage nur Strings speichert und eine Größenbeschränkung von ca. 5 MB pro Domain hat, eignet es sich nicht zum Speichern großer Datenmengen. Dennoch ist es ein sehr praktisches Werkzeug für viele Webanwendungen, insbesondere für die Persistenz von Benutzerinformationen und die Optimierung der Benutzererfahrung.

SessionStorage

SessionStorage ist ebenfalls eine Web-Storage-API, das die temporäre Speicherung von Daten im Browser ermöglicht, solange die aktuelle Browsersitzung aktiv ist. Im Gegensatz zum LocalStorage, bei dem die gespeicherten Daten dauerhaft erhalten bleiben, werden die Daten im SessionStorage gelöscht, sobald der/die Benutzer/in den Browser-Tab oder das Fenster schließt.

SessionStorage wird häufig verwendet, um sitzungsbezogene Daten zu speichern, wie z. B:

- Temporäre Benutzerdaten, die nur während einer Sitzung benötigt werden
- Zwischengespeicherte Formulareingaben, um Datenverluste während der Navigation zu vermeiden
- Session-spezifische Einstellungen oder Statusinformationen innerhalb einer Web-Anwendung

Ähnlich wie beim LocalStorage werden die Daten als Schlüssel-Wert-Paare gespeichert und sind nur für die Domain und den Tab/Fenster zugänglich, in dem sie gespeichert wurden. Das bedeutet, dass sie nicht auf andere Tabs oder Fenster der gleichen Website übertragen werden können.

Ein wesentlicher Vorteil des SessionStorage ist, dass er eine einfache Möglichkeit bietet, temporäre Daten sicher zu speichern, ohne dass sich der/die Benutzer/in um das manuelle Löschen kümmern muss. Es eignet sich besonders für Anwendungen, bei denen die Daten nur während einer einzigen Sitzung benötigt werden, wie z.B. temporäre Authentifizierungsdaten oder Cache-Daten während der Nutzung einer Web-Anwendung.

ngOnInit - Methode

Die Methode ngOnInit ist eine zentrale Lebenszyklusmethode in *Angular*, die automatisch aufgerufen wird, sobald die Komponente vollständig initialisiert wurde. Sie ist Teil der von *Angular* bereitgestellten OnInit-Schnittstelle und dient dazu, wichtige Vorbereitungen und Initialisierungen durchzuführen, die erst nach dem Laden der Komponente erfolgen sollten. Im Gegensatz zum Konstruktor, der hauptsächlich für die Instanziierung von Variablen und die Bereitstellung von Abhängigkeiten verwendet wird, ist ngOnInit besonders nützlich für asynchrone Operationen oder das Abrufen

von Daten, die erst nach der Konstruktion der Komponente zur Verfügung stehen sollen.

Ein typisches Anwendungsbeispiel für `ngOnInit` ist das Laden von Daten aus einer API, indem ein HTTP-Request an das Backend gesendet wird, um z.B. Benutzerdaten oder Konfigurationswerte abzurufen. Ebenso wird `ngOnInit` häufig zum Laden von Werten aus dem `localStorage` oder `sessionStorage` verwendet, was besonders nützlich ist, um gespeicherte Benutzerdaten beim Laden der Anwendung wiederherzustellen. Eine weitere häufige Anwendung ist das Registrieren von Event-Listnern oder das Aufrufen bestimmter Dienste, um initiale Zustände zu setzen.

Ein entscheidender Vorteil von `ngOnInit` ist, dass es nur einmal pro Instanz einer Komponente ausgeführt wird. Das bedeutet, dass alle enthaltenen Initialisierungsschritte nicht mehrfach ausgeführt werden, auch wenn sich Daten innerhalb der Komponente ändern. Dies erhöht die Effizienz der Anwendung, da keine unnötigen Operationen wiederholt werden. Darüber hinaus stellt `ngOnInit` sicher, dass bestimmte Aufgaben erst dann ausgeführt werden, wenn die Komponente tatsächlich bereit ist, wodurch Fehler vermieden werden, die beim Zugriff auf noch nicht vorhandene Abhängigkeiten auftreten können.

Damit *Angular* erkennt, dass die Methode `ngOnInit` existiert und korrekt ausgeführt werden kann, muss die Komponente das `OnInit`-Interface implementieren. Dies bedeutet, dass in der Klassendefinition `implements OnInit` angegeben wird, was *Angular* dazu veranlasst, `ngOnInit` als Teil des Lebenszyklus der Komponente zu behandeln und sie automatisch zum richtigen Zeitpunkt aufzurufen. Diese Struktur stellt sicher, dass Entwickler eine klare Trennung zwischen der reinen Konstruktion der Klasse und ihrer tatsächlichen Initialisierung aufrechterhalten können.

```
styleUrl: './cv-area.component.css'  
})  
export class CvAreaComponent implements OnInit {  
  generalInfoService: GeneralInfoService = inject(GeneralInfoService);  
  profileService: ProfileService = inject(ProfileService);  
}
```

Abbildung 85: Implements OnInit

Registrierung

Nachdem das Registrierungsformular vollständig ausgefüllt wurde, wird die Methode `onRegister()` in der entsprechenden Component aufgerufen. Diese Funktion durchläuft

dann Schritt für Schritt den Prozess der Erstellung eines neuen Benutzerprofils und eines neuen Portfolios.

Zuerst wird geprüft, ob das Formular gültig ist. Ist das Formular ungültig - z.B. weil Pflichtfelder nicht ausgefüllt sind oder ungültige Eingaben gemacht wurden - wird die Registrierung abgebrochen und eine entsprechende Fehlermeldung ausgegeben. Ist das Formular hingegen gültig, wird ein neues Profilobjekt mit den eingegebenen Daten erzeugt.

Anschließend wird eine Methode im `profileService` aufgerufen, die dieses neu erstellte Profil mit einem POST-Request an das Backend sendet. Das Backend verarbeitet die Anfrage, speichert das Profil in der Datenbank und gibt eine Bestätigung über den erfolgreichen Vorgang zurück.

Nachdem das Profil erfolgreich in der Datenbank angelegt wurde, müssen noch die 1-zu-1-Beziehungen zwischen dem Profil und anderen Entitäten, insbesondere dem Portfolio, hergestellt werden. Da das Profil bereits existiert, muss zunächst ein zugehöriges `GeneralInfo`-Objekt angelegt werden. Dieses Objekt enthält allgemeine Informationen über das Portfolio und ist direkt mit dem Profil verknüpft.

Für die Erstellung des `GeneralInfo`-Objekts wird der entsprechende Service verwendet, der seinerseits eine POST-Anfrage an das Backend sendet. Dadurch wird ein leeres `GeneralInfo`-Objekt in der Datenbank angelegt, das später mit weiteren Informationen gefüllt werden kann.

Im nächsten Schritt wird das Portfolio-Objekt angelegt. Ein Portfolio ist in *Melius* fest mit einem Profil und einem `GeneralInfo`-Objekt verknüpft. Daher wird das neu angelegte Portfolio so konfiguriert, dass es sowohl das zugehörige Profil als auch das `GeneralInfo`-Objekt referenziert. Auch dieses Portfolio-Objekt wird schließlich mit einem POST-Request an das Backend übertragen und dauerhaft in der Datenbank gespeichert.

Damit der/die Benutzer/in nach der Registrierung angemeldet bleibt, wird das erstellte Profil im `LocalStorage` des Browsers gespeichert. Dies hat den Vorteil, dass auch bei einem Reload der Seite kein Ausloggen erfolgt, und somit kein neuer Anmeldeprozess notwendig ist. Stattdessen wird das gespeicherte Profil aus dem `LocalStorage` geladen und in der Anwendung verwendet.

Durch diesen mehrstufigen Prozess werden die Grundstrukturen für ein neues Portfolio erfolgreich angelegt. Die Kombination aus Profil, GeneralInfo und Portfolio bildet die Grundlage für weitere Anpassungen und Ergänzungen innerhalb der Plattform.

```
onRegister() : void {
  this.saveForm.markAllAsTouched();
  if (this.saveForm.valid) {
    let newProfile: Profile = {
      id: 0,
      firstName: this.saveForm.controls['firstName'].value!,
      lastName: this.saveForm.controls['lastName'].value!,
      email: this.saveForm.controls['email'].value!,
      phoneNumber: this.saveForm.controls['phoneNumber'].value!,
      password: this.saveForm.controls['password'].value!
    };
    this.userService.handleUserRegistration(newProfile).subscribe(p => {
```

Abbildung 86: onRegister1

```
//this.userService.loggedInUser = response;
localStorage.setItem("loggedInUser", JSON.stringify(p));
console.log('Profile registered successfully.', p);

let generalInfo: GeneralInfo = {
  id: 0,
  address: "",
  city: "",
  gender: "",
  zipCode: "",
};

this.generalInfoService.addGeneralInfo(generalInfo).subscribe(g : GeneralInfo => {
  let portfolio: Portfolio = {
    profile: p,
    generalInfo: g,
    languageCode: "de",
    color: "#fff"
  };
  console.log(portfolio);

  this.portfolioService.addPortfolio(portfolio).subscribe() : void => {
    this.router.navigate(['/home']);
  }
})
```

Abbildung 87: onRegister2

Login

Der Login-Prozess in *Melius* folgt einem sehr ähnlichen Ablauf wie der Registrierungsprozess, mit dem Unterschied, dass hier ein bereits bestehendes Profil authentifiziert wird, anstatt ein neues Profil zu erstellen.

Nachdem der/die Benutzer/in das Login-Formular ausgefüllt hat, wird die Methode `onLogin()` in der entsprechenden Component aufgerufen. Diese Funktion übernimmt die gesamte Steuerung des Anmeldevorgangs. Zunächst werden die eingegebene E-Mail-Adresse und das Passwort aus dem Formular ausgelesen und per POST-Request an das Backend gesendet.

Das Backend prüft dann, ob ein Profil mit der angegebenen E-Mail-Adresse existiert und ob das eingegebene Passwort mit dem in der Datenbank gespeicherten Passwort übereinstimmt. Wenn die Anmeldedaten korrekt sind, sendet das Backend das entsprechende Profilobjekt zurück. Wenn die E-Mail-Adresse nicht existiert oder das Passwort falsch ist, wird eine Fehlermeldung zurückgegeben und die Anmeldung abgebrochen.

Sobald das Backend ein gültiges Profil zurückgibt, muss dieses auf der Client-Seite gespeichert werden, damit der User für die aktuelle Sitzung angemeldet bleibt und sich nicht bei jeder Interaktion erneut authentifizieren muss. Dabei wird unterschieden, ob sich der/die Nutzer/in nur für die aktuelle Sitzung anmelden möchte oder ob seine Anmeldeinformationen dauerhaft gespeichert werden sollen.

Zu diesem Zweck enthält das Anmeldeformular eine Checkbox mit der Option „Benutzerdaten speichern“. Die Entscheidung beeinflusst, wo das Profil gespeichert wird:

- Wird die Checkbox aktiviert, werden die Profildaten im LocalStorage gespeichert. Dies bedeutet, dass die Anmeldeinformationen auch nach dem Schließen des Browsers erhalten bleiben, so dass sich der/die Benutzer/in bei einem späteren Besuch der Website nicht erneut anmelden muss.
- Wenn das Kontrollkästchen nicht aktiviert ist, wird das Profil nur im SessionStorage gespeichert. Der SessionStorage ist nur so lange gültig, wie der Browser-Tab geöffnet ist. Sobald der Tab oder der gesamte Browser geschlossen werden sollten, werden die gespeicherten Daten automatisch gelöscht.

Durch diese flexible Speicherung wird sichergestellt, dass der Login-Prozess den individuellen Präferenzen der Anwenderinnen und Anwender entspricht. Gleichzeitig bleibt die Benutzerfreundlichkeit erhalten, da nicht bei jedem Seitenneuaufbau eine erneute Anmeldung erforderlich ist.

```
onLogin() : void { Show usages  ⚡ JakSchlager +1
  const loginData: UserLoginData = {
    email: this.saveForm.controls['email'].value || '',
    password: this.saveForm.controls['password'].value || ''
  };

  this.profileService.handleUserLogin(loginData).subscribe({
    next: (userInfo: Profile) : void => {
      localStorage.setItem("rememberUser", this.saveForm.controls['remember'].value!.toString());

      if(localStorage.getItem("rememberUser") === "true") {
        localStorage.setItem("loggedInUser", JSON.stringify(userInfo));
      } else {
        sessionStorage.setItem("loggedInUser", JSON.stringify(userInfo));
      }

      this.router.navigate(['/home']);
      console.log('Profile logged in successfully', userInfo);
    },

    error: error : any => {
      document.getElementById("login_error")!.innerHTML = "*Email oder Passwort falsch!";
      console.log('Error during the login process.', error);
    }
  });
};
```

Abbildung 88: onLogin

Laden der Daten aus dem Backend

Bei jedem Redirect oder Reload der Seite werden die Daten aus dem Portfolio-Objekt in der ngOnInit-Methode geladen. Dies geschieht praktisch auf jeder Seite, da das Portfolio-Objekt alle Informationen für die verschiedenen Unterseiten enthält. Um

eine zentrale Datenhaltung zu gewährleisten, wird das Portfolio-Objekt direkt in der `app.component.ts` geladen.

Die Entscheidung, diesen Vorgang in der `app.component.ts` durchzuführen, liegt darin begründet, dass die `app.component.ts` als übergeordnete Komponente für alle Unterseiten dient. Dadurch wird sichergestellt, dass die Daten für jede Seite direkt zur Verfügung stehen, ohne dass jede Unterseite sie einzeln abrufen muss.

In der Methode `ngOnInit` der `app.component.ts` läuft der Prozess wie folgt ab:

- Zuerst wird die ID des Profils aus dem `localStorage` bzw. `sessionStorage` geholt.
- Anschließend wird mit dieser ID das entsprechende Profil aus dem Backend geladen. Wird ein Fehler beim Laden des Profils geworfen, wird der/die Benutzer/in aus dem Local Storage gelöscht und man wird zu Login Page geschickt.
- Unmittelbar danach wird das zugehörige Portfolio-Objekt ebenfalls mit der gleichen ID aus dem Backend geholt.

Diese Methode stellt sicher, dass alle benötigten Daten direkt nach einem Seitenneuaufbau oder einer Weiterleitung zur Verfügung stehen, so dass die Benutzer:innen nahtlos weiterarbeiten können, ohne sich erneut anmelden oder Daten neu laden zu müssen.

```
ngOnInit() : void { no usages 1 sStieg +1
  if(localStorage.getItem("rememberUser") === "true") {

    if(localStorage.getItem("loggedInUser") !== null) {
      this.profileService.handleUserLogin(JSON.parse(localStorage.getItem("loggedInUser"))).subscribe({
        next: (user: Profile) : void => {
          this.profileService.loggedInUser = user;
          console.log(user)

          this.portfolioService.getPortfolioById(user.id).subscribe(portfolio : Portfolio => {
            this.portfolioService.currPortfolio = portfolio;
            this.translate.use(portfolio.languageCode);
            this.portfolioService.backgroundColorSubject.next(portfolio.color);
            console.log(portfolio);
          })
        },
        error: error : any => {
          localStorage.removeItem("loggedInUser");
          sessionStorage.removeItem("loggedInUser");
          this.router.navigate(["/"]);
        }
      });
    } else {
      this.router.navigate(["/"]);
    }
  } else {
    if(sessionStorage.getItem("loggedInUser") !== null) {
      this.profileService.loggedInUser = JSON.parse(sessionStorage.getItem("loggedInUser"));
    } else {
      this.router.navigate(["/"]);
    }
  }
}
```

Abbildung 89: Laden der Daten aus dem Backend

Verwendung der Daten

Nachdem die benötigten Daten in der Methode `ngOnInit` der `app.component.ts` aus dem Backend geladen und im lokalen Speicher bzw. in Variablen abgelegt wurden, müssen sie auf den jeweiligen Unterseiten weiterverarbeitet und für die Anzeige aufbereitet werden. Dies geschieht ebenfalls in der `ngOnInit`-Methode der jeweiligen Komponente, die für die einzelnen Seiten zuständig ist.

Um sicherzustellen, dass die Daten tatsächlich vollständig aus dem Backend geladen wurden, bevor sie auf den Unterseiten weiterverwendet werden, wird für alle Seiten eine `setTimeout`-Funktion mit einer Verzögerung von mindestens 200 Millisekunden verwendet. Diese kurze Verzögerung gibt dem System genügend Zeit, um die asynchronen Anfragen an das Backend abzuschließen und die zurückgegebenen Daten korrekt in den Speicher zu übernehmen, bevor sie weiterverarbeitet oder gerendert werden.

Dieser zusätzliche Sicherheitsmechanismus ist besonders wichtig, da *Angular* sehr schnell arbeitet und asynchrone Prozesse, wie z.B. HTTP-Anfragen an das Backend, nicht immer genau zum Zeitpunkt des Aufrufs der `ngOnInit`-Methode abgeschlossen sind. Würde man versuchen, direkt auf die Daten zuzugreifen, bevor diese tatsächlich verfügbar sind, könnte dies zu Fehlermeldungen oder leeren Anzeigen auf den Seiten führen. Durch den kurzen `setTimeout` wird dieses Problem umgangen, da die Verarbeitung der Daten erst nach einer minimalen Wartezeit erfolgt, in der sichergestellt wird, dass alle relevanten Informationen vom Backend eingetroffen sind.

Insbesondere bei komplexeren Datenstrukturen, wie z.B. verschachtelten Objekten oder Beziehungen zwischen verschiedenen Entitäten in der Datenbank, kann es vorkommen, dass einzelne Abfragen länger dauern als andere. Durch die Verwendung von Delays wird die Konsistenz der Daten auf allen Seiten gewährleistet und verhindert, dass Benutzer:innen auf eine Seite gelangen, bevor der gewünschte Inhalt tatsächlich verfügbar ist.

Home Seite:

Lebenslauf

Auf der CV-Seite werden zunächst die HTML-Elemente für die jeweiligen Datenboxen mittels `getElementById()` erfasst und gespeichert. Dieser Schritt ist wichtig, da die Boxen später im Frontend an den richtigen Spaltenpositionen platziert werden müssen.

Anschließend wird in der `setTimeout`-Funktion das `generalInfo`-Objekt aus dem gespeicherten Portfolio geladen. Die darin enthaltenen Daten werden dann Schritt für Schritt in die entsprechenden Felder des Formulars eingetragen. Dabei werden einige Werte direkt aus dem `generalInfo`-Objekt übernommen, während andere wie Vorname, Nachname, E-Mail-Adresse und Telefonnummer aus dem Profil des aktuell angemeldeten Profils (`loggedInUser`) stammen.

```
ngOnInit() : void { no usages  sStieg

    this.generalInfoBox = document.getElementById("generalInfoBox");
    this.educationsBox = document.getElementById("educationsBox");
    this.workExperienceBox = document.getElementById("workExperienceBox");

    setTimeout() : void => {
        let generalInfo: GeneralInfo = this.portfolioService.currPortfolio!.generalInfo;

        this.generalInfoForm.controls['id'].setValue(generalInfo!.id);
        this.generalInfoForm.controls['gender'].setValue(generalInfo!.gender);
        this.generalInfoForm.controls['firstName'].setValue(this.profileService.loggedInUser!.firstName);
        this.generalInfoForm.controls['lastName'].setValue(this.profileService.loggedInUser!.lastName);
        this.generalInfoForm.controls['email'].setValue(this.profileService.loggedInUser!.email);
        this.generalInfoForm.controls['phoneNumber'].setValue(this.profileService.loggedInUser!.phoneNumber);
        this.generalInfoForm.controls['zipCode'].setValue(generalInfo!.zipCode);
        this.generalInfoForm.controls['city'].setValue(generalInfo!.city);
        this.generalInfoForm.controls['address'].setValue(generalInfo!.address);
    }
}
```

Abbildung 90: Lebenslauf ngOnInit - Methode (1)

Im nächsten Schritt werden die im Portfolio gespeicherten `education`- und `workExperience`-Objekte sortiert und in die entsprechenden Bereiche eingefügt. Zuerst wird geprüft, ob die Daten zu `education` im `currPortfolio` vorhanden sind. Wenn ja, werden die Daten kopiert, nach dem `fromDate`-Wert sortiert und anschließend mit `addEducationInfo()` verarbeitet.

Dasselbe geschieht mit den `workExperience`-Daten: Falls vorhanden, werden diese kopiert, ebenfalls nach `fromDate` sortiert und dann mit der Methode `addJobExperiencesInfo()` verarbeitet. Schließlich wird `moveBoxes()` aufgerufen, um sicherzustellen, dass die Boxen richtig positioniert sind.

```
if(this.portfolioService.currPortfolio!.educations != undefined) {
    let educations: Education[] = this.portfolioService.currPortfolio!.educations.slice().sort((a: Education, b) => {
        return new Date(a.fromDate).getTime() - new Date(b.fromDate).getTime();
    });
    for (let currEducation of educations) {
        this.addEducationInfo(currEducation);
    }
}

if(this.portfolioService.currPortfolio!.workExperiences != undefined) {
    let workExperiences: WorkExperience[] = this.portfolioService.currPortfolio!.workExperiences.slice().sort((a: WorkExperience, b) => {
        return new Date(a.fromDate).getTime() - new Date(b.fromDate).getTime();
    });
    for (let currWorkExperience of workExperiences) {
        this.addJobExperiencesInfo(currWorkExperience);
    }
}

this.moveBoxes()

200
```

Abbildung 91: Lebenslauf ngOnInit - Methode (2)

Projekte

Auf der Projektseite übernimmt `ngOnInit` die Aufgabe, die gespeicherten GitHub-Repositories abzurufen und entsprechend anzuzeigen. Diese Repositories stammen ursprünglich von der GitHub API, werden aber nach dem ersten Abruf in der Datenbank gespeichert. Der Grund dafür liegt in der begrenzten Anzahl von API-Anfragen, die GitHub pro Stunde zulässt.

Würden die Repositories bei jeder Aktualisierung direkt über die API abgerufen, könnte es passieren, dass sie vorübergehend nicht mehr verfügbar sind, sobald das Abruflimit erreicht ist. Um dieses Problem zu umgehen und eine kontinuierliche Anzeige der Projekte zu gewährleisten, werden die Repositories nach dem ersten Abruf lokal gespeichert. Dadurch bleiben sie unabhängig von den Einschränkungen der API jederzeit abrufbar und verfügbar, auch wenn die API vorübergehend nicht erreichbar ist.

```
getRepos(user: string): void { Show usages ⚙️ sStieg v1 *
  var requestURL: string = 'https://api.github.com/users/' + user + '/repos';
  var request = $.get(requestURL, function () :void { })
  .done(() :void => {
    request = request.responseJSON;
    if (!Array.isArray(request) || !request.length) {
      $("#repo-box").html('<div class="error-box"><h1 class="error-m
    } else {
      let repos: GHRepo[] = [];
      request.forEach((repo: any, index: number) :void => {
        repos.push({
          id: 0,
          description: repo.description || "No Description",
          language: repo.language || "-",
          repoName: repo.name,
          url: repo.html_url,
          username: repo.owner.login,
          portfolio: this.portfolioService.currPortfolio!
        });
      });
      this.ghRepoService.addAllRepos(repos).subscribe()
      this.reloadPage();
    }
  });
}
```

Abbildung 92: Projekte - GitHub Repos laden und speichern

Kompetenzen

Auf der Kompetenzseite wird zu Beginn der Methode `ngOnInit` ein ähnlicher Ansatz wie auf der Lebenslaufseite verwendet. Zuerst werden die verschiedenen Boxen, in denen die Kompetenzen dargestellt werden sollen, erfasst und mittels `getElementById()` gespeichert. Dies dient dazu, die Elemente später korrekt in den vorgesehenen Spalten auf der Seite anzuordnen.

Im nächsten Schritt werden alle verfügbaren Eigenschaften, die der User auswählen kann, aus der Datenbank geholt. Diese Merkmale umfassen verschiedene Fähigkeiten

oder Eigenschaften, die später über Checkboxes ausgewählt werden können. Nach dem erfolgreichen Laden werden die Daten für die Checkbox-Elemente gespeichert, so dass der/die Benutzer/in sie auswählen kann.

```
ngOnInit(): void { no usages  ± sStieg +1
  this.characteristicsBox = document.getElementById("characteristics");
  this.knownLanguagesBox = document.getElementById("knownLanguages");
  this.programmingKnowledgesBox = document.getElementById("programmingKnowledges");
  this.softwareKnowledgesBox = document.getElementById("softwareKnowledges");

  this.characteristicService.loadAllCharacteristics().subscribe(c => {
    console.log("Characteristics loaded", c)
    this.characteristics = c
  });
}
```

Abbildung 93: Kompetenzen - ngOnInit (1)

Im nächsten Schritt wird geprüft, ob bereits bestimmte Merkmale im aktuellen Portfolio gespeichert sind. Dazu wird der bestehende Datenspeicher des Portfolios durchsucht, um festzustellen, ob dort bereits ausgewählte Merkmale gespeichert sind. Ist dies der Fall, werden diese gespeicherten Merkmale in das Array `selectedCharacteristics` eingefügt.

```
setTimeout(() => {
  this.selectedCharacteristic = [];
  if(this.portfolioService.currPortfolio!.characteristics !== undefined) {
    for (let currChar of this.portfolioService.currPortfolio!.characteristics) {
      this.selectedCharacteristic.push({
        id: currChar.id,
        label: currChar.label,
        value: currChar.value,
      })
    }
  }
}
```

Abbildung 94: Kompetenzen - ngOnInit (2)

Am Ende des Initialisierungsprozesses wird geprüft, ob bereits Daten zu bekannten Sprachen (`knownLanguages`), Programmierkenntnissen (`programmingKnowledges`) oder Softwarekenntnissen (`softwareKnowledges`) im Portfolio gespeichert sind.

Wenn solche Informationen vorhanden sind, werden sie der Reihe nach durchlaufen und in die entsprechenden Abschnitte der Seite eingefügt. Dazu wird jedes gespeicherte Element iteriert und mit Hilfe der entsprechenden Methoden (`addKnownLanguage`, `addProgrammingLanguage`, `addSoftware`) in die dafür vorgesehenen Boxen eingefügt.

Nachdem alle relevanten Kenntnisse geladen und dargestellt wurden, wird die Funktion `moveBoxes` aufgerufen. Diese sorgt für die korrekte Anordnung der einzelnen Boxen auf der Seite.

```

let knownLanguages = this.portfolioService.currPortfolio!.knownLanguages;
if(knownLanguages != undefined) {
  for (let currKnownLanguage of knownLanguages) {
    this.addKnownLanguage(currKnownLanguage)
  }
}

let programmingKnowledges = this.portfolioService.currPortfolio!.programmingKnowledges
if(programmingKnowledges != undefined) {
  for (let currProgrammingKnowledge of programmingKnowledges) {
    this.addProgrammingLanguage(currProgrammingKnowledge)
  }
}

let softwareKnowledges = this.portfolioService.currPortfolio!.softwareKnowledges
if(softwareKnowledges != undefined) {
  for (let currSoftwareKnowledge of softwareKnowledges) {
    this.addSoftware(currSoftwareKnowledge)
  }
}

this.moveBoxes();
200)

```

Abbildung 95: Kompetenzen - ngOnInit (3)

Gruppen Seite

Auf der Gruppenseite findet in der Methode `ngOnInit` nur eine minimale Verarbeitung statt. Hier wird lediglich die Gruppenliste des aktuell angemeldeten Profils aus dem entsprechenden Dienst geladen und gespeichert. Anschließend werden die benötigten Daten direkt im HTML-Template verwendet, so dass keine weitere Verarbeitung oder Manipulation in `ngOnInit` notwendig ist. Diese Vorgehensweise stellt sicher, dass die Anzeige der Gruppen dynamisch und effizient erfolgt. Die Gruppendaten werden direkt aus der gespeicherten Liste ausgelesen und ohne weitere Zwischenschritte an den entsprechenden Stellen in der Benutzeroberfläche angezeigt.

```

ngOnInit(): void { no usages  sStieg
  this.groupsService.getAllGroups().subscribe(g : Group[] => {
    this.groups = g;
  })
}

```

Abbildung 96: Gruppen - ngOnInit

```

<div class="dropdown-list min-h-64">
  <div *ngFor="let group of filteredGroups" class="grid grid-cols-5 items-center text-gray-500 p-4">
    <h1 class="min-w-fit flex justify-start">{{ group.name }}</h1>
    <h1 class="text-center">{{ group.company }}</h1>
    <h1 class="text-center">{{ group.department }}</h1>
    <h1 class="text-center">{{ group.members.length | 0 }} / {{group.maxMembers}}</h1>
    <div class="flex justify-center">
      @if(isMemberOfGroup(group)) {
        <button routerLink="/groups/{{group.id}}" class="bg-white text-primary-color px-2 py-1 rounded">
        } @else {
        <button (click)="showPasswordInputBox(loggig: true, group)" class="bg-primary-color text-white">
        }
      }
    </div>
  </div>

```

Abbildung 97: Gruppen - HTML

Settings Seite

Auf der Seite Einstellungen gibt es nur wenige Elemente, die angezeigt werden können, da der Schwerpunkt hier auf dem Speichern der Portfolio Einstellungen liegt. Die

einzigsten sichtbaren Informationen sind bereits gespeicherte Einstellungen aus dem Portfolio, nämlich die Sprache und die Hintergrundfarbe. Beide Einstellungen werden in separaten Unterkomponenten verwaltet, in denen die gespeicherten Werte direkt aus dem Portfolio geladen und angezeigt werden. Die Seite verfügt über einen zentralen „Speichern“-Button, mit dem alle Änderungen, die der/die Benutzer/in vornimmt, gleichzeitig gespeichert werden.

```
confirmSettings() : void { Show usages sStieg +1
  let portfolio : Portfolio = this.portfolioService.currPortfolio!;
  if (this.selectedBackgroundImageUrl != null) {

  }

  if (this.selectedBackgroundColor != '') {
    portfolio.color = this.selectedBackgroundColor;
  }

  portfolio.languageCode = this.selectedLanguage;
  console.log(portfolio);
  this.portfolioService.updatePortfolio(portfolio).subscribe();

  this.reloadPage()
}
```

Abbildung 98: Settings - Speichern

Portfolio View Seite

Die Portfolio-View-Seite dient als zentrale Übersicht, in der alle relevanten Informationen aus den verschiedenen Startseiten zusammengeführt werden. Ziel ist es, dem User eine vollständige und übersichtliche Darstellung seines gesamten Portfolios auf einer einzigen Seite zu bieten.

Zu Beginn des Ladevorgangs werden alle Datenboxen mit `getElementById()` erfasst und gespeichert. Dies stellt sicher, dass sie später an den richtigen Stellen auf der Seite platziert werden können, um eine strukturierte und ansprechende Darstellung zu gewährleisten.

Ein weiterer wichtiger Schritt ist die Überprüfung, ob bereits gespeicherte GitHub-Repository-Daten im Portfolio vorhanden sind. Ist dies der Fall, werden diese direkt auf der Seite eingefügt, so dass sie sichtbar sind.

Anschließend wird über ein `setTimeout` sichergestellt, dass alle weiteren relevanten Daten aus den anderen Bereichen des Portfolios korrekt verarbeitet werden. Insbesondere wird

hier geprüft, welche Objekte bereits auf der Lebenslaufseite und der Kompetenzseite vorhanden sind. Wenn solche Daten im Portfolio gespeichert sind, werden sie geladen und an den entsprechenden Stellen auf der Portfolio-Ansichtsseite angezeigt.

Durch diesen strukturierten Ablauf wird sichergestellt, dass alle Informationen des Nutzers, sei es die Ausbildung, die Berufserfahrung, die technischen Fähigkeiten oder die GitHub-Projekte, vollständig und übersichtlich auf einer Seite dargestellt werden.

```
ngOnInit() : void {
  no usages  ± sStieg

  this.generalInfoBox = document.getElementById("generalInfoBox");
  this.educationsBox = document.getElementById("educationsBox");
  this.workExperiencesBox = document.getElementById("workExperiencesBox");
  this.characteristicsBox = document.getElementById("characteristicsBox");
  this.knownLanguagesBox = document.getElementById("knownLanguagesBox");
  this.programmingKnowledgesBox = document.getElementById("programmingKnowledgesBox");
  this.softwareKnowledgesBox = document.getElementById("softwareKnowledgesBox");

  this.portfolioService.getPortfolioById(id).subscribe(p : Portfolio => {
    this.portfolio = p;
    console.log(this.portfolio);
    if(this.portfolioService.currPortfolio!.ghRepos !== undefined && this.portfolio.ghRepos) {
      this.placeRepos();
    }
  });

  setTimeout(() : void =>{
    if (this.portfolio.generalInfoPosition) {
      this.moveToColumn(this.generalInfoBox, this.portfolio.generalInfoPosition);
    }
  }, 200)
}
```

Abbildung 99: Portfolio View - ngOnInit (1)

```
if (this.portfolio.educationsPosition) {
  this.moveToColumn(this.educationsBox, this.portfolio.educationsPosition);
}

if (this.portfolio.workExperiencesPosition) {
  this.moveToColumn(this.workExperiencesBox, this.portfolio.workExperiencesPosition);
}

if(this.portfolio.characteristicsPosition) {
  this.moveToColumn(this.characteristicsBox, this.portfolio.characteristicsPosition);
}

if(this.portfolio.knownLanguagesPosition) {
  this.moveToColumn(this.knownLanguagesBox, this.portfolio.knownLanguagesPosition);
}

if(this.portfolio.programmingKnowledgesPosition) {
  this.moveToColumn(this.programmingKnowledgesBox, this.portfolio.programmingKnowledgesPosition);
}

if(this.portfolio.softwareKnowledgesPosition) {
  this.moveToColumn(this.softwareKnowledgesBox, this.portfolio.softwareKnowledgesPosition);
}

this.backgroundColor = this.portfolio.color;
}, 200)
```

Abbildung 100: Portfolio View - ngOnInit (2)

6 Zusammenfassung

Melius ist eine innovative Plattform zur Erstellung und Verwaltung von digitalen Portfolios. Ziel des Projektes ist es, den Nutzer:innen eine einfache Möglichkeit zu bieten, ihre beruflichen Fähigkeiten, Erfahrungen und Projekte ansprechend und übersichtlich zu präsentieren.

Die Plattform besteht aus einem modernen, mit Angular entwickelten Frontend und einem leistungsfähigen, auf Java basierenden Backend mit dem Quarkus Framework. Für die Datenspeicherung wird eine PostgreSQL-Datenbank verwendet, die in einem Docker-Container läuft.

Die Benutzeroberfläche bietet eine zentrale Portfolioansicht, in der alle relevanten Informationen wie Lebenslauf, Kompetenzen und Projekte auf einer Seite zusammengefasst sind. Über Einstellungen können Sprache und Hintergrundfarbe angepasst werden, um das Portfolio individuell zu gestalten.

6.0.1 Features für die Zukunft

Als Erweiterung des bereits implementierten Funktionsumfangs wurden Überlegungen zu folgenden Funktionen angestellt:

- Die Möglichkeit, Standardvorlagen im Portfolio zu verwenden, die bereits ein Hintergrundbild, eine bestimmte Anordnung der Boxen oder ähnliches enthalten. Dadurch erhalten die Nutzer:innen eine Starthilfe und Inspiration beim Erstellen des Portfolios.
- Die Möglichkeit, dem Portfolio konfigurierbare Formulare hinzuzufügen, bei denen die Überschrift und die Art der Eingabe festgelegt werden können. Dies soll zusätzliche Möglichkeiten bieten, die eigenen Stärken individuell darzustellen. Die verfügbaren Eingabearten entsprechen den bereits im Portfolio vorhandenen, wie z.B. Text, Slider oder Sternebewertung.
- Für die Gruppenfunktion eine Erweiterung, die es Gruppenleiter:innen ermöglicht, in der Gruppenansicht Notizen an die Mitarbeitenden anzuhängen. Dies kann beispielsweise für Feedback oder Anmerkungen genutzt werden.

Abbildungsverzeichnis

List of Listings

1	Einbindung eines benutzerdefinierten Validators	53
2	Klassen die von den Spalten benutzt werden	54